UNSW SYDNEY

Enhancing Static Analysis through API Specification Generation

Author:
Kan, Shuangxiang

Publication Date:
2025

DOI:
https://doi.org/10.26190/unsworks/31653

License:
https://creativecommons.org/licenses/by/4.0/
Link to license to see what you are allowed to do with this resource.

Downloaded from http://hdl.handle.net/1959.4/106025 in https://unsworks.unsw.edu.au on 2026-04-20

Enhancing Static Analysis through API Specification Generation

by

Shuangxiang Kan

A THESIS IN FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

October 3, 2025

School of Computer Science and Engineering
Faculty of Engineering
The University of New South Wales

All rights reserved. This work may not be reproduced in any form without the permission of the author. © Shuangxiang Kan 2025

Thesis submission for the degree of Doctor of Philosophy

Thesis Title and Abstract | **Declarations** | Inclusion of Publications Statement | Corrected Thesis and Responses

ORIGINALITY STATEMENT

☑ I hereby declare that this submission is my own work and to the best of my knowledge it contains no materials previously published or written by another person, or substantial proportions of material which have been accepted for the award of any other degree or diploma at UNSW or any other educational institution, except where due acknowledgement is made in the thesis. Any contribution made to the research by others, with whom I have worked at UNSW or elsewhere, is explicitly acknowledged in the thesis. I also declare that the intellectual content of this thesis is the product of my own work, except to the extent that assistance from others in the project's design and conception or in style, presentation and linguistic expression is acknowledged.

COPYRIGHT STATEMENT

☑ I hereby grant UNSW or its agents a non-exclusive licence to archive and to make available (including to members of the public) my thesis or dissertation in whole or part in the University libraries in all forms of media, now or here after known. I acknowledge that I retain all intellectual property rights which subsist in my thesis or dissertation, such as copyright and patent rights, subject to applicable law. I also retain the right to use all or part of my thesis or dissertation in future works (such as articles or books).

For any substantial portions of copyright material used in this thesis, written permission for use has been obtained.

AUTHENTICITY STATEMENT

☑ I certify that the Library deposit digital copy is a direct equivalent of the final officially approved version of my thesis.

Thesis submission for the degree of Doctor of Philosophy

Thesis Title and Abstract | Declarations | **Inclusion of Publications Statement** | Corrected Thesis and Responses

Candidates including publications in their thesis are instructed to select either Category 2 or Category 3. If the thesis is a combination of Category 2 and 3, they must select Category 3 and provide the details.

Here is a summary of the 3 available categories:

Category 1	The candidate's thesis contains no publications, either published or submitted for publication.
Category 2	Some of the work described in the candidate's thesis has been published, but they have not included the entire publication(s) in place of any chapter(s). Instead, the work has been rewritten/edited/expanded to incorporate it into the thesis, and this has been clearly acknowledged within the thesis.
Category 3	The candidate's thesis contains one or more publications that have been incorporated **in place of** a chapter/s (whether published or submitted for publication). All the following statements are true for each publication they are using in this way: • The research was completed **during the candidate's current HDR candidature**. • The candidate contributed **more than 50%** to the publication. • The candidate is listed as the **primary author**. • The publication is **not restricted** by any contracts or third-party agreements. • The chapter is included **word-for-word** as it appears in the publication (e.g. the **author-accepted version**).

☑ **Category 3:** The candidate has declared that **their thesis contains one or more publications that have been incorporated *in place of* a chapter/s (whether published or submitted for publication). All the following statements are true for each publication they are using in this way:**.

- The research was completed **during their current HDR candidature**.
- They contributed **more than 50%** to the publication.
- They are listed as the **primary author**.
- The publication is **not restricted** by any contracts or third-party agreements.
- The chapter is included **word-for-word** as it appears in the publication (e.g. the **author-accepted version**).

Publication Details #1

Full Title:	Spectre: Automated Aliasing Specifications Generation for Library APIs with Fuzzing.
Authors:	Shuangxiang Kan, Yuekang Li, Weigang He, Zhenchang Xing, Liming Zhu, and Yulei Sui.
Journal or Book Name:	ACM Transactions on Software Engineering and Methodology
Volume/Page Numbers:	N/A
Date Accepted/Published:	2025
Status:	accepted
The Candidate's Contribution to the Work:	I am the first author of this publication. I was responsible for the design and execution of all experiments, data analysis, and drafting the majority of the manuscript. My contribution exceeds 50% of the overall work.
Location of the work in the thesis and/or how the work is incorporated in the thesis:	This publication is incorporated as Chapter 3 of the thesis. Minor edits have been made to ensure consistency in formatting and referencing across the thesis.

Publication Details #2

Full Title:	Cross-Language Taint Analysis: Generating Caller-Sensitive Native Code Specification for Java
Authors:	Shuangxiang Kan, Yuhao Gao, Zexin Zhong, and Yulei Sui
Journal or Book Name:	IEEE Transactions on Software Engineering
Volume/Page Numbers:	vol. 50, no. 6, pp. 1518-1533
Date Accepted/Published:	2024
Status:	published
The Candidate's Contribution to the Work:	I am the first author of this publication. I was responsible for the design and execution of all experiments, data analysis, and drafting the majority of the manuscript. My contribution exceeds 50% of the overall work.
Location of the work in the thesis and/or how the work is incorporated in the thesis:	This publication is incorporated as Chapter 4 of the thesis. Minor edits have been made to ensure consistency in formatting and referencing across the thesis.

Publication Details #3

Full Title:	Specguru: Hierarchical LLM-Driven API Points-to Specification Generation with Self-Validation
Authors:	Shuangxiang Kan, Yuekang Li, Xiao Cheng, Yulei Sui.
Journal or Book Name:	48st International Conference on Software Engineering
Volume/Page Numbers:	N/A
Date Accepted/Published:	
Status:	submitted
The Candidate's Contribution to the Work:	I am the first author of this publication. I was responsible for the design and execution of all experiments, data analysis, and drafting the majority of the manuscript. My contribution exceeds 50% of the overall work.
Location of the work in the thesis and/or how the work is incorporated in the thesis:	This publication is incorporated as Chapter 5 of the thesis. Minor edits have been made to ensure consistency in formatting and referencing across the thesis.

Candidate's Declaration

I declare that I have obtained written permission for the reproduction of all third-party material included in this thesis. I confirm that where I have used a publication in place of a chapter, the listed publication(s) above meet(s) the requirements to be included in the thesis as per the Higher Degree Research Examination Procedure.

Abstract

Static analysis faces significant challenges when analyzing programs that rely heavily on external APIs. These API calls often disrupt the continuity of data and control flow, making it difficult to reason accurately about their effects. As a result, analyzing behaviors that span API boundaries becomes particularly challenging. API specification generation offers an effective solution to this problem. By enabling precise modeling of third-party library behaviors within static analysis frameworks, it enhances the overall precision and effectiveness of static analysis.

Despite its potential, existing methods for API specification generation face challenges in precision, automation, and scalability when applied to real-world software. To address these limitations, this thesis makes three key contributions targeting C/C++ library APIs: First, we present Spectre, a fuzzing-based approach that automatically infers aliasing specifications for C libraries without source code. Second, we develop CSS, a caller-sensitive specification technique that enhances cross-language taint analysis precision for Java-native (JNI) programs by modeling C/C++ library behaviors. Third, we propose Specguru, an LLM-powered framework that hierarchically infers and validates points-to specifications for C/C++ libraries, generating more correct summaries than existing methods. Together, these advances enable more accurate API-related static analysis results while maintaining practical applicability.

Publications

Thesis-related publications:

- *Spectre: Automated Aliasing Specifications Generation for Library APIs with Fuzzing.*

  Shuangxiang Kan, Yuekang Li, Weigang He, Zhenchang Xing, Liming Zhu, and Yulei Sui.

  In ACM Transactions on Software Engineering and Methodology, 2025 (**Relates to Chapter 3**).

- *Cross-Language Taint Analysis: Generating Caller-Sensitive Native Code Specification for Java.*

  Shuangxiang Kan, Yuhao Gao, Zexin Zhong, and Yulei Sui.

  In IEEE Transactions on Software Engineering, vol. 50, no. 6, pp. 1518-1533 (**Relates to Chapter 4**).

- *Specguru: Hierarchical LLM-Driven API Points-to Specification Generation with Self-Validation.*

  Shuangxiang Kan, Yuekang Li, Xiao Cheng, Yulei Sui.

  In 48th International Conference on Software Engineering, Major Revision (**Relates to Chapter 5**).

Other publications:

- *Interactive Cross-Language Pointer Analysis for Resolving Native Code in Java Programs.*

  Chenxi Zhang, Yufei Liang, Tian Tan, Chang Xu, **Shuangxiang Kan**, Yulei Sui, and Yue Li.

  In 47th International Conference on Software Engineering.

Acknowledgments

Throughout the demanding journey of my doctoral research, I have been exceptionally fortunate to receive assistance, encouragement, and guidance from numerous individuals. I extend my deepest gratitude to each one of them.

I would first like to express my heartfelt appreciation to my supervisors, Prof. Yulei Sui, Yuekang Li, and Jingling Xue. Their invaluable guidance and unwavering support over the years have been instrumental in the completion of this thesis. They not only taught me to identify and solve problems, but also guided me in presenting ideas, writing research papers, and fostering collaborations. Their patience, dedication, and commitment have transformed me from an average student into a competent PhD holder, a change that will continue to shape my professional trajectory.

I am also deeply grateful to the members of my PhD qualification examination committee, Prof. David Lie from University of Toronto and Prof. Xiaoning Du from the Monash University, for agreeing to examine my thesis and provide insightful feedback and constructive criticism to help me strive for excellence in my future research endeavors.

I would like to acknowledge the invaluable contributions of my friends and colleagues at The University of New South Wales (UNSW). Their support has had a profound impact on me, both academically and personally. In no particular order,

I wish to thank Yuxiang Lei, Xiao Cheng, Guanqin Zhang, Pei Xu, Yuhao Gao, Feng Xu, Jiawei Wang, Jiawei Ren, Weigang He, Ruijun Feng, Kaijie Liu, Zexin Zhong, Jiahao Zhang, Jiawei Deng, Penghao Jiang, Ziqi Ding, Zihao Xu, Junchen Ding, Mingxiu Wang, and Xudong Wang. They have been my family at UNSW during this journey.

Finally, my profoundest gratitude is reserved for my family. Their unwavering support and unconditional love have been my beacon of inspiration and strength, pulling me out of moments of despair and enabling me to complete my PhD studies.

Contents

Abstract **i**

Publications **ii**

Acknowledgments **iv**

List of Figures **xi**

List of Tables **xiii**

List of Algorithms **xiv**

1 Introduction **1**

1.1 Challenges in Automated API Specification Generation 5

1.2 Contributions 6

1.3 Thesis Organisation 9

2 Preliminaries **12**

2.1 Specification Generation 12

2.1.1 Alias Analysis and Specifications 13

2.1.2 Taint Specifications 14

2.1.3 Points-to Specifications and Properties 15

2.2 Cross-Language Programming with JNI 17
2.3 Fuzzing 19
2.4 Large Language Model for Code-related Tasks 21

3 Fuzzing-Based API Aliasing Specification Generation 23

3.1 Overview 23
3.1.1 Motivation 24
3.1.2 Challenges 25
3.1.3 Our Solution 26
3.1.4 Framework Overview 27
3.2 Motivating Example 29
3.3 Approach 33
3.3.1 Alias-Check Observer 33
3.3.2 New Fuzzing Features 41
3.4 Evaluation 48
3.4.1 Experiment setup 49
3.4.2 Generation Capability 55
3.4.3 Usefulness 58
3.4.4 Ablation Study 60
3.4.5 Comparative Analysis 62
3.5 Related Work 64
3.5.1 Specification Inference 65
3.5.2 Fuzzing 66
3.6 Chapter Summary 66

4 Cross-Language Native Code Specification Generation 68

4.1 Overview 68

4.1.1 Motivation 69
4.1.2 Challenges 69
4.1.3 Our Solution 72
4.1.4 Framework Overview 74
4.2 A Motivating Example 75
4.3 Approach 79
4.3.1 Caller Specification Extraction on Java Side 80
4.3.2 Callee's Value-flow Specification Extraction on C/C++ Side 82
4.4 Evaluation 89
4.4.1 Results on *NativeFlowBench* benchmark 91
4.4.2 Results on *JavaNativeBench* benchmark 94
4.4.3 Results on real-world JNI programs 100
4.5 Related Work 104
4.6 Chapter Summary 106

5 LLM-Driven Hierarchical API Specification Inference 107
5.1 Overview 107
5.1.1 Motivation 108
5.1.2 Challenges 110
5.1.3 Our Solution 110
5.1.4 Framework Overview 112
5.2 A Motivating Example 114
5.3 Approach 117
5.3.1 Preprocessing 117
5.3.2 Bottom-Up Specification Inference 118
5.3.3 Specification Self-Validation 121
5.4 Evaluation 125

5.4.1 Experiment Setup 125
5.4.2 RQ1: Comparison with SOTA 129
5.4.3 RQ2: Bottom-Up Analysis 132
5.4.4 RQ3: Specification Validation 134
5.4.5 RQ4: Practical Usefulness 135
5.5 Related Work 138
5.6 Threats to validity 139
5.7 Chapter Summary 140

6 Conclusions and Future Work **141**
6.1 Conclusions 141
6.2 Future Work 142

Bibliography **144**

List of Figures

1.1 Overview of the challenges and how each contribution addresses them. 10

2.1 JNI interoperation 17

3.1 The design overview of SPECTRE. Grey rectangles are unique features of SPECTRE 28

3.2 Client code with aliasing specification of library APIs 30

3.3 Aliasing specification inference for lib_getUserValue() 30

3.4 Three test cases execution traces of the lib_extractUserInfo() 30

3.5 Distribution of correct aliasing specifications in the manual and SPECTRE-*Base* methods (Venn diagram) on Musl-1.2.4. 56

3.6 Comparison of specifications generated by *c-summary* and SPECTRE-*Full*. 63

4.1 A data leakage JNI example (simplified) 71

4.2 The workflow of our cross-language taint analysis with caller-sensitive specifications 74

4.3 An illustrating JNI example with Java and native code 76

4.4 Three-tuple caller specification 80

4.5 Caller's specification extraction 82

4.6 A fragment of LLVM IR and its memory SSA form of the motivating example in Figure 4.3 85
4.7 Four-tuples callee specification 88
4.8 Callee's specification extraction 88
4.9 Code in *Java2JavaFieldNoAlias* 95
4.10 *base64encode* example 103

5.1 SPECGURU framework overview. 113
5.2 A motivating example illustrating how SPECGURU performs bottom-up points-to specification inference and validation for the library API `cJSON_addItemToObject` extracted from the `cJSON`. 113
5.3 Leaf and Non-leaf function specification inference 119
5.4 Specification candidates S'_{f_1}, S'_{f_2}, S'_{f_3}, and S'_{f_4} pass the differential test (inliers), candidate S'_{f_5} fails (outlier). 124
5.5 Comparison of four strategies in terms of precision, recall, and F1 Score. 134
5.6 Static pointer alias analysis (upper) and taint analysis (lower). . . . 136

List of Tables

2.1 The syntax and specification examples of aliasing specifications . . 17

3.1 Eight header files in Musl-1.2.4 49

3.2 Eight selected C third-party libraries 50

3.3 Comparison of Manual and SPECTRE Methods on Musl-1.2.4. *#Alias-Specs* are the numbers of manually-written or SPECTRE-Base-generated aliasing specifications; 54

3.4 Specification generation on eight third-party libraries. *#Specs* are the total numbers of SPECTRE-*Base*-generated specifications . . . 54

3.5 Comparison of static pointer analysis performance using four different approaches for handling library APIs. 58

3.6 Performance of SPECTRE-Base, SPECTRE-Cov, and SPECTRE-Full on Musl-1.2.4 and third-party libraries. 60

4.1 Results of *NativeFlowBench* benchmarks. FD, JS, JC, FD-CSS denotes the result of FlowDroid, JN-SAF, Jucify and FlowDroid-CSS. 92

4.2 Results (R) and Time(s) (T) of *JavaNativeBench* benchmarks . . . 94

4.3 Comparison of quality, taint analysis results and runtime between SemanticSpec and CSS 98

4.4 Value-flow analysis results of three static analyzers JN-SAF, JuCify and, FlowDroid-CSS on ten Android JNI applications. The metrics include the total number of lines of code in thousands (Loc), the percentage of Java code (Java), the percentage of C/C++ code (C/C++), the number of true positive results (Recall) out of the total true positive results (Total) in each application, and the precision of the tools (R/T, Recall-to-Total). The table also reports the time taken in Java and C/C++ separately in seconds (Time) for each tool. 102

5.1 Statistics of 15 C third-party libraries. #APIs is the number of function definitions in the library, #LoC is the lines of code, #TSpec is the number of verified specifications, #Stars is the github/gitlab stars. 126

5.2 The number of specifications generated by Spectre, c-summary, and SPECGURU across libraries. Numbers in parentheses indicate unique specifications not found by the other two methods. 127

5.3 Comparison of four strategies for specification inference: SPECGURU-IA/CCA/BUA, and SPECGURU. **#Diff** is the average number of test cases for each validated specification. **#Tokens** is the average number of tokens in the prompt for generating specifications for each API function. 132

List of Algorithms

1 Alias-Check observer Generation 40

2 Params-Ret operations Instrumentation 43

Chapter 1

Introduction

Static program analysis [93] has emerged as a cornerstone of program verification and software quality assurance, enabling the examination of code without execution to support applications such as vulnerability detection [28,29,94,113,114], program optimization [20,41,131], program slicing [48,140]. By analyzing the structure and behavior of a program, static analysis tools provide valuable insights into code correctness and reliability, making them indispensable in modern software development. However, despite significant advancements, static analysis faces challenges when dealing with complex program behaviors, particularly those involving interactions with external Application Programming Interfaces (APIs). These challenges form the central focus of this dissertation.

Modern software development heavily relies on libraries and frameworks that expose their functionality through APIs. While these APIs accelerate development by providing pre-built functionality, they simultaneously introduce complexity that impedes comprehensive static analysis. When a program invokes APIs, the analysis must account for the behavior of code that exists outside the analyzed program's boundaries. Several factors contribute to the difficulty of analyzing external API

calls. First, source code for these APIs is often unavailable, especially for proprietary libraries or closed-source frameworks, making it impossible to directly analyze their implementation. Second, the behavior of APIs may be complex, context-dependent, or rely on runtime conditions that are difficult to model statically. Third, many APIs implement cross-language functionality through mechanisms like Foreign Function Interfaces (FFIs) [58, 98, 103, 129], creating boundaries that single-language static analyzers struggle to cross effectively. Fourth, repeatedly analyzing these libraries for each client application incurs computational overhead, often consuming more analysis time than the client code itself, despite the fact that the core functionality of these APIs rarely changes.

To address the challenges posed by library API calls, current static analyzers typically adopt one of four strategies, each balancing precision, soundness, and scalability differently:

Whole-Code Analysis: This approach [9, 128] involves analyzing the source code of all invoked APIs together with the client code to enable precise modeling of API behavior. However, as noted earlier, this approach is often impractical due to unavailable source code, high computational costs of analyzing large codebases, and difficulties in handling cross-language interactions.

Optimistic Assumptions: The optimistic approaches [16, 18, 148] assumes API calls have minimal impact on the client code, treating arguments as unmodified and return values as fresh, untainted data. While this simplifies and accelerates static analysis, it risks false negatives by overlooking side effects, such as tainted data propagation or pointer aliasing, thus compromising soundness.

Pessimistic Assumptions: In contrast, pessimistic approaches [16, 18, 148] ensure soundness by over-approximating the effects of API calls, assuming that arguments and return values may be extensively modified or tainted. While this reduces

false negatives, these assumptions are often so imprecise that they introduce excessive false positives, rendering the static analysis results unusable.

Specifications: Specifications [18, 99] (also known as stubs or models) offer a balanced solution by summarizing API behaviors (e.g., data flows, aliasing, or side effects) without requiring full implementation analysis. In practice, most static analyzers rely on manually crafted specifications [8,120,128], which are labor-intensive and difficult to scale for large or evolving library APIs.

Compared to other approaches for handling external API calls in static analysis, specification-based methods provide three key advantages: (1) they are lightweight, avoiding the need to analyze entire library implementations; (2) they are flexible, allowing customization of the abstraction level based on analysis needs; and (3) they enable cross-language analysis by focusing on interface behaviors rather than different language implementation details. These characteristics make specification-based methods suitable for handling API calls in static analysis.

However, the effectiveness of this specification-based approach critically depends on the quality of the specifications. The primary challenge is how to create accurate, comprehensive specifications without prohibitive manual effort. Current practice often requires human experts to manually craft API specifications that summarize the effects of library APIs on program state, such as memory operations, pointer relations, and side effects [8, 44, 120]. While this approach allows for precise and customized modeling tailored to specific analyses, it is time-consuming, error-prone, and demands deep understanding of API semantics and behaviors. Moreover, maintaining these specifications across evolving libraries and large-scale codebases is labor-intensive and difficult to scale.

Another direction is the automatic inference of specifications through static analysis of the API implementation. This approach can reduce manual effort and

improve coverage when source code is available. However, static inference becomes infeasible when the source code is completely unavailable [1,16,148]—as is often the case for proprietary libraries or closed-source systems where only binary interfaces are accessible. Additionally, traditional static analysis often struggles with complex language features or highly optimized code, which can complicate the inference process [84]. Furthermore, generating accurate API specifications often requires specialized design, which can be time-consuming and labor-intensive.

Dynamic analysis provides an alternative to static methods by observing actual API executions and mining runtime behaviors to infer specifications. In particular, prior work on specification mining via dynamic analysis [18, 33] does not require source code and can uncover behavioral patterns. However, capturing and generating aliasing specifications during dynamic API execution remains challenging. Additionally, traditional dynamic analysis [17,33,92] for specification generation often requires well-designed test cases to maximize coverage of all possible execution paths, relying heavily on the quality and completeness of execution traces. Without comprehensive input coverage or carefully crafted test suites, dynamic methods may fail to capture critical behaviors and generate incomplete specifications.

Therefore, while specifications present an attractive solution for static analysis of API-rich software, the crucial question becomes: how can we automatically generate accurate, comprehensive specifications with minimal manual intervention? This thesis focuses on *automated specification generation* for C/C++ library APIs. By automating specification generation, our approach achieves improvements in both correctness (through more accurate behavior modeling) and coverage (by discovering more specifications compared to other methods). This advancement makes static analysis practical for real-world software that depends on external APIs, while eliminating the maintenance burden associated with manual specifi-

cation writing. The techniques developed in this thesis not only enhance static analysis capabilities but also open new possibilities for analyzing complex, API-dependent systems that were previously intractable.

1.1 Challenges in Automated API Specification Generation

Despite the importance of specification-based approaches for addressing external API challenges in static program analysis, their automated inference remains challenging in real-world scenarios. Below, we outline the key challenges that motivate this thesis.

(1) Incomplete Inference Due to Lack of Source Code. The generation of API specifications is fundamentally constrained when source code is inaccessible — a common scenario with entirely closed-source third-party libraries or proprietary APIs. Current static specification generation techniques, which typically require implementation details to model behaviors precisely, become inapplicable without source access. Traditional dynamic specification inference methods face inherent coverage limitations, as they can only observe behaviors triggered by specific executions, leaving critical edge cases or internal invariants unverified. This gap underscores the need for novel approaches capable of deriving comprehensive specifications despite source code unavailability.

(2) Semantic Discontinuities in Cross-Language API Boundaries. Modern software systems increasingly combine components written in different programming languages, creating fundamental analysis challenges. Each language maintains its own distinct memory model, type system, and execution semantics that often conflict at API boundaries. These semantic mismatches create analy-

sis blind spots where critical behaviors - including memory aliasing, data flows, and type safety properties - cannot be consistently tracked across language barriers. Current static analyzers, typically designed for single-language contexts, struggle to maintain analysis precision when these language domains interact, as they lack unified models to represent and reason about the composite semantics. This fundamental limitation prevents accurate whole-program analysis for modern multi-language systems.

(3) Complexity of API Semantics and Scalability Trade-offs. C/C++ libraries are often highly optimized, incorporating low-level operations to maximize efficiency. The complexity is exacerbated by incomplete access to the full implementation stack, which is distinct from the complete lack of source code described in Challenge (1). This often occurs when APIs rely on closed-source components, standard library functions without available implementations (e.g., memcpy, strcpy), or platform-specific native code. Repeatedly analyzing these library APIs is time-consuming and resource-intensive. Existing methods for generating specifications often require designing complex, handcrafted rules, which are labor-intensive and difficult to scale across diverse or evolving codebases. This underscores the need for automated, scalable techniques that can effectively handle API complexity and implementation gaps while maintaining precision and minimizing overhead.

1.2 Contributions

Our research advances the field of static program analysis by addressing critical challenges in automated C/C++ API specification generation. This thesis makes three major contributions to improve the generation and application of API specifications for cross-language and library API analysis:

(1) *Generating Aliasing Specifications via Fuzzing without Source Code.* We present SPECTRE, an automated framework that utilizes fuzzing techniques to infer aliasing specifications for C library APIs without complete source code access.

(2) *Enhancing Cross-Language Analysis with Caller-Sensitive Taint Specifications.* We propose CSS (Caller-Sensitive Specification), which generates taint specifications by incorporating cross-language calling contexts. Specifically, CSS captures how Java callers interact with native code (C/C++) in Java Native Interface (JNI) programs, enabling the generation of more precise and compact specifications that support accurate cross-language taint analysis.

(3) *Automating Points-to Specification Generation with Hierarchical LLMs.* We introduce SPECGURU, a bottom-up approach that employs Large Language Models (LLMs) to infer points-to specifications for third-party C library APIs, incorporating self-validation to ensure correctness and scalability.

Next, we elaborate on the three contributions outlined above and Figure 1.1 provides an overview of the key challenges addressed in this thesis and their corresponding contributions:

(1) Generating Aliasing Specifications via Fuzzing without Source Code

We introduce SPECTRE, a novel framework for automated aliasing specification generation that adapts to varying levels of source code availability. For APIs without source code access, the framework integrates lightweight alias-check observers code into API driver programs, systematically exploring parameter-return value relationships through input generation. When source code or partial source code is available, SPECTRE employs specialized grey-box fuzzing features including: (i) selective instrumentation of pointer operations, (ii) alias-aware seed scheduling, and (iii) guided mutation strategies optimized for aliasing relationship discovery

among parameters and return values. The framework automatically synthesizes specifications by detecting aliasing patterns through systematic fuzzing exploration. Memory aliasing occurs when multiple pointers refer to the same memory location, which can complicate static and dynamic analysis of programs. By generating precise aliasing specifications, SPECTRE helps capture these relationships, providing the necessary foundation for accurate pointer and API-related analyses. This contribution addresses **Challenge (1)** by enabling specification inference even in the absence of source code, overcoming the limitations of static analysis and enhancing precision beyond traditional dynamic inference methods. The evaluation shows that SPECTRE significantly improves the precision of downstream pointer analysis compared to existing methods.

(2) Enhancing Cross-Language Analysis with Caller-Sensitive Taint Specifications. We present CSS, a new cross-language framework that enables interaction between Java and C/C++ through three key innovations: First, it establishes a caller-sensitive bidirectional analysis framework where Java-level calling contexts (including concrete object aliases, method parameters, and taint labels) directly inform and constrain C/C++ API specification generation. This context sensitivity enables precise summarization of C/C++ API behaviors specific to each Java call site. Second, the approach introduces a unified specification representation that preserves cross-language semantic relationships while respecting each language's memory model. Third, CSS implements an iterative refinement process where native code specifications are dynamically optimized based on observed calling patterns from Java. This contribution addresses **Challenge (2)** by resolving semantic discontinuities at cross-language API boundaries. By incorporating caller contexts and unifying semantic representations, CSS enables precise and consistent taint analysis across Java and C/C++ domains. The evaluation

shows this caller-sensitive approach significantly improves analysis precision for real-world JNI applications compared to traditional context-insensitive methods.

(3) Hierarchical Points-to Specification Generation using LLMs. We introduce SPECGURU, a novel framework for automated points-to specification generation using LLMs in a hierarchical manner. The system employs a bottom-up approach to generate API specifications for the whole library. Unlike traditional methods that require manually designed analysis rules, SPECGURU leverages LLMs' pre-trained knowledge of code semantics to infer specifications, even when dealing with complex language features and incomplete code implementations. The LLM is capable of inferring the semantics of both complex, low-level operations and calls to closed-source third-party APIs by analyzing a function's signature and contextual usage. This directly addresses **Challenge (3)** by providing a scalable and automated approach for handling complex API semantics and partial implementation gaps, reducing reliance on handcrafted rules. For each function, SPECGURU generates candidate specifications by leveraging LLMs' understanding of code semantics, then validates them through differential testing against the original API implementations. These validated specifications serve as verified abstractions for progressively analyzing higher-level functions. The evaluation demonstrates that this hierarchical approach effectively generates more comprehensive points-to specifications for complex C libraries compared to traditional static analysis methods.

1.3 Thesis Organisation

The remainder of this thesis is organized as follows. Chapter 2 introduces the foundational concepts of our work, including API specification inference, the definition of specifications, fuzzing techniques, cross-language programming, and the

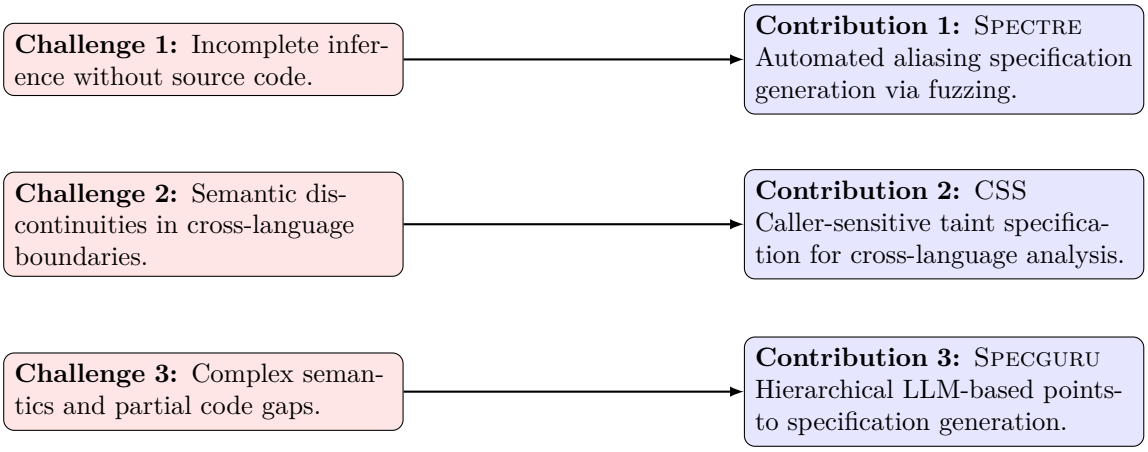


Figure 1.1: Overview of the challenges and how each contribution addresses them.

application of large language models in code analysis. Chapter 3 details our fuzzing-based approach for aliasing specification generation without complete source code access, including both black-box and grey-box analysis techniques. The chapter presents our novel alias-check observer design and its integration with guided input generation. Chapter 4 describes our caller-sensitive specification framework for cross-language analysis. This chapter explains our bidirectional analysis protocol and its application to JNI program analysis. Chapter 5 presents our hierarchical LLM-based approach for points-to specification generation. The chapter elaborates on our bottom-up validation methodology and differential testing framework. Finally, Chapter 6 summarizes our contributions and discusses future directions for improving automated API specification generation.

Thesis Statement. This thesis demonstrates that API specification generation can fundamentally enhance the precision and applicability of static analysis when analyzing programs that depend on external libraries. By introducing fuzzing-based (aliasing specification), caller-sensitive (taint specification), and LLM-driven (points-to specification) approaches for C/C++ library APIs, the thesis shows that it is possible to automatically produce more accurate and scalable

specifications than existing methods, thereby overcoming long-standing challenges in analyzing behaviors across API boundaries.

Chapter 2

Preliminaries

This chapter provides the foundational concepts for our study of specification inference, focusing on the automatic generation of C/C++ API library specifications to enhance static program analysis and verification. We first outline the basic concepts of specification generation, including alias, taint and points-to specifications, which enable precise analysis of data flows and pointer interactions (Section 2.1). We then explore the complexities of cross-language programming using the Java Native Interface (JNI), which helps us understand program behavior across language boundaries (Section 2.2). Following this, we discuss fuzzing techniques that support specification inference by dynamically exploring program behaviors (Section 2.3). Finally, we examine how large language models contribute to specification inference through advanced code comprehension across diverse programming contexts (Section 2.4).

2.1 Specification Generation

Specification generation is the process of automatically deriving specifications that describe the expected behavior, constraints, or properties of programs or func-

tions. These inferred specifications aid static program analysis, verification, and understanding by providing simplified yet behaviorally equivalent representations of complex implementations.

Definition 2.1 (Specification Generation Problem) *Given a program or function f and a set of properties Φ of interest, specification generation aims to produce a simplified representation S_f that preserves the essential behaviors of f with respect to Φ: $\forall \phi \in \Phi, \mathcal{B}(f, \phi) \Leftrightarrow \mathcal{B}(S_f, \phi)$, where $\mathcal{B}(f, \phi)$ represents f's behavior regarding property ϕ.*

By providing concise and accurate representations of program behaviors, specifications enable static analysis tools to reason about program properties across module boundaries. Next, we present definitions for three key types of specification generation relevant to our work: alias specifications, taint specifications, and points-to specifications. Each type focuses on different aspects of program behavior and serves specific analysis purposes.

2.1.1 Alias Analysis and Specifications

Alias analysis is a key technique in static program analysis that determines whether two or more pointer variables refer to the same memory location. It serves as the foundation for many advanced analyses, including points-to analysis [53, 68, 147], program slicing [117], and memory leak detection [30, 121]. When performing pointer analysis on client code that invokes APIs without available source code, a common practice is to over-approximate the aliasing behavior of those APIs , which assumes all possible aliasing relationships that cannot be ruled out, potentially including spurious aliases. This pessimistic approach maintains soundness by conservatively assuming worst-case scenarios, though it may sacrifice precision.

Conversely, optimistic approaches trade soundness for precision, which suffices for many practical applications. This approach is effective because client code typically interacts with external APIs only through their inputs and outputs, without access to implementation details.

Definition 2.2 (Aliasing Analysis Properties) *Given a library API f with parameters P and return value R, let $V = P \cup R$. We define the aliasing properties Φ_{alias} as the set of all possible aliasing relationships between variables in V: $\Phi_{alias} = \{(u, v) | u, v \in V$ and u may alias $v\}$, where two pointer variables u and v are considered aliases if there exists at least one common memory location m such that both u and v can point to m.*

To make aliasing analysis practical across API boundaries, we need compact representations that capture these aliasing relationships without requiring implementation details. This motivates the generation of aliasing specifications:

Definition 2.3 (Aliasing Specification) *Given a library API f and aliasing properties Φ_{alias} from Definition 2.2, an Aliasing Specification S_f is a simplified implementation that preserves aliasing behavior: $\forall \phi \in \Phi_{alias}, \mathcal{B}(f, \phi) \Leftrightarrow \mathcal{B}(S_f, \phi)$, where $\mathcal{B}(f, \phi)$ represents f's aliasing behavior regarding property ϕ.*

Aliasing specifications can help static analysis tools reason about pointer relationships across API boundaries without requiring implementation details. They support client code analysis such as memory leak detection and use-after-free detection by providing accurate aliasing information.

2.1.2 Taint Specifications

Taint analysis is a specialized form of information flow analysis that tracks the propagation of potentially untrusted or sensitive data through a program. It is

critical for security-related analysis, such as identifying vulnerabilities such as data leaks or injection attacks. To perform a taint analysis across API boundaries, we first need to characterize how tainted data can flow through API functions.

Definition 2.4 (Taint Properties) *Given a library API f with parameters P and return value R, let $V = P \cup R$. We define the taint properties Φ_{taint} as the set of all possible taint flow relationships: $\Phi_{taint} = \{(x, y) | x, y \in V$ and taint can flow from x to $y\}$, where $(x, y) \in \Phi_{taint}$ indicates that tainted data from x can propagate to y when f executes.*

To make taint analysis practical across library boundaries, we aim to track how APIs propagate tainted data in a simplified manner without requiring analysis of their source code. This motivates the generation of taint specifications:

Definition 2.5 (Taint Specification) *Given a library API f and taint properties Φ_{taint} from Definition 2.4, a Taint Specification S_f is a simplified implementation that preserves identical taint propagation behavior: $\forall \phi \in \Phi_{taint}, \mathcal{B}(f, \phi) \Leftrightarrow \mathcal{B}(S_f, \phi)$, where $\mathcal{B}(f, \phi)$ represents f's taint behavior regarding property ϕ.*

Taint specifications enable accurate tracking of potentially malicious inputs through API boundaries in static analysis. They facilitate security vulnerability detection in client code that uses third-party libraries, ensuring security analyses can operate even when API implementations are unavailable.

2.1.3 Points-to Specifications and Properties

Points-to analysis is a fundamental technique in static program analysis that identifies the specific memory locations that pointers may reference. This analysis is vital for reasoning about pointer interactions, enabling precise modeling of data flows

and memory manipulations in programs. Accurate points-to analysis is essential for reducing false negatives and ensuring the correctness of static analysis results. To characterize pointer relationships in API functions, we define four fundamental pointer operations:

Definition 2.6 (Pointer Analysis Properties) *Given a library API f, let $V = P \cup R$, where P is the set of parameters and R is the return value of f. We define $\Phi = \{\phi_{copy}, \phi_{load}, \phi_{store}, \phi_{field}\}$ as a set of pointer analysis properties that describe pointer interactions between variables $p, q \in V$ in f:*

- $\phi_{copy} : p = q$, *indicates that the value of q is assigned to p.*
- $\phi_{load} : p = *q$, *indicates that the value at the memory location pointed to by q is loaded into p.*
- $\phi_{store} : *p = q$, *indicates that the memory location pointed to by p is updated with the value of q.*
- $\phi_{field} : p = \&q \rightarrow f$, *indicates that the field f within the structure pointed to by q is accessed and assigned to p.*

Table 2.1 shows the syntax and examples of the four basic operations involved in our specifications. These operations form the foundation of our specification language and cover a wide range of interactions between arguments and return values in APIs.

These pointer properties form the basis for defining points-to specifications, which capture essential pointer behaviors in a simplified form suitable for static analysis.

Definition 2.7 (Points-to Specification) *Given a library API f and pointer analysis properties $\Phi_{pointer}$ from Definition 2.6, a Points-to Specification S_f is*

Table 2.1: The syntax and specification examples of aliasing specifications

Operation	*Syntax*	*Specification Example*
Copy	$p = q$	char *strchr(char *str, int c){ return str;}
Load	$p = *q$	char *dereference_char(char **p) { return *p;}
Store	$*p = q$	double strtod(char *str, char **endptr){*endptr =str;}
Field Access	$p = &q \rightarrow f$	char * json_get_value(json * item){ return item->value;}

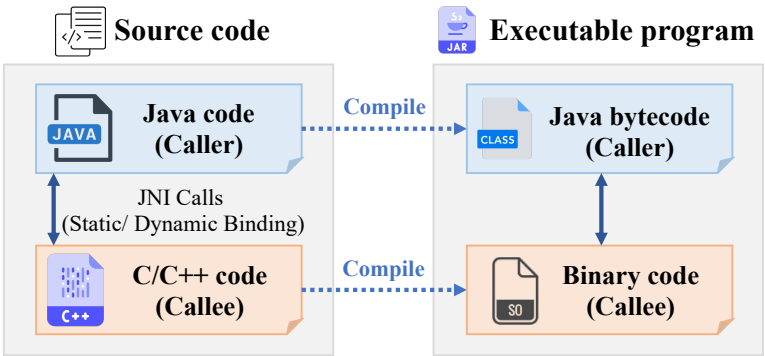


Figure 2.1: JNI interoperation

a simplified implementation that preserves identical pointer behavior: $\forall \phi \in \Phi_{pointer}, \mathcal{B}(f, \phi) \Leftrightarrow \mathcal{B}(S_f, \phi)$, *where* $\mathcal{B}(f, \phi)$ *represents* f*'s pointer behavior regarding property* ϕ.

Points-to specifications can improve the scalability and accuracy of static analysis tools by providing simplified yet behaviorally equivalent representations of API pointer operations. By abstracting away implementation complexity, they allow analyses to focus on pointer behaviors that directly impact client code.

2.2 Cross-Language Programming with JNI

Cross-language programming, which involves using multiple programming languages within a single software project, has become increasingly relevant in today's diverse computing environment. Java, as a widely-used language, employs

the Java Native Interface (JNI) to interact with native code written in languages like C and C++ (in this thesis, the terms ”native code” and ”C/C++ code” are used interchangeably). JNI enables Java methods to call native functions and vice versa, allowing access to functionalities in native libraries such as OpenCV, as well as platform-specific features or performance-critical libraries. However, while JNI integration expands Java's capabilities, it also introduces significant development complexity [79] and potential security vulnerabilities [24,54,64,70,73,77,79,91,109]. These risks are particularly concerning when handling sensitive data exchanges, as malicious information flows can exploit JNI vulnerabilities to bypass security checks and enable activities such as identity theft and unauthorized account access [24, 54, 64, 109, 125].

To better understand how Java and native code interoperate, we briefly introduce the typical workflow and mechanisms of JNI, as illustrated in Figure 2.1.

JNI calls. JNI enables the development of cross-language programs, allowing developers to use Java and C/C++ seamlessly. This process involves declaring and calling a “*native*” method in Java, which corresponds to a C++ callee function in C/C++. The Java code is compiled to Java bytecode, while the C/C++ code is compiled to binary code within a dynamic library. The Java Virtual Machine (JVM) facilitates JNI calls, enabling communication between the Java bytecode and binary code during program execution.

Binding modes. There are two binding modes for JNI development: static binding and dynamic binding. Static binding links a native method directly to its implementation during compilation, with a fixed function name format like *Java_PackageName_ClassName_MethodName*, determined at compile time. Additional macros and keywords, such as *JNIEXPORT* and *JNICALL*, ensure proper linkage between Java and native code. The *JNIEnv** pointer refers to the *JNIEnv*

interface, providing JNI functions to interact with the Java runtime, and the *jobject* or *jclass* is a reference to the invoking Java object or class. On the other hand, dynamic binding allows registering native methods at runtime using any naming scheme through the *RegisterNatives()* function and the *JNI_OnLoad()* function during native library initialization.

JNI functions. JNI provides a set of functions, such as locating classes, accessing methods and fields, and creating objects, that facilitate communication between Java and native code. JNI functions serve as a bridge between the two worlds, enabling seamless interaction and communication. By using these functions, developers can extend the capabilities of Java applications by leveraging platform-specific features and performance-critical native libraries. For example, the JNI function *FindClass()* can locate a Java class using its fully qualified name.

The JNI's cross-language interactions create significant blind spots for Java static analyzers, which typically cannot analyze the native C/C++ code being called. To address this challenge, we propose CSS, which automatically generates precise C/C++ API taint specifications for native libraries. These specifications capture critical data flows behaviors, enabling Java analyzers to accurately model JNI calls without accessing the native implementation. This approach allows security and bug detection to span both Java and C/C++ code boundaries.

2.3 Fuzzing

Fuzzing is an automated software testing technique designed to uncover bugs and security vulnerabilities. It employs a tool called a *fuzzer* to generate large volumes of random or semi-random inputs. In API fuzzing, these inputs are supplied to the targeted API through a specialized program known as a *fuzz driver* (or *fuzz*

harness), which is carefully designed to invoke the API under test. According to the awareness of the intrinsic information about the targeted program, fuzzing techniques can be classified as black-box, white-box, and grey-box. Black-box techniques [36, 123, 124] have no knowledge about the internals of the targeted program. So they can be applied to complex programs but may not be effective. On the contrary, white-box fuzzing techniques [23, 50, 51] utilize heavy-weight program analysis techniques (e.g. concolic execution) to improve effectiveness at the cost of scalability. To have the best of both worlds, grey-box fuzzing techniques [21, 34, 42, 71, 78, 142] are advocated to achieve scalability yet effectiveness. Therefore, grey-box fuzzing can address the limitations of the dynamic analysis techniques used in existing hybrid specification generation techniques.

Coverage-guided fuzzing, a prominent approach in grey-box testing [42, 71, 78] uses code coverage information to guide the fuzzing process towards discovering bugs in a program. It employs *instrumentation* inserted into the program to collect coverage data and maintains a pool of seed inputs, prioritizing those that have revealed previously unexplored code regions. This process, known as *seed selection*, guides the fuzzer's focus towards promising areas of the input space. A power scheduling mechanism, which allocates *energy* to seeds based on their historical performance, determines which seed to mutate next. Once selected, the seed undergoes *mutation* operations, including bit flips, byte swaps, arithmetic operations, and splicing, to generate new test cases. By continually exploring the input space and refining the seed corpus, coverage-guided fuzzing effectively increases the likelihood of discovering vulnerabilities and edge cases.

The dynamic nature of fuzzing makes it suitable for inferring API aliasing specifications, especially when complete source code is unavailable. By automatically generating a large and diverse set of random inputs without requiring manual de-

sign, fuzzing significantly reduces costs. These inputs enable exploration of various execution paths through targeted API invocations, effectively uncovering potential aliasing relationships between parameters and return values. To leverage these advantages, we propose SPECTRE, a coverage-guided fuzzing approach specifically designed for specification generation that combines the scalability of grey-box fuzzing with precise specification inference capabilities.

2.4 Large Language Model for Code-related Tasks

Recently, Large Language Models (LLMs) like the GPT series have demonstrated promising capabilities to understand code [141, 146]. Leveraging LLMs for specification inference offers the following two unique advantages over traditional code analysis methods:

- **Code level**: LLMs can handle complex code structures, such as deeply nested loops and recursive calls, by recognizing their intent and functionality based on learned patterns. This allows LLMs to provide better analysis even for unfamiliar codebases, unlike traditional static analysis tools that require complex designs specifically tailored to recognize these structures. Furthermore, LLMs are able to generalize across different programming languages and paradigms, making them versatile tools for cross-language code understanding and migration tasks.

- **Semantic level**: LLMs can understand the code snippets based on context and naming conventions, even without complete source code or documentation [136, 145]. For example, given the function call sort(arr, n, cmpfunc) (a hy-

pothetical example illustrating the LLM's ability to infer code semantics without full source code access), LLMs can infer that this is a sorting function and that cmpfunc is a custom comparison function. This capability is particularly valuable for analyzing real-world C libraries, which often invoke APIs from other libraries, such as the C standard library or proprietary third-party libraries, whose source code is unavailable. This poses significant challenges for traditional static analysis tools that rely on complete source code, as LLMs can leverage contextual understanding to infer functionality and intent, enabling more robust analysis in such scenarios.

Beyond these, LLMs have shown strong performance in tasks such as code summarization, bug detection, and automated code review. Their ability to learn from large-scale code corpora enables them to capture both explicit and implicit programming knowledge, making them powerful assistants for software engineering tasks. To harness these capabilities for specification generation, we propose SpecGuru, an LLM-based approach that leverages the semantic understanding and code comprehension abilities of large language models to automatically infer precise API specifications from source code.

Chapter 3

Fuzzing-Based API Aliasing Specification Generation

3.1 Overview

We introduce Spectre, an automated framework that leverages fuzzing techniques to generate aliasing specifications for library APIs without complete source code. Spectre operates efficiently both with and without source code access. When source code is unavailable, Spectre integrates alias-check observers into the driver programs after the API call sites and performs black-box fuzzing to explore API aliasing behaviors. If a check is satisfied, the corresponding aliasing specification is generated. When source code is available, Spectre incorporates new grey-box fuzzing features specifically tailored for aliasing specification inference, further enhancing its ability to generate aliasing specifications.

We conducted extensive experiments to evaluate the performance of Spectre. Without source code access, Spectre demonstrated its specification generation capability across both Musl, a lightweight C standard library, and eight C third-

party libraries. For Musl, SPECTRE recovered 96.7% of correct manually written specifications and identified 40.0% more aliasing specifications than those written by external experts. For C third-party libraries, all SPECTRE-generated aliasing specifications were validated as correct through static analysis of the API source code. SPECTRE is also more complete than other specification inference tool generating 16.7% more correct specifications for third-party libraries. The practicality of the generated specifications was confirmed, as they improved aliasing analysis in static pointer analysis of client code while maintaining a balance between accuracy and efficiency. The effectiveness of the tailored grey-box fuzzing features was demonstrated by SPECTRE identifying 20% more specifications compared to when these features were disabled. These results show that SPECTRE is an effective tool for inferring aliasing specifications and facilitating static analysis.

3.1.1 Motivation

Static analysis often encounters substantial challenges when analyzing client code that interacts with third-party libraries, especially when the source code of these libraries is unavailable. In such scenarios, the lack of visibility into the internal behavior of external APIs impedes precise reasoning about their side effects, data dependencies, and aliasing relationships. This loss of semantic information can compromise the soundness or precision of the analysis, leading to missed bugs, inaccurate dependency tracking, or conservative over-approximations that reduce the utility of the analysis results.

These challenges are particularly pronounced in languages like C or C++, where external APIs may perform complex memory manipulations, invoke inline assembly, or use opaque types and function pointers. Moreover, the presence of dynamically linked or proprietary closed-source libraries makes it impossible to statically

inspect API implementations, forcing static analyzers to treat them as black boxes. As a result, static analysis must either skip over such APIs—losing critical interprocedural context—or rely on approximations that may not generalize well.

To enable effective analysis in these settings, it is essential to have accurate and scalable ways of modeling or approximating the behavior of external APIs without requiring access to their source code.

3.1.2 Challenges

To address the limitations posed by the lack of source code for external APIs, researchers have explored several strategies, each with notable drawbacks in terms of precision, scalability, and applicability. In the absence of source code, manually crafting specifications often requires developers to meticulously study library documentation to understand API behavior, including its effects on program state, such as memory operations, pointer relations, and side effects [8, 44, 120]. This process demands significant expertise and effort, as developers must interpret potentially complex or poorly structured documentation to create accurate and comprehensive specifications. However, this approach is fraught with challenges, as library documentation is frequently incomplete, outdated, or ambiguous, leading to specifications that may not fully capture the API's behavior. Moreover, maintaining these manually written specifications becomes increasingly cumbersome as libraries evolve or scale, making this method impractical for large or frequently updated codebases.

Other techniques attempt to infer API behavior from available documentation or from usage patterns observed in client code [38, 132] to automatically generate specifications. These approaches aim to reduce the manual effort by leveraging natural language processing or machine learning to extract behavioral insights from

documentation or by analyzing how APIs are used in existing codebases. While promising, these methods face significant limitations. Documentation-based inference depends heavily on the quality and clarity of the documentation, which may lack critical details or contain errors. Similarly, usage-based inference risks inheriting flawed or non-representative patterns from the analyzed code, leading to specifications that may be incomplete or incorrect. Consequently, these automated approaches often struggle to produce reliable, generalizable specifications, particularly for complex APIs or diverse usage contexts.

3.1.3 Our Solution

We present SPECTRE, an automated framework for generating aliasing specifications for C library APIs using *black/grey-box fuzzing* based on the availability of API source code. SPECTRE first integrates API fuzz drivers (programs containing targeted API calls) with specially designed aliasing check code (alias-check observers). Then, during the fuzzing process driven by fuzzer (e.g., AFL), these alias-check observers monitor and generate specifications for aliasing relationships between arguments and return values[1]. The key advantage of this approach is that it enables inference of aliasing relationships without requiring access to the API source code, though this lack of source code inherently limits the ability of SPECTRE to fully explore argument-return value aliasing patterns, potentially missing some specifications.

When the source code of APIs is available, SPECTRE incorporates a suite of new grey-box fuzzing features based on traditional grey-box fuzzer to enhance aliasing specifications inference. The challenge behind these new grey-box features is that traditional grey-box fuzzing primarily focuses on maximizing code coverage rather

[1]Only mention the relationship between arguments (also the values passed through them) and the return values for brevity in the following

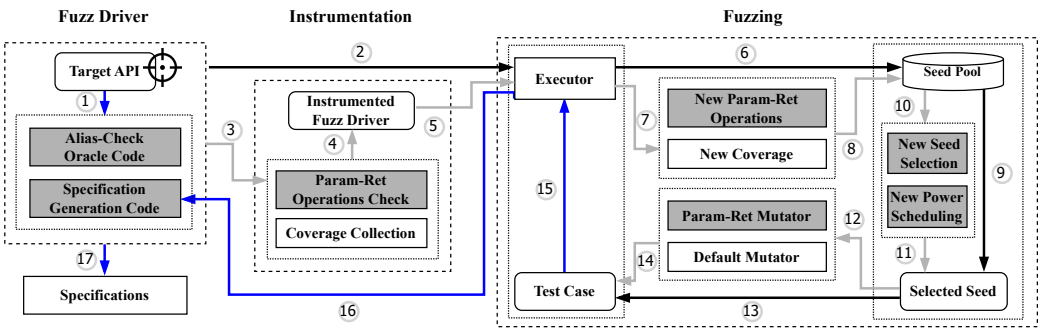
than inferring aliasing specifications, which may overlook some aliasing specifications. To address this, we perform lightweight instrumentation on critical code parts in the source code of the targeted API that may expose relationships between parameters and return values, complemented by specially designed seed selection, power scheduling, and seed mutation. These new features help the fuzzing process more effectively reveal interesting aliasing relationships and overcome the limitations of traditional grey-box fuzzing, which only prioritizes code coverage, and enhance the capability of Spectre to generate aliasing specifications.

In summary, the following are our contributions:

- We introduce Spectre, a new tool based on fuzzing that incorporates an alias-check observer module to automatically generate aliasing specifications for C library APIs without available source code.
- We augment Spectre with tailored grey-box fuzzing features for aliasing specification inference when API source code is available, enhancing its ability to discover more aliasing specifications.
- We conduct a thorough evaluation of Spectre with C standard library and C third-party library APIs. The experimental results demonstrate the superior performance of Spectre in inferring aliasing specifications.

3.1.4 Framework Overview

Figure 3.1 illustrates an overview of Spectre. The grey rectangles represent modules added by Spectre in addition to the conventional grey-box fuzzing framework, specifically designed to infer aliasing specifications. Spectre supports two modes of fuzzing: *black-box mode* and *grey-box mode*, depending on the availability of API source code.



Black-box step: →; Grey-box step: →; Common step: →;
The execution sequence in black-box mode: ①②⑥⑨⑬⑮⑯⑰;
The execution sequence in grey-box mode : ①③④⑤⑦⑧⑩⑪⑫⑭⑮⑯⑰;

Figure 3.1: The design overview of Spectre. Grey rectangles are unique features of Spectre

Given a fuzz driver of the targeted API, SPECTRE first injects *alias-check observer code* and *specification generation code* after the call site of the targeted API (step ①). The injected code is essential for monitoring API execution and capturing aliasing specifications during the fuzzing process. Depending on whether the API source code is available, SPECTRE proceeds with either black-box or grey-box mode fuzzing for the subsequent compilation and fuzzing stages.

- **Black-box mode**: When API source code is unavailable, SPECTRE operates in black-box mode. Due to the lack of internal API implementation details, SPECTRE cannot fully utilize the default coverage instrumentation (*Coverage Collection*) or specialized instrumentation (*Param-Ret Operation Check*) modules for alias inference. The fuzz drivers with *alias-check observer code* and *specification generation code* are then directly executed by the *executor* (step ②). The fuzzing process relies on analyzing input-output behaviors: test cases are drawn from the *seed pool* and executed by the executor (steps ⑥⑨⑬⑮), while the *alias-check observer code* and *specification generation code* in fuzz driver mon-

itors API behavior and generates aliasing specifications during fuzzing (steps ⑯⑰).

- **Grey-box mode**: When API source code is available, SPECTRE operates in grey-box mode and can leverage all grey-box fuzzing features for more efficient exploring aliasing behaviors. The fuzz drivers are first instrumented through *Coverage Collection* and *Param-Ret Operation Check* to generate instrumented fuzzer drivers (steps ③④), which is then executed by the *executor* (step ⑤). During execution, SPECTRE collects information about *new Param-Ret operations* and *new coverage* (steps ⑦⑧), and employs specialized *seed selection* and *power scheduling* strategies to prioritize seeds that reveal new behaviors (steps ⑩⑪). It then uses a custom *Param-Ret mutator* along with *default mutation* strategies (steps ⑫⑬⑭) to generate new test cases for the next round of API execution (step ⑮). Similarly, the *alias-check observer code* and *specification generation code* in the fuzz drivers monitor API behavior and generate aliasing specifications (steps ⑯⑰).

3.2 Motivating Example

In this section, we first illustrate the necessity of specification inference and how SPECTRE generates aliasing specifications through a simplified C-like code example in Figure 3.2 where the source code of APIs is *inaccessible*. We then explain why tailored grey-box fuzzing features can more effectively generate aliasing specifications when the source code of APIs is *accessible*, as shown in Figure 3.4.

The client code snippet in Figure 3.2 manages user information using library APIs. It stores user name user and associated information info using lib_setUserInfo(). Later, it retrieves this information with lib_getUserValue() and

```
1   Client Code
2   typedef struct Node{char *user; char *info;} Node;
3
4   char *user = getUserName();
5   char *info = getUserInfo();
6   Node node;
7   lib_setUserInfo(&node, user, info);
8   ...
9   char *key = getKey();
10  char *value = lib_getUserValue(&node, user);
    ...
18  if (value != NULL)
19    char *result = lib_extractUserInfo(value, key);
20    ...
```

```
21  Specifications of Library APIs
22  void lib_setUserInfo(Node* node, char *user, char *info)
23  {
24    node->user = user;
25    node->info = info;
26  }
27  char* lib_getUserValue(Node* node, char *user)
28  {
29    return node->info;
30  }
31  char* lib_extractUserInfo(char *info, char *key)
32  {
33    return info;
34  }
```

Figure 3.2: Client code with aliasing specification of library APIs

```
9   ...
10  char *value = lib_getUserValue(&node, user);
11  if (node.user == value){
12     // Aliasing specification generation
13  }
```

```
14  if (node.user == value) {...}
15  if (node.info == value) {...}
16  if (node.info == user) {...}
17  if (user == value) {...}
18  ...
```

Figure 3.3: Aliasing specification inference for lib_getUserValue()

processes it using lib_extractUserInfo() based on a specific key. When performing a static pointer analysis on the client code, the main challenge arises from calls to these three closed-source APIs. Without access to their source code, the static analyzer cannot perform pointer analysis, making it difficult to track how info is handled within these API calls.

For example, if the behaviors of lib_setUserInfo(), lib_getUserValue(), and lib_extractUserInfo() are summarized in the form of C-like specifications (where aliases $\overset{a}{\leftrightarrow}$ are established: in lib_setUserInfo(), node→user $\overset{a}{\leftrightarrow}$ user and node→info $\overset{a}{\leftrightarrow}$ info; in lib_getUserValue(), return $\overset{a}{\leftrightarrow}$ node→info; in lib_extractUserInfo(), return $\overset{a}{\leftrightarrow}$ info), as shown in lines 21–34 in Figure 3.2, and provided to the static analyzer, it would enable the analyzer to infer the aliasing relationships between info, value, and result.

```
1   char *lib_extractUserInfo(char *info, char *key)
2   {
3      size_t key_len = strlen(key);
4      size_t info_len = strlen(info);
5      if (key_len >= 3 && info_len > key_len)
6          return strstr(info, key);
7      else
8          return NULL;
9   }
```

Input (1): info = "Mike123", key = "Jack" — ① → ③ → ④ → ⑤ → ❻

Input (2): info = "Mike123", key = "Ja" — ① → ③ → ④ → ⑤ → ⑦ → ❽

Input (3): info = "Mike123", key = "Mike" — ① → ③ → ④ → ⑤ → ❻

Figure 3.4: Three test cases execution traces of the lib_extractUserInfo()

Therefore, in the absence of source code for these library APIs, to perform precise pointer analysis on info, the static analyzer needs to be "*informed*" about the relationships between the arguments and return values of these three APIs. For example, if the behaviors of lib_setUserInfo(), lib_getUserValue(), and lib_extractUserInfo() are summarized in the form of C-like specifications, as shown in lines 21–34 in Figure 3.2, and provided to the static analyzer, it would enable the analyzer to infer the aliasing relationships between info, value, and result. This information would help the analyzer to construct sound points-to sets, facilitating subsequent analysis phases.

In real-world scenarios, the source code of library APIs may be inaccessible due to proprietary distribution models, licensing constraints, or precompiled binaries. For example, macOS includes proprietary file parsing libraries in its SDK for handling various file types, which are distributed only as binaries. Fortunately, these APIs can typically be called and executed, and useful information can be discovered from execution results. Inspired by this, Spectre infers aliasing specifications through API execution results. Specifically, based on the API signatures, Spectre recursively decomposes the structure of arguments and return values, identifying all potential aliasing relationships and inserting appropriate alias check code after each API callsite (alias-check observer) in the driver program. Then Spectre leverages fuzzing to extensively explore API alias behaviors by generating a large number of diverse and random inputs. The alias-check observer is responsible for validating and generating aliasing specifications throughout multiple fuzzing executions. For example, as shown in Figure 3.3, because the first argument node of lib_getUserValue() is a struct type, Spectre recursively decomposes node and inserts check code between all type-compatible fields, arguments, and the return value. The five if statements are inserted after the callsite of lib_getUserValue() to

check for potential aliases. When aliases of interest are identified, they are recorded in the form of aliasing specifications.

During the fuzzing process, aliasing specifications are generated as long as they are observed at least once, without requiring them to be true in all executions. The rationale behind this design choice is that if an aliasing relationship is possible in any execution, it should be considered in static analysis to enhance soundness even if an alias only occurs in specific conditions or rare cases, whereas most classical alias analyses ignore unknown or unsummarized APIs. This approach aligns with our goal of supporting static analysis by capturing all potential aliasing relationships, rather than just the most common ones. For example, consider the lib_setUserInfo() API function in Figure 3.2. During the fuzzing process, we might observe two aliasing relationships in two different executions: (1) node->user = user; (2) node->info = info; These relationships are recorded separately as observed. After fuzzing, the two aliasing relationships are merged into a single comprehensive specification for lib_setUserInfo(). This specification captures both aliasing behaviors, providing static analysis tools with a more complete picture of the potential aliasing scenarios that could arise during program execution.

Traditional grey-box fuzzing primarily focuses on maximizing code coverage, which is insufficient for inferring aliasing specifications. In the case of lib_extractUserInfo(), the key aliasing relationship occurs when this function returns a pointer to the info parameter (e.g., via strstr(info, key)). This creates an aliasing relationship between info and the return value of the function. Traditional coverage-based fuzzing might generate inputs such as: Input (1): info = "Mike123", key = "Jack"; Input (2): info = "Mike123", key = "Ja"; While these inputs achieve 100% path coverage by executing both the if and else branches, they fail to reveal the critical aliasing relationship because: In Input (1), key is not a substring of info,

and in Input (2), key is too short (key_len < 3), so the function returns NULL in both cases. Additionally, once these paths have been executed, traditional fuzzing often deprioritizes further exploration of these paths, as its primary goal is to discover new coverage. This means that even if certain paths contain potential aliasing relationships, they may still be overlooked because the fuzzer deprioritizes further exploration of these paths after they have been executed. Therefore, to avoid missing such aliasing specifications, it is crucial to focus on key instructions that are likely to generate aliasing relationships, such as return strstr(info, key).

To address this limitation, in addition to traditional coverage-based instrumentation, SPECTRE also employs lightweight instrumentation targeting critical instructions that are likely to generate aliasing relationships, specifically those that may exhibit aliasing between arguments and return values. During fuzzing, if a seed triggers these instrumented instructions, it is assigned a higher priority and repeatedly executed to explore potential aliasing behaviors. Additionally, SPECTRE employs a specially designed seed mutator to generate inputs that are more likely to reveal aliasing relationships. For example, the mutator can produce Input (3): info = "Mike123", key = "Mike", where key is a substring of info, causing strstr(info, key) to return a pointer into info and revealing the aliasing relationship. Through specially designed fuzzing features, SPECTRE does not quite ensure that aliasing-critical paths are thoroughly explored, but it does increase the odds.

3.3 Approach

3.3.1 Alias-Check Observer

When using dynamic methods to infer the behaviors of an API, it is essential to have a mechanism that extracts and analyzes the execution information of the API.

Similar to the use of oracles in software testing and traditional grey-box fuzzing to distinguish between desired, correct behavior and potentially incorrect behavior [15], we employ *alias-check runtime observers* to detect aliasing relationships that emerge during the execution of library APIs. In order to fuzz a new API, SPECTRE requires knowledge of the API's function prototype, which can typically be obtained from the corresponding header files. This information allows the tool to correctly construct inputs and monitor aliasing behaviors during execution.

To formalize this analysis, consider an API F with the callsite $\mathcal{C}_F$: $y = F(x_1, x_2, ..., x_n)$. We define the following sets:

- $X = \{x_1, x_2, ..., x_n\}$: set of arguments,
- $Y = \{y\}$: set containing the return value, or $\emptyset$ if F has no return value,
- $V = X \cup Y$: set of all variables involved in the API call.

The following is the definition of *Alias-Check Observer*:

Definition 3.1 ***(Alias-Check Observer)*** *Alias-Check Observer is a mechanism designed to determine whether two variables u and v in V are aliases after the execution of F. It can be represented as $\mathcal{O} : V \times V \rightarrow \{true, false\}$, where $\mathcal{O}(u, v) =$ true if u and v could be aliases after the execution of F, and false otherwise.*

The aliasing specifications are essentially simplified implementations of APIs, as shown in Table 2.1. The aliasing specification of the targeted API consists of two components: the API function signature and the API function body. Both the function signature and multiple potential function bodies are integrated into the fuzz driver, with each function body representing one possible aliasing relationship. The specific function body used is determined by the alias-check observer based on the alias behaviors detected (as demonstrated in lines 11–17 of Figure 3.3). In other words, the actual simplified implementation of the function body depends on

the alias behaviors captured during fuzzing, with the alias-check observer making this determination. Therefore, the specifications can be automatically generated during the fuzzing process. There are four categories of alias-check observer: $\mathcal{O} = \{ \mathcal{O}_{\text{copy}}, \mathcal{O}_{\text{store}}, \mathcal{O}_{\text{load}}, \text{ and } \mathcal{O}_{\text{field_access}}\}$, each addressing a specific type of aliasing relationship. The implementation of alias-check observer in SPECTRE is to integrate a set of check code into the driver program that calls the API. This check code determines whether each execution of the API has resulted in aliasing between specific pairs of variables and generates corresponding aliasing specifications where aliases are confirmed. By analyzing pairs (u, v), SPECTRE can identify and characterize the aliasing behaviors of the API under test. Next, we will detail the generation of alias-check observers, starting with *Copy*, *Load*, and *Store* alias-check observers, and then covering *Field Access* alias-check observer.

3.3.1.1 Copy, Load, Store Alias-Check Observers

The inference of four categories of alias-check observers $\mathcal{O} = \{ \mathcal{O}_{\text{copy}}, \mathcal{O}_{\text{store}}, \mathcal{O}_{\text{load}}, \text{ and } \mathcal{O}_{\text{field_access}}\}$ focuses specifically on pairs where both u and v are pointer types, as aliasing occurs when two pointers reference the same objects. By examining the indirection level of variables (i.e., the number of pointer dereferences needed to access the actual value), we can determine which observers need to be inserted for each variable pair. This allows for the exclusion of pairs where neither u nor v is a pointer, reducing the number of pairs to be checked.

For the remaining pointer pairs, two key factors are considered: pointer type compatibility and indirection level. Pointer type compatibility is essential for establishing meaningful aliasing relationships. Our alias-check observer code includes checks for simple type casts, such as those between void* and char*. When variables of these types point to the same address, the aliasing relationship is recorded.

However, more complex type casts (e.g., between unrelated pointer types or custom types) are not currently supported in the specification. The indirection level of pointers plays a crucial role in determining the specific type of aliasing relationship. Here we introduce some notations:

- P: the set of pointer types,
- z^t: the type of variable z,
- $\sim$: signifies type compatibility between two pointer types,
- $\varphi(t)$: a function that returns the indirection level of a type t.

Then the generation of three types of alias-check observers can be defined as follows:

Copy Alias-Check Observer When u and v have the same indirection level, a *Copy* alias-check observer is generated:

$$\mathcal{O}_{\text{copy}} : \frac{(u, u^t), (v, v^t) \in V \times P,\ u^t \sim v^t,\ \varphi(u^t) = \varphi(v^t)}{genObserver(u, v)}$$

The $genObserver(u, v)$ is a function that generates alias-check observer code for the pointer pair (u, v) and produces the corresponding aliasing specification if the aliasing relationship is confirmed, as exemplified in the *#Specification Example* in Table 2.1.

Load and Store Alias-Check Observers Similar alias-check observer for *Load* and *Store* are derived, which differ in their handling of indirection levels:

$$\mathcal{O}_{\text{load}} : \frac{(u, u^t), (v, v^t) \in V \times P,\ u^t \sim v^t,\ \varphi(u^t) = \varphi(v^t) + 1}{genObserver(u, *v)},$$

$$\mathcal{O}_{\text{store}} : \frac{(u, u^t), (v, v^t) \in V \times P,\ u^t \sim v^t,\ \varphi(u^t) = \varphi(v^t) - 1}{genObserver(*u, v)}.$$

The key distinction between $\mathcal{O}_{\text{load}}$ and $\mathcal{O}_{\text{store}}$ lies in the level of indirection they handle: $\mathcal{O}_{\text{load}}$ is used when u has one more level of indirection than v, indicating u aliases the dereferenced value of v; $\mathcal{O}_{\text{store}}$ applies when u has one less level of indirection than v, indicating the dereferenced value of u aliases v.

For example, as shown in Table 2.1, for the API strchr(), since the argument str and the return value strchr_ret have the same pointer type char * (type compatibility), a copy alias-check observer if (strchr_ret == str) would be inserted after the char * strchr_ret = strchr(str) callsite. Similarly, a load alias-check observer if (deref_char_ret == *p) and a store alias-check observer if (*endptr == str) would be inserted after the char *dereference_char_ret = deref_char(p) and double d = strtod(str, endptr) API callsites, respectively.

3.3.1.2 Field Access Alias-Check Observer

For the *Field Access* alias-check observer, we specifically targeted scenarios where the arguments or return values involve structs. To achieve this, we categorize the set of all pointer types P into two subsets: P_S and P_B. Here, P_S represents struct pointer types, while P_B represents non-struct pointer types. We consider the following two scenarios for inserting alias-check observers that checks for potential aliasing relationships between u and v based on their types and possible field access relationships:

(1) When both u and v are struct pointer types: $(u, u^t), (v, v^t) \in V \times P_S$.

(2) When one variable is a struct pointer type and the other is a non-struct pointer type: $(u, u^t) \in V \times P_S,\ (v, v^t) \in V \times P_B$ or $(u, u^t) \in V \times P_B,\ (v, v^t) \in V \times P_S$.

When performing analysis on variables of struct types, an issue may arise with recursive data structures. Consider a linked list node defined as struct Node {int data; struct Node next;}. In this struct definition, the field next is a pointer to the

same struct type, struct Node. Starting from a variable node of type struct Node, we can trace a series of field accesses from node, such as node->next, node->next->next, and so on. This sequence, known as *access path*, can potentially extend infinitely. This recursive nature of access paths could pose challenges in the analysis and need to be properly addressed when generating the field access alias-check observers.

One solution to address infinite access paths is the application of k-limiting [57]. This technique sets a maximum length k for all access paths within an analysis. Let L_{node} denote the access path originating from a given *node*, which represents a sequence of field or reference accesses starting from *node* and terminating at the specific field within the data structure. If the length of an access path L_{node} exceeds k, it will be truncated, thereby ensuring that $|L_{node}|$, the length of the access path, will not exceed k. This k-limiting effectively prevents the generation of infinitely access paths and manages structs with direct or indirect recursive self-references.

In the field access alias-check observer generation, we also adopt k-limiting to manage the potentially infinite length of access paths and field access alias-check observers can be derived as $\mathcal{O}_{\text{field_access}}$.

$$\mathcal{O}_{\text{field_access}} : \frac{(u, u^t) \in V \times P_S,\ (v, v^t) \in V \times P_B,\ F = \{f \mid f \in \text{fields}(u^t),\ u.f^t \sim v^t,\ |L_u| \leq k\}}{\forall f \in F\ \vdash\ \mathcal{O}(u.f, v)}$$

$$\mid \frac{(u, u^t) \in V \times P_B,\ (v, v^t) \in V \times P_S,\ F = \{f \mid f \in \text{fields}(v^t),\ u^t \sim v.f^t,\ |L_v| \leq k\}}{\forall f \in F\ \vdash \mathcal{O}(u, v.f)}$$

$$\mid \frac{\begin{array}{c}(u, u^t), (v, v^t) \in V \times P_S,\\ F = \{(f, g) \mid f \in \text{fields}(u^t),\ g \in \text{fields}(v^t),\ u.f^t \sim v.g^t,\ |L_u| \leq k,\ |L_v| \leq k\}\end{array}}{\forall (f, g) \in F\ \vdash \mathcal{O}(u.f, v.g)}$$

Under the specified k, we identify the set of fields F, which includes all type-compatible fields that also satisfy the k-limiting constraint on access path length. The set F forms the basis for deriving field access alias-check observers. Then, based on F, we derive ($\vdash$) the field access alias-check observer $\mathcal{O}(x, y)$. The $\mathcal{O}(x, y)$

in $\mathcal{O}_{\text{field_access}}$, including $\mathcal{O}(u.f, v)$, $\mathcal{O}(u, v.f)$ and $\mathcal{O}(u.f, v.g)$, is to recursively select the appropriate observer $\mathcal{O}_{\text{copy}}$, $\mathcal{O}_{\text{load}}$, $\mathcal{O}_{\text{store}}$, or $\mathcal{O}_{\text{field_access}}$ based on the types of the variables x and y. It is important to note that there are only three fundamental types of observers: $\mathcal{O}_{\text{copy}}$, $\mathcal{O}_{\text{load}}$, and $\mathcal{O}_{\text{store}}$. The $\mathcal{O}_{\text{field_access}}$ observer recursively decompose struct variables into their constituent parts, which can then be processed by the three fundamental observers. However, for the sake of convenience and clarity in our representation, we still use these four observer types in our formulation.

For example, continuing with the struct Node{int data;struct Node *next;} example. When we need to determine if the next field of an argument node of type struct Node and a return value ret are aliases and k is set to 2. We would insert the code if(ret == node->next) and if(ret == node->next->next) after the API callsite. But we would avoid inserting a check like if(ret == node->next->next->next) as it exceeds the set value of k.

While k-limiting simplifies the generation of field access alias-check observers, it may lead to uncomplete aliasing specifications. For instance, in the above struct Node example, we might overlook potential aliasing relationships beyond the third next node. A potential solution could be to increase the value of k, allowing for more fields to be checked. However, this approach also does not guarantee coverage of all possible aliasing scenarios. Particularly with long linked lists or complex recursive structures, there may still be missed aliases. Additionally, increasing the value of k could result in higher computational complexity and longer runtime.

3.3.1.3 Algorithm

Algorithm 1 shows the steps for generating API alias-check observers and ultimately producing the alias-check code like lines 11-17 in Figure 3.3. It begins by verifying

Algorithm 1: Alias-Check observer Generation

```
Input: The signature of the API F and callsite C_F.
Output: Alias-Check Observers O.
1  O ← [];
2  for each argument pair (arg1, arg2) ∈ C_F do
3      if arg1.isPointer ∧ arg2.isPointer then
4          CheckRelation(arg1, arg2);
5      end
6  end
7  if ret.isPointer then
8      for each argument arg ∈ C_F do
9          if arg.isPointer then
10             CheckRelation(arg, ret);
11         end
12     end
13 end
14 return O;
15 Procedure CheckRelation(op1, op2)
16     if op1.isStruct ∧ op2.isStruct then
17         if typesMatch(op1, op2) then
18             CopyLoadStoreCheck(op1, op2);
19         end
20         for each field pointer f ∈ op1 and each field pointer m ∈ op2 do
21             if typeMatch(op1 → f, op2 → m) ∧ kLimit(op1 → f) ∧ kLimit(op2 → m)
                 then
22                 CheckRelation(op1 → f, op2 → m);
23             end
24         end
25     end
26     else if op1.isStruct then
27         for each field pointer f ∈ op1 do
28             if typeMatch(op1 → f, op2) ∧ kLimit(op1 → f) then
29                 CheckRelation(op1 → f, op2);
30             end
31         end
32     else if op2.isStruct then
33         for each field pointer m ∈ op2 do
34             if typeMatch(op1, op2 → m) ∧ kLimit(op2 → m) then
35                 CheckRelation(op1, op2 → m);
36             end
37         end
38     else if typesMatch(op1, op2) then
39         CopyLoadStoreCheck(op1, op2);
40 Procedure CopyLoadStoreCheck(v1, v2)
41     if IndirectionLevels(v1) == IndirectionLevels(v2) then
42         O ← O∪ IFAliasCodeGeneration(v1, v2);
43     end
44     else if IndirectionLevels(v1) == IndirectionLevels(v2) + 1 then
45         O ← O∪ IFAliasCodeGeneration(v1, *v2);
46     else if IndirectionLevels(v1)== IndirectionLevels(v2) - 1 then
47         O ← O∪ IFAliasCodeGeneration(*v1, v2);
```

type compatibility between two pointer variables and then compares their indirection levels to determine the appropriate observer types $\mathcal{O}_{\text{copy}}$, $\mathcal{O}_{\text{load}}$, and $\mathcal{O}_{\text{store}}$ based on whether the levels are equal or differ by one. When dealing with variables of the struct type, the algorithm employs k-limiting to prevent infinite recursion in nested structs and maps the variables to $\mathcal{O}_{\text{copy}}$, $\mathcal{O}_{\text{load}}$, and $\mathcal{O}_{\text{store}}$ respectively based on their types. The observer generation is implemented through the IFAliasCodeGeneration() function (lines 29, 31, 33). Apart from generating the code to check whether the arguments and return values are aliases, IFAliasCodeGeneration() also includes the generation code for corresponding aliasing specifications. Note that the if statements in Figure 3.3 omits the check for whether two pointer variables are null, but the actual code does include these checks.

3.3.2 New Fuzzing Features

Spectre introduces specific fuzzing features to generate aliasing specifications more effectively when API source code is accessible. These features include new lightweight instrumentation, seed selection, power scheduling, and seed mutation.

3.3.2.1 Instrumentation

Traditional grey-box fuzzing primarily aims to maximize code coverage such as branch coverage. Once certain critical code parts have been covered, they may no longer be a point of interest in subsequent executions, potentially causing some aliasing specifications to be overlooked. As shown in Figure 3.4, *Input (1)* and *Input (2)* execute the if and else branches, achieving 100% branch coverage but failing to reveal the aliasing relationship between info and key. To uncover such an aliasing relationship, it is necessary to mutate the input and repeatedly execute the critical code part (line 6). However, traditional grey-box fuzzing might shift

its focus to exploring other unexecuted branches after executing the if branch, consequently overlooking the alias relationship between info and key. Therefore, extra instrumentations other than coverage collection ones are needed to record aliasing information and prevent such omissions.

Given that SPECTRE focuses on the four types of aliasing relationships shown in Table 2.1, which primarily involve interactions between parameters and return values, our insight is that the instructions in the API source code that operate on parameters and return values are likely to exhibit the aliasing relationships of interest. Therefore, these instructions need to be given special attention, like the instruction return strstr(info, user) on line 6 in Figure 3.4. SPECTRE first performs instrumentation on instructions of available source code that involve parameters and return values through static analysis. Their traces are then recorded during the dynamic fuzzing process. Specifically, SPECTRE targets the instrumentation of *Params-Ret instructions*—two instruction types within library API source code:

- Instructions that include two or more parameters;
- Instructions that at least one parameter is used as a return value.

SPECTRE considers that there may exist aliasing relationships of interest if an instruction in the source code of a library API is categorized as one of the Params-Ret instructions types. Operations on parameters in instructions come in two types: direct and indirect. Direct operation uses the parameters as passed, while indirect operation involves transferring the parameters to other variables for additional manipulation before use. For example, in void func(void *p, void *q) { q = p; void *temp = p; q = temp;}, the assignment q = p; is a direct operation on parameters p and q. In contrast, the use of temp in q = temp; is an indirect operation on the parameter p.

Algorithm 2: Params-Ret operations Instrumentation

```
Input: The function 𝓕 and tracked parameters 𝓐.
Output: The instrumented Instructions 𝓘.
1  𝓘 ← [], 𝓐 ← 𝓕.Params;
2  𝓥 ← [] // Visited set to track instrumented functions
3  Procedure instrumentFunc(𝓕, 𝓐)
4      if 𝓕 ∈ 𝓥 then
5          return // Skip if already visited
6      end
7      𝓥 ← 𝓥 ∪ 𝓕 // Mark as visited
8      for BasicBlock bb ∈ 𝓕 do
9          for Instruction inst ∈ bb do
10             if inst.operandsNum ≥ 2 then
11                 P ← [];
12                 for Operand op ∈ inst do
13                     if isDirectParam(op, 𝓐) ∨ isIndirectParam(op, 𝓐) then
14                         P ← P ∪ op;
15                     end
16                 end
17                 if P.size ≥ 2 then
18                     if inst.isCallInst then
19                         callee ← inst.getCallee();
20                         instrumentFunc(callee, P);
21                     end
22                     else
23                         inst_i ← instrument(inst);
24                         𝓘 ← 𝓘 ∪ inst_i;
25                     end
26                 end
27             end
28             if inst.operandsNum = 1 then
29                 if inst.isReturn then
30                     op ← inst.getOperand(0);
31                     if isDirectParam(op, 𝓐) ∨ isIndirectParam(op, 𝓐) then
32                         if inst.isCallInst then
33                             callee ← inst.getCallee();
34                             instrumentFunc(callee, op);
35                         end
36                         else
37                             inst_i ← instrument(inst);
38                             𝓘 ← 𝓘 ∪ inst_i;
39                         end
40                     end
41                 end
42             end
43         end
44     end
45     return 𝓘_i;
```

Leveraging the def-use chains provided by LLVM, we can determine whether an operand of a instruction is a direct or indirect parameter. Algorithm 2 describes the inter-procedural process of instrumenting Params-Ret instructions in the source code of library API. The algorithm iterates through each basic block in the function F. For every instruction within these blocks, it performs the following checks: (1) Whether the instruction has two or more operands that are either direct or indirect parameters (lines 9-20), and (2) Whether the instruction is a return instruction and its operand involves at least one direct or indirect parameter (lines 21-30). If either condition is met, the instruction is instrumented.

Example: In Figure 3.4, one of the return instructions of lib_extractUserInfo() (line 6) has the operands info and key. Since info and key are direct parameters, SPECTRE performs instrumentation at the location of this return instruction.

3.3.2.2 Seed Selection

During the fuzzing process, SPECTRE retains only those test inputs that either cover new execution paths or new Params-Ret instructions as seeds. To track these discoveries effectively, we maintain two separate sets of records: *coverage_map* and *params_ret_map*. The *coverage_map* records unique execution paths covered by existing seeds, while the *params_ret_map* tracks unique Params-Ret instructions executed during fuzzing. After a test input is executed, the coverage and executed Params-Ret instructions are extracted from its execution trace and compared with *coverage_map* and *params_ret_map*. If the test input brings new code or Params-Ret instruction coverage, it is kept as a seed. Otherwise, it is discarded.

The seeds that reveal new paths are pivotal for enhancing code coverage and for thoroughly investigating the behavior of the library API; seeds that execute Params-Ret instructions are more likely to reveal aliasing relationships involving

parameters and return values. Seeds that do not fall into these two categories are discarded. This seed-selecting approach enables efficient exploration of aliasing specifications.

Example: For the example in Figure 3.4, assume the Params-Ret instruction in line 6 has not been covered by previous test inputs yet. Since *Input (1)* can trigger the execution of the return statement in line 6, it will be saved in the seed pool as a seed for generating new test inputs in the next iteration.

3.3.2.3 Power scheduling

A seed is kept based on whether it activates new paths or triggers Params-Ret instructions. Leveraging the *coverage_map* and *params_ret_map*, we categorize the seeds into three types:

- *Seed_1*: Seed that only triggers new paths.
- *Seed_2*: Seed that only triggers Params-Ret instructions.
- *Seed_3*: Seed that both triggers new paths and Params-Ret instructions.

Power scheduling in SPECTRE assesses seeds and allocates more energy to those showing high potential in producing test cases that activate diverse interactions between parameters and return values in library APIs. SPECTRE adopts *Energy (seed)* to calculate the energy of three different kinds of seed, where $P(seed)$ and $I(seed)$ represent the number of new paths and Params-Ret instructions triggered by the seed, respectively. While $\overline{P}_{mutant}(seed)$ and $\overline{I}_{mutant}(seed)$ represent the average number of new paths and new Params-Ret instructions triggered by testcases

mutated from the seed,

$$Energy(seed) = \begin{cases} \frac{\overline{P}_{mutant}(seed)}{P(seed)} + 1_{\{I(seed)==0 \wedge \overline{I}_{mutant}(seed)>0\}} & Seed_1 \\ \frac{\overline{I}_{mutant}(seed)}{I(seed)} + 1_{\{P(seed)==0 \wedge \overline{P}_{mutant}(seed)>0\}} & Seed_2 \\ \frac{\overline{P}_{mutant}(seed)}{P(seed)} + \frac{\overline{I}_{mutant}(seed)}{I(seed)} & Seed_3 \end{cases}$$

This equation describes the energy calculation for three different types of seeds. For the *Seed_1*, energy is calculated as the ratio of the average number of new paths triggered by the mutant test cases to the number of new paths triggered by the original seed. Additionally, if the original seed didn't trigger any Params-Ret instructions but the mutant test cases did, an extra unit of energy is added. For the *Seed_2*, energy is calculated similarly but focuses on the Params-Ret instructions triggered. An additional unit of energy is added if the original seed did not trigger any new paths but the mutant test cases did. For the *Seed_3*, the energy is the sum of the ratios of both the new paths and Params-Ret instructions triggered by the mutant test cases compared to the original seed. This approach allows for a dynamic allocation of energy based on the performance of mutant test cases relative to the original seeds.

Example: Consider the example in Figure3.4, *Input (1)* and *Input (2)* of lib_extractUserInfo() trigger the if and else branches respectively, but *Input (1)* also triggers the Params-Ret Instruction (line 6). According to the power scheduling strategy , *Input (1)* falls into the category of *Seed_3* while *Input (2)* belongs to *Seed_1*. Therefore, by applying the energy calculation equation, SPECTRE will assign more energy to *Input (1)* than *Input (2)*.

3.3.2.4 Seed Mutation

SPECTRE enhances traditional coverage-guided fuzzing with targeted mutation strategies specifically designed to expose potential aliasing relationships. Based on the observation from Table 2.1 that aliasing operations mainly involve dependencies and containment relationships among parameters and the return value, we propose three specialized seed mutators: (1) *A copy mutator* that copies a random segment of the seed content to overwrite another random segment within the same seed; (2) *A swap mutator* that exchanges two random segments of content within the seed; (3) *An append mutator* that appends a random segment of the seed to another random segment within the same seed.

Although traditional coverage-guided mutations also include similar operations, these mutations are random and generic, primarily aimed at maximizing code coverage rather than systematically exploring dependencies between arguments and return values. In contrast, the new mutators added by SPECTRE are highly targeted: they are only applied when the instrumented Param-Ret instructions are triggered during fuzzing, focusing on intensive manipulation of existing seeds to exploit the dependencies and containment relationships within parameters and return values. With the specialized mutators, SPECTRE can increase the likelihood of detecting relevant aliasing relationships, which might be missed by traditional coverage-guided fuzzers.

Example: For the example in Figure 3.4, SPECTRE can apply the *copy mutator* to *Input (1)* and copy the first four bytes from info to key, mutating key to "Mike". This could lead to the discovery of a new seed *Input (3)*, which can indicate the existence of an aliasing specification where info and the return value point to the same object.

3.4 Evaluation

C programs typically incorporate two categories of libraries: the C standard library and C third-party libraries. To conduct a thorough assessment of the performance of SPECTRE in inferring aliasing specifications, we employed datasets from both the C standard library and C third-party libraries. For the C standard library, some static analysis tools have integrated manually-written specifications due to their widespread use. This allows us to directly compare SPECTRE-generated specifications with existing manually-written ones. In contrast, C third-party libraries typically lack such ready-made manually-written specifications. Since our goal is to use aliasing specifications to replace source code of APIs called in programs during the static analysis, we use the static pointer analysis results with API source code as a baseline to evaluate the SPECTRE-generated specifications. If a program calls an API and static analysis is performed on this program, the static pointer analysis results on arguments or return values should be consistent whether we use the API source code or the SPECTRE-generated aliasing specification. Following the generation of aliasing specifications, we assessed their effectiveness in facilitating static pointer analysis of client code that uses these library APIs. In addition, we explored the effect of the tailored grey-box fuzzing features of SPECTRE. Therefore, our evaluation aims to answer the following research questions:

- **RQ1 (Generation Capability)** How effective is SPECTRE in generating aliasing specifications?
- **RQ2 (Usefulness)** How do SPECTRE-generated aliasing specifications benefit the static analysis of client code that uses library APIs?
- **RQ3 (Ablation Study)** How effective are the tailored grey-box fuzzing features of SPECTRE in improving aliasing specification generation?

- **RQ4 (Comparative Analysis)** How does SPECTRE compare with other methods?

In this section, we use three versions of SPECTRE to answer these research questions: **Spectre-Base** refers to the black-box fuzzing version with only the alias-check observer module, **Spectre-Cov** represents the traditional grey-box fuzzing version with the alias-check observer module, while **Spectre-Full** represents the complete implementation with traditional grey-box fuzzing, tailored fuzzing features, and the alias-check observer module.

We have implemented SPECTRE as an automated alising specification generation framework for C libraries. It includes an LLVM 14-based instrumentation module, a fuzzing module adapted from AFL++ [42], and an alias-check observer written in Python. The source code can be accessed at `https://zenodo.org/records/14649381`.

3.4.1 Experiment setup

Benchmarks. We select two sets of datasets for evaluation.

Table 3.1: Eight header files in Musl-1.2.4

#No	*#Name*	*#APIs*	*#Loc*
1	stdio.h	112	3.9k
2	stdlib.h	33	561
3	string.h	38	1.2k
4	math.h	144	17.7k
5	time.h	130	1.7k
6	strings.h	83	135
7	iconv.h	9	686
8	mntent.h	175	128

Table 3.2: Eight selected C third-party libraries

#No	*#Name*	*#Version*	*#APIs*	*#Loc*	*#Stars*
1	cJSON	1.7.15	112	3.4k	10.1k
2	zlib	1.3	84	43.1k	5.4k
3	libmagic	5.45	33	1.7k	1.2k
4	parson	1.5.3	144	2.7k	1.3k
5	utf8	1.0.0	38	1.7k	1.6k
6	miniz	3.0.2	175	9.2k	2.1k
7	jansson	2.14	95	13.1k	3.0k
8	lodepng	20230410	74	4.5k	2.0k

- **Musl C Standard Library.** Musl-1.2.4 [96] is a lightweight C standard library widely used in various Linux distributions and embedded systems, with many of its APIs having specifications written by static analysis researchers.

- **Eight third-party C/C++ libraries.** The selected libraries include *cJSON*, *zlib*, *libmagic*, *parson*, *utf8*, *miniz*, *jansson*, and, *lodepng*. These libraries offer specialized capabilities not inherently present in the C standard library. All of the selected libraries have either been evaluated by OSS-Fuzz [111] or have more than 1000 stars on GitHub. The selected libraries offer diverse functionality covering common use cases in C/C++ applications, including parsing, compression, and text processing. These libraries are widely used in real-world applications, making their specification generation valuable. Additionally, we selected libraries under active maintenance, ensuring they receive regular updates and security patches.

The datasets used in our evaluation consist of libraries with accessible source code. While having source code eliminates the need for black-box experiments in practice, we intentionally conducted experiments in both black-box and grey-box modes to rigorously evaluate the capabilities of Spectre. The black-box experiments (using only executable files for RQ1 and RQ2) are designed to simulate

closed-source scenarios and validate the performance and effectiveness of SPECTRE-*Base* in such settings. This experimental setup allows us to assess the ability of SPECTRE to generate specifications without source code, while leveraging the available source code to verify the correctness of the inferred specifications—something that would be difficult to achieve with truly closed-source libraries. On the other hand, the grey-box experiments (using source code for RQ3 and RQ4) demonstrate the additional benefits of SPECTRE-*Cov* and SPECTRE-*Full* when source code is available. Table 3.1 and Table 5.1 present the details of eight selected header files from Musl-1.2.4 and 8 third-party libraries, including the number of APIs (*#APIs*) and lines of code (*#Loc*) for each header file and library. For the third-party libraries, the version (*#Version*) and the number of stars on GitHub (*#Stars*) are also provided.

Baselines. We compared SPECTRE with the following three baselines to evaluate its performance:

- **Manually-written specifications (Section 3.4.2.1).** We use the manually-written specifications from the C/C++ static analyzer SVF [120] for C standard library APIs. These specifications are all related to pointer operations and have been extensively verified by researchers and users. We analyze 33 relevant aliasing specifications to compare against the SPECTRE-generated aliasing specifications.

- **Static analysis results of API source code (Section 3.4.2.2).** We use the static pointer results from the API source code as the baseline and compare them against the results derived from SPECTRE-generated aliasing specifications for the selected C third-party libraries. We employ SVF for this validation, which includes built-in API specifications for the C standard library, enabling

a direct comparison with SPECTRE-generated specifications. Additionally, SVF is a widely used C/C++ static analysis tool that provides advanced pointer analysis capabilities, making it suitable for our evaluation.

- **c-summary(Section 3.4.5).** c-summary [67] is a state-of-the-art static analysis tool that extracts semantic summaries of input and output relationships for C library functions. Originally designed for JNI programs, it performs interprocedural modular analysis and converts these summaries into Java code. We modified c-summary to extract semantic summaries for C functions without converting them into Java code. While other recent techniques [7, 17, 33, 38, 92] exist for automated specification generation, they are tailored for Java and rely on its language-specific features, making them unsuitable for our experiments.

Number of specifications. The number of APIs with specifications varies significantly across different libraries. This variation can be primarily attributed to two factors. First, APIs with null or non-pointer primitives like int or double typically lack relevant specifications. Second, even for APIs with pointer-type parameters and return values, specifications might still be absent. For instance, the API int printf(const char *format, …) takes a pointer as the first parameter, but it does not exhibit any aliasing specifications of interest. Therefore, we filter out library APIs that clearly lack specifications before fuzzing.

Fuzz driver. Fuzz drivers are used in the experiments to invoke the targeted library APIs. For simpler APIs, fuzz drivers were automatically generated using the *pycparser* [19] Python package. For complex APIs, such as those requiring intricate call sequences for usage, we adapt existing client code into fuzz drivers by leveraging *pycparser* to identify API call sites within the client code. We then extract the arguments and return values of the API at each call site and, following

Algorithm 1, insert alias-checking code after the API call to capture potential aliasing specifications.

k-limiting. The value of k used for k-limiting in the generation of field-access alias-check observers was set to 2. This choice stems from our observation that, for most scenarios, $k = 2$ sufficiently captures relevant aliasing patterns. In addition, aliases at access paths longer than 3 (e.g., node->next->next->next) can, through fuzzing, manifest at shorter paths (e.g., node->next). Thus, $k = 2$ sufficiently captures relevant aliasing patterns while maintaining efficiency.

Specification usage. The SPECTRE-generated specifications are simplified implementations of APIs that can be compiled like standard C code, making them compatible with any static analysis tool that handles C code. In the evaluation, the fuzzer drivers and the SPECTRE-generated specifications are first compiled separately into LLVM bitcode files. These IR files are subsequently linked using llvm-link to produce a unified bitcode file, which is then analyzed using static pointer analysis in SVF.

Configurations. Our evaluations were performed on an Intel Core i7-12700 12th Gen processor with 16GB RAM, running Ubuntu 22.04 LTS with Linux kernel 6.2.0-33-generic 64-bit. To ensure consistency and reliability, each fuzzing experiment was repeated five times, with each session lasting three hours. The results were aggregated by calculating the mean across the five runs. Based on observations, we empirically figured out that 24 hours is enough for the fuzzing process to converge for individual library APIs and produce effective specifications.

Table 3.3: Comparison of Manual and Spectre Methods on Musl-1.2.4. *#Alias-Specs* are the numbers of manually-written or Spectre-Base-generated aliasing specifications;

#Header	*#APIs*	*Manual*				Spectre-*Base*			
		#Alias-Specs	*#TP*	*#Miss*	*#Error*	*#Alias-Specs*	*#TP*	*#FN*	*#FP*
stdio.h	88	3	2	3	1	5	5	0	0
stdlib.h	87	7	7	4	0	10	10	1	0
string.h	45	16	14	2	2	16	16	0	0
math.h	209	0	0	0	0	0	0	0	0
time.h	31	4	4	1	0	5	5	0	0
strings.h	12	1	1	1	0	2	2	0	0
iconv.h	3	1	1	0	0	1	1	0	0
mntent.h	6	1	1	1	0	2	2	0	0
Total	***481***	***33***	***30***	***12***	***3***	***41***	***41***	***1***	***0***

Table 3.4: Specification generation on eight third-party libraries. *#Specs* are the total numbers of Spectre-*Base*-generated specifications

#Library	***#APIs***	***#Specs***	***#Pass***	***#Fail***
cJSON	78	18	18	0
zlib	84	5	5	0
libmagic	18	8	8	0
parson	103	14	14	0
utf8	38	14	14	0
miniz	117	5	5	0
jansson	95	16	16	0
lodepng	74	4	4	0
Total	607	84	84	0

3.4.2 Generation Capability

3.4.2.1 C Standard Library

Table 3.3 presents a comparison between manually-written aliasing specifications and SPECTRE-*Base*-generated aliasing specifications for APIs in 8 commonly used headers of the Musl-1.2.4 C standard library. *#TP* indicates the number of correct aliasing specifications; *Manual-#Miss/#Error* are the numbers of aliasing specifications that were not summarized and incorrectly summarized due to human oversight or error; *SPECTRE-Base-#FN/#FP* are the numbers of aliasing specifications that were not or incorrectly inferred by SPECTRE.

Out of 481 library APIs, 33 APIs had manually-written specifications, while SPECTRE-*Base*-generated specifications for 41 APIs. Among these headers, those with numerous APIs involving pointer type parameters and return values exhibit a relatively higher number of aliasing specifications. For example, *string.h* contains the highest number of aliasing specifications. Conversely, headers that either lack pointer type parameters and return values or primarily handle mathematical operations have fewer aliasing specifications, such as *math.h*, which has the lowest number of aliasing specifications.

We manually reviewed the aliasing specifications that differed between the manually-written set and SPECTRE-*Base*-generated set via the source code. Three inaccuracies (Manual-#*Error*) were found in the manually-written specifications, primarily due to incorrectly specifying the alias relationship between arguments and return values, such as specifying $Arg0_{api} \xrightarrow{alias} Ret_{api}$ as $Arg1_{api} \xrightarrow{alias} Ret_{api}$. We also identified one false negative in the SPECTRE-*Base*-generated specifications. The missed aliasing specification was from the API void *bsearch(void *key, void *base, size_t nitems, size_t size, int (*compare)(void *, void *)) in stdlib.h, the

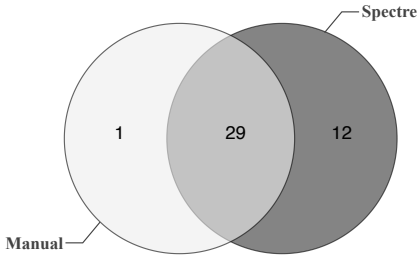


Figure 3.5: Distribution of correct aliasing specifications in the manual and Spectre-*Base* methods (Venn diagram) on Musl-1.2.4.

compare() was not correctly specified, which prevented it from executing properly during fuzzing. This led to the $Arg1_{bsearch} \xrightarrow{alias} Ret_{bsearch}$ being overlooked.

As illustrated in the Venn diagram of Figure 3.5, there is a significant intersection between the manually-written specifications and those generated by SPECTRE. SPECTRE-*Base* successfully recovered 29 out of 30 (96.7%) correct manually-written specifications. Furthermore, SPECTRE-*Base* identified 40.0% more correct aliasing specifications (12 additional to the 30 correct manually-written ones) than the manual method. Among these 12 new specifications, 8 are commonly used APIs in C programs, including strcasestr() for case-insensitive string searching, while the remaining 4 are less frequently used APIs such as fgets_unlocked(), which are rarely used due to their non-thread-safe nature. We have submitted a pull request to SVF, which includes corrections for the three incorrect specifications and 12 new specifications discovered by SPECTRE-*Base*. These corrections and new findings have been accepted and incorporated.

3.4.2.2 C Third-Party Library

Table 3.4 presents the results of SPECTRE-*Base* in generating aliasing specifications for C third-party libraries. The eight third-party libraries cover 607 APIs, with 84 specifications generated in total. *#Pass* are the specifications that passed static

analysis results compared to using API source code; *#Fail* are the specifications that produced inconsistent results. The library with the most specifications is cJSON, with 18 specifications, followed closely by jansson with 16 specifications. In contrast, lodepng has the least number of specifications, with only 4 aliasing specification generated.

Given the absence of manually-written specifications for these third-party libraries, we validated the SPECTRE-*Base*-generated specifications by comparing their static pointer analysis results on API arguments and return values with those directly derived from the API source code. Consistency between the two sets of results indicated the correctness of the SPECTRE-*Base*-generated specifications. From the table, all 84 specifications generated by SPECTRE-*Base* were validated successfully, with no discrepancies found between the generated specifications and the static analysis of the original API source code. This consistency demonstrates the reliability and accuracy of SPECTRE-*Base* in capturing the correct aliasing relationships for C third-party libraries.

In summary, for the Musl C standard library, SPECTRE-*Base* successfully recovered 96.7% of correct manually-written specifications and identified 40.0% more correct aliasing specifications than the manual method. Additionally, SPECTRE-*Base* helped identify and correct three inaccuracies in the manually-written specifications. For C third-party libraries, static pointer analysis using SPECTRE-*Base* specifications are all consistent with those using API source code. Overall, SPECTRE-*Base* demonstrates its effectiveness and reliability in generating aliasing specifications across different types of C libraries.

3.4.3 Usefulness

Table 3.5: Comparison of static pointer analysis performance using four different approaches for handling library APIs.

Library	*Stubs*	*Alias-Empty*		*Alias-Manual*		*Alias-Source*		*Alias*-Spectre-*Base*	
		#ValidAlias	*#Time(s)*	*#ValidAlias*	*#Time(s)*	*#ValidAlias*	*#Time(s)*	*#ValidAlias*	*#Time(s)*
Musl-1.2.4	54	0	7.45	48	7.61	54	45.78	52	7.65
Third-party	108	0	10.26	-	-	108	38.56	108	10.53

To show that SPECTRE-*Base*-generated aliasing specifications have a place in static pointer analysis on client code calling APIs, we conducted a comparative analysis of four different methods for handling library APIs in static analysis. The results are presented in Table 3.5. The four methods compared are *Alias-Empty*, *Alias-Manual*, *Alias-Source*, and *Alias-SPECTRE-Base*. *Alias-Empty* removes SVF manually-written aliasing specifications, assuming API calls have no impact on client code. *Alias-Manual* uses SVF manually-written specifications, which only cover the C standard library and not third-party libraries. *Alias-Source* utilizes the source code of the APIs. *Alias-SPECTRE-Base* employs SPECTRE-*Base*-generated aliasing specifications. Two metrics were used to evaluate the effectiveness and efficiency of aliasing specifications: (1)*#ValidAlias*, the number of aliasing relationships in the generated specifications was correctly validated using "whole-program" static analysis. For validation, we use the SVF tool to analyze the entire source code base (application + library code). The alias results from SVF are used as the ground truth to validate whether the same aliasing relationships exist in the generated specifications. (2)*#Time*, the duration required to complete the static pointer analysis. The experiment covered both the C standard library and third-party libraries. For client code selection, we used the test suites of Musl and SVF for the C standard library analysis, applying 54 stub functions. For third-party

libraries, we utilized the test cases provided by the libraries themselves, applying 108 stub functions.

The validation process uses stub functions that compute points-to sets of pointer variables. When the API source code indicates that two pointer variables are aliases (i.e., their points-to sets have a non-empty intersection), our specifications must correctly identify this same aliasing relationship. The #ValidAlias metric counts how many aliasing relationships our specifications correctly validate when compared against the source code. Therefore, when our specifications correctly validate all aliasing relationships present in the source code, they can effectively replace the source code in static pointer analysis while enabling faster analysis times. This shows the usefulness of our specifications: by providing correct aliasing information, they avoid the need for expensive source code analysis, improving efficiency.

The *Alias-Source* method is undoubtedly the most effective, as it contains complete API information, enabling full validation of all aliasing relationships with stub functions. However, it had the longest analysis time compared to the other three methods. The *Alias-Empty* method provided no information about the APIs and optimistically assumed they have no impact on the client code. This approach is clearly unsound, as it completely ignores potential aliasing relationships created by API calls. While it results in the shortest analysis time, the stub functions failed to capture any aliasing relationships between variables caused by these APIs.

The *Alias-Manual* and *Alias-*Spectre*-Base* methods utilize SVF manually-written specifications and Spectre-*Base*-generated specifications, respectively. *Alias-Manual* identified 48 successful cases, while *Alias-*Spectre*-Base* discovered 52. The lower success rate of *Alias-Manual* is attributed to its fewer specifications and the inclusion of three false positives. *Alias-*Spectre*-Base* missed two successful cases compared to *Alias-Source* due to the overlooking specification for

bsearch(), which was used twice in the client code. However, *Alias-SPECTRE-Base* significantly reduced analysis time compared to *Alias-Source*, demonstrating its ability to maintain high efficiency while providing correct analysis.

In summary, SPECTRE-Base-generated aliasing specifications improve the static pointer analysis of client code using library APIs, achieving good accuracy with shorter analysis time.

3.4.4 Ablation Study

Table 3.6: Performance of Spectre-Base, Spectre-Cov, and Spectre-Full on Musl-1.2.4 and third-party libraries.

Musl-1.2.4				***Third-party libraries***			
Header	**Spectre-*Base***	**Spectre-*Cov***	**Spectre-*Full***	***Library***	**Spectre-*Base***	**Spectre-*Cov***	**Spectre-*Full***
stdio.h	5	6	6	cJSON	18	21	23
stdlib.h	10	12	14	zlib	5	5	5
string.h	16	17	19	libmagic	8	8	8
math.h	0	0	0	parson	14	16	18
time.h	5	5	5	utf8	15	15	15
strings.h	2	3	3	miniz	5	7	8
iconv.h	1	1	1	jansson	16	18	22
mntent.h	2	2	2	lodepng	4	5	6
Total	41	46	50	-	85	95	105

Table 3.6 compares the results of SPECTRE-*Base*, SPECTRE-*Cov*, and SPECTRE-*Full* on Musl-1.2.4 and eight third-party libraries (excluding false positives). These variants differ in their components. SPECTRE-*Base* operates in black-box mode and includes only the alias-check observer module. SPECTRE-*Cov* operates in grey-box mode, combining the alias-check observer module with traditional coverage-guided fuzzing features. SPECTRE-*Full* builds upon SPECTRE-*Cov* by incorporating all its components and adding tailored fuzzing features.

In the Musl-1.2.4, SPECTRE-*Base* identified 41 specifications across the various headers. When incorporating the traditional coverage-guided features, SPECTRE-*Cov* increased the number of specifications to 46. This implies that the coverage-

guided strategy was able to explore more paths and hence identify more aliasing relationships. The number of specifications further increased to 50 when using SPECTRE-*Full*, which employs tailored aliasing fuzzing features. This suggests that the aliasing fuzzing features were able to further explore the aliasing relationships in the library, leading to a more complete set of specifications. In addition, compared to SPECTRE-*Base*, SPECTRE-*Full* discovered more specifications that were not manually summarized in SVF (21 versus 12 for SPECTRE-*Base*). These additional 9 new specifications were also submitted to SVF and were accepted. In the third-party libraries, SPECTRE-*Base* identified 85 specifications. The number of specifications increased to 95 when using SPECTRE-*Cov*, and further increased to 105 when using SPECTRE-*Full*. This again demonstrates the effectiveness of the coverage-guided and tailored aliasing fuzzing features in exploring the aliasing relationships.

To ensure that the observed improvements were not merely due to experimental variability, we conducted Mann-Whitney U tests [62] on the results from 5 independent runs of each tool variant. For Musl-1.2.4, the improvement from SPECTRE-*Base* to SPECTRE-*Cov* was statistically supported ($U = 0.0$, $p = 0.0079 < 0.01$), indicating that the coverage-guided features effectively enhanced specification discovery. Similarly, the improvement from SPECTRE-*Cov* to SPECTRE-*Full* was also statistically supported ($U = 0.0$, $p = 0.0117 < 0.05$), demonstrating the benefits of our tailored fuzzing features. The comparison between SPECTRE-*Base* and SPECTRE-*Full* revealed a consistent overall improvement ($U = 0.0$, $p = 0.0117 < 0.05$).

For third-party libraries, we observed consistent statistical support for the improvements. The transition from SPECTRE-*Base* to SPECTRE-*Cov* showed a statistically supported improvement ($U = 0.0$, $p = 0.0079 < 0.05$), as did the transition

from SPECTRE-*Cov* to SPECTRE-*Full* (U = 0.0, p = 0.0079 < 0.01). The overall improvement from SPECTRE-*Base* to SPECTRE-*Full* was similarly supported (U = 0.0, p = 0.0079 < 0.05). These results strongly indicate that both the coverage-guided features and the tailored fuzzing features play a crucial role in enhancing the tool's effectiveness in discovering aliasing specifications.

Overall, the results suggest that the source-based approach (SPECTRE-*Cov*) is more effective than the source-unavailable approach (SPECTRE-*Base*) in generating aliasing specifications. The tailored aliasing fuzzing features in SPECTRE-*Full* further enhance the effectiveness of the source-based approach. However, SPECTRE-*Base* also demonstrated considerable effectiveness. In the Musl standard library, SPECTRE-*Base* identified 41 out of 50 specifications discovered by SPECTRE-*Full*, which is approximately 82.0% of the total. Similarly, in the third-party libraries, SPECTRE-*Base* identified 85 out of 105 specifications discovered by SPECTRE-*Full*, which is roughly 81% of the total. This highlights the effectiveness of the alias-check observer module employed by SPECTRE-*Base*, even without access to the source code of APIs.

In summary, SPECTRE demonstrates effectiveness in identifying aliasing specifications both with and without source code access. When source code is available, the tailored aliasing fuzzing features help uncover more specifications in library APIs. When source code is inaccessible, Spectre still identifies approximately 80% of the aliasing specifications found in source code available scenarios.

3.4.5 Comparative Analysis

Figure 3.6 compares the specification generation capabilities of *c-summary* (a static analysis-based specification inference tool) and *SPECTRE-Full* across Musl-1.2.4

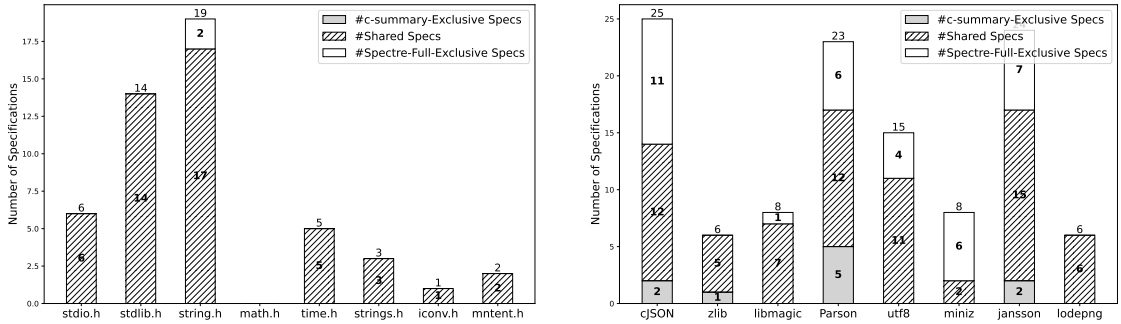


Figure 3.6: Comparison of specifications generated by *c-summary* and Spectre-*Full*.

and third-party libraries. The figure focuses on three metrics: *#c-summary-Exclusive Specs*, which represents the number of specifications generated exclusively by *c-summary*; *#Shared Specs*, which denotes the specifications discovered by both tools, reflecting their overlap; and *#Spectre-Full-Exclusive Specs*, which indicates the number of specifications generated solely by *Spectre-Full*.

In Musl-1.2.4, both tools demonstrate comparable performance across most standard headers, with *Spectre-Full* showing slightly higher coverage (50 specifications) compared to *c-summary* (47 specifications). The only notable difference appears in string.h, where *Spectre-Full* generated 19 specifications, 2 more than *c-summary* generated 17. In third-party libraries, *Spectre-Full* demonstrated broader coverage with 105 specifications compared to *c-summary* generating 90, representing a 16.7% increase in specifications discovered by *Spectre-Full*.. The two tools shared 70 specifications, while *Spectre-Full* had 35 exclusive specifications and *c-summary* had 10 exclusive specifications.

The performance gap between the tools can be attributed to limitations in *c-summary* handling complex C language constructs. As a static analysis tool, *c-summary* only supports a subset of C language features, struggling with sophisticated elements such as global variables, arrays, and arithmetic operations. This limitation is illustrated in the case of the strtok() function from string.h in Musl-

1.2.4, where *c-summary* fails to generate a specification due to its inability to handle complex pointer arithmetic and offset operations.

While *Spectre-Full* shows better coverage and does not suffer from the language feature limitations as the static analysis of *c-summary*, it faces challenges in capturing certain alias behaviors due to the inherent nature of dynamic analysis. Specifically, fuzzing may not execute all possible program paths, leading to missed specifications that are not encountered during execution. This limitation is evident in the 10 specifications that *Spectre-Full* missed but were successfully identified by *c-summary* across all third-party libraries. An example is the `json_object_get_value(object, name)` API function in the `parson` library, where *c-summary* successfully identified that it returns a specific element from the `object->value` array. *Spectre-Full* failed to capture this specification because during fuzzing, the specific case of returning the first array element never occurred. This case is particularly challenging for fuzzing to trigger as it requires constructing a valid JSON object with specific key-value pairs and then precisely matching the key name during the API call. This highlights a limitation of dynamic analysis in capturing aliasing behaviors not encountered during execution.

In summary, *Spectre-Full* demonstrates superior performance in generating API specifications compared to *c-summary*, but it may miss certain specifications in cases where specific code paths are not executed during analysis.

3.5 Related Work

Static analysis is a technique that examines source code without executing the program to detect potential errors, vulnerabilities, and coding issues. Access to all program source code is commonly assumed in static analysis. However, this assump-

tion is challenged as programs increasingly rely on third-party libraries that are often too complex to analyze or have no access to the source code. Without analyzing library source code, three approaches are commonly used to handle library calls in static analysis: the optimistic approach assumes library APIs do not cause adverse effects, simplifying analysis but potentially underreporting issues [18]; the pessimistic approach assumes library APIs might do anything, ensuring soundness but may render results useless [148]; using specifications to summarize the behaviors of library APIs offers a middle ground. Manually crafted specifications provide more precision and flexibility but are time-consuming. Therefore, many methods have been proposed for automatically inferring specifications [1, 16–18, 33, 59, 92, 148].

3.5.1 Specification Inference

Specification inference is the task of inferring or modeling the specifications that describe the behavior, constraints, and properties of a program or a function. Existing work for specification inference can be mainly classified as static or dynamic. The static methods [16, 38, 83, 92, 105, 115, 148] analyze the source code without executing it, inferring specifications by examining the code structure, variable types, data flow, and control flow. The dynamic method [7, 17, 33] involves observing the program's behavior during execution to infer specifications through techniques like runtime monitoring, instrumentation, and profiling. Additionally, some work leverages API documentation or API usage within application code to infer specifications [38, 132]. The kinds of specifications include, but are not limited to, control-flow [2, 13, 14, 85, 139], taint analysis [16, 33, 118], points-to analysis [12, 17, 18, 21, 37, 76], and assertions [92]. In the previous works, [38] and [132] are the closest to our approach. *However, both approaches focus on Android li-*

braries, whereas SPECTRE *targets C libraries, making direct comparisons in the evaluation infeasible.*

3.5.2 Fuzzing

Fuzzing is a widely adopted dynamic software testing technique, proven highly effective in discovering software bugs, crashes, and potential security vulnerabilities. By generating and applying diverse inputs to a targeted system, fuzzing can uncover hidden defects that traditional testing methods might miss. Recent research, such as [92], has demonstrated the potential of using fuzzing to generate class specifications through automated assertion inference. While their work focuses on class specifications, our work targets a more specialized domain: API aliasing specifications. Since current grey-box fuzzing techniques predominantly aim to maximize code coverage, their objective that does not always align with aliasing specification inference. To address this limitation, we introduce SPECTRE, a novel approach specifically designed with an alias-check observer and innovative fuzzing features to capture and generate comprehensive aliasing specifications.

3.6 Chapter Summary

This chapter presents SPECTRE, an innovative tool based on fuzzing for automatically generating aliasing specifications for library APIs. The primary advantage of SPECTRE lies in its ability to infer specifications without the need to access API source code. By introducing alias-check observers, SPECTRE effectively extracts aliasing relationships from API executions. Furthermore, when API source code is available, the custom fuzzing features in SPECTRE- including tailored instrumentation, seed selection, power scheduling, and seed mutation strategies - further

enhanced specification generation efficiency, resulting in more generated specifications compared to environments where source code access is not possible. Our experiments demonstrate that SPECTRE performs effectively in generating correct aliasing specifications. When tested on the C standard library, SPECTRE successfully recovered the vast majority of correct manually-written specifications and identified a significant number of additional correct aliasing specifications compared to the manual method. In addition, SPECTRE helped identify and fix three incorrect specifications in the manually-written specification set. For third-party libraries, all of the specifications were validated correct by static analysis of the API source code. The SPECTRE-generated aliasing specifications improve the static pointer analysis of client code that uses these library APIs and maintains correctness and efficiency. The experimental results demonstrate that SPECTRE surpasses existing specification inference tool, generating 16.7% more aliasing specifications in third-party libraries. In summary, SPECTRE offers a practical approach to generating aliasing specifications of APIs, functioning effectively whether source code is available or not.

Chapter 4

Cross-Language Native Code Specification Generation

4.1 Overview

This chapter introduces *Caller-Sensitive Specification* (CSS), a new approach designed to enhance the static taint analysis of Java programs employing JNI to interface with C/C++ code. This approach takes into consideration the calling context of the invoked C/C++ functions (or cross-language context), resulting in more precise and concise specifications for the side effects of native code. Furthermore, CSS specifically enhances the capabilities of Java analyzers, enabling them to perform precise static taint analysis across language boundaries into native code. We have conducted comprehensive experiments aimed at comparing the value-flow analysis using CSS in JNI programs. Through the application of CSS, existing Java static analyzers demonstrate enhanced performance in tracing value-flow within JNI programs, outperforming current cross-language approaches. The experimental results show that CSS can accurately summarize value-flow informa-

tion and enhance the ability of Java monolingual static analyzers for cross-language taint flow tracking.

4.1.1 Motivation

Cross-language programming is a common practice within the software development industry, offering developers a multitude of advantages such as expressiveness, interoperability, and cross-platform compatibility for developing large-scale applications. As an important example, Java employs the Java Native Interface (JNI) to interact with code written in other programming languages, such as C or C++, enabling access to functionalities like those in C/C++ libraries such as OpenCV. However, conventional static analysis based on a single programming language faces challenges when it comes to tracing the flow of values across multiple modules that are coded in different programming languages. Moreover, JNI's use introduces potential security vulnerabilities [24, 54, 64, 70, 73, 77, 79, 91, 109], particularly concerning in the context of sensitive data exchanges, where malicious information flows can infiltrate from an app to native code by exploiting security vulnerabilities through JNIs, enabling activities such as identity theft and unauthorized account access [24, 54, 64, 109, 125].

4.1.2 Challenges

When undertaking cross-language static analysis, there are typically two strategies to consider. The first strategy involves compiling different programming languages into a singular intermediate representation (IR) [22, 77, 95, 101, 108, 127]. The limitation of this approach is the intrinsic and distinct features among different programming languages, creating a common IR to encompass programs written in distinct languages can be very challenging. Another limitation is the potential loss

of information and inconsistencies when using a unified IR for analyzing different languages. In these tools [11,44,55], different languages are compiled into a unified IR separately. To accommodate multiple languages, compromises in the unified IR are often made due to separate compilation for distinct semantics of each language. Representing precisely the semantics at the border between two languages is challenging. For example, in JNI programs, Java and C/C++ have static and dynamic binding modes and distinct naming conventions. In addition, C/C++ consistently represents all Java objects as *jobject* without distinguishing specific Java object information. Even if a unified IR representing both Java and C/C++ code is used for later analysis, it may still fail to produce the desired results due to the loss of *jobject* information. The unified IR has to preserve all these API behaviors. As high-level languages evolve, modifying the design or implementation of the unified IR is also challenging. Accommodating all possible behaviors from high-level languages by modifying the unified IR is very hard and sometimes cost-ineffective.

The second strategy utilizes specifications (including summaries and stub functions) to encapsulate the value-flow side-effects of native methods [77,112,148]. The specification-based approach represents another way of performing cross-language analysis where the existing single-language analyzers (e.g., Java analyzers) together with their various analysis algorithms can be reused with light or even no modifications. The specification-based approach, which centers on analyzing a summarized or condensed version of a library function rather than the entire function, has demonstrated its practicality in real-world cross-language and cross-module static analysis scenarios. However, automatically generating highly precise specifications for large-sized programs that involve multiple languages can pose significant challenges. This is due to the fact that specifications heavily depend on the analyzers, such as different value-flow or vulnerability checkers. The current cross-language

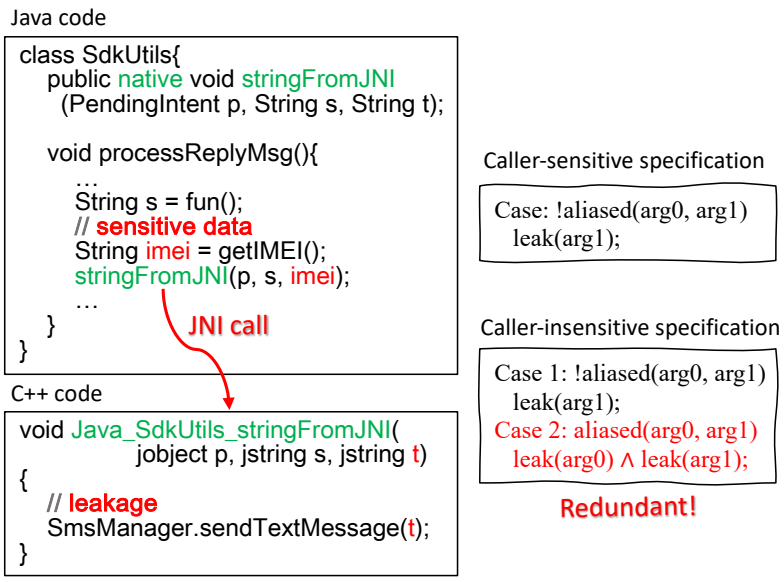


Figure 4.1: A data leakage JNI example (simplified)

static analysis approach often entails the creation of specifications either manually or automatically for each language independently, without considering the cross-language calling contexts, like calling context in Java that makes use of native libraries.

To illustrate, Figure 4.1 is a simplified Android case of sensitive information leakage within the *Gumen* family in the Android Malware Dataset [137]. This case shows data leakage by transmitting the device's IMEI from the Java layer's *processReplyMsg()* method to the C++ code's *Java_SdkUtils_stringFromJNI()* function (the counterpart of *stringFromJNI()* method in the Java code). A conventional approach to specification generation prioritizes analysis soundness by accounting for all conceivable caller information and calling contexts. Nonetheless, such specifications frequently suffer from redundancy and imprecision. For example, in the initial caller-insensitive specification, the "*Case 2*" branch within the C++ program's specification becomes redundant if it's established that there is only one calling context for *stringFromJNI()* from the Java program callers. A more concise

and precise caller-sensitive specification can be created, ensuring that s and t are always non-aliases. This accurately summarizes all feasible execution scenarios by considering all available code in both the Java and C++ programs. Our insight is that specifications for available source code (written in multiple languages) benefit from cross-language caller-sensitive summarization, which produces more precise and compact specifications than a single-language (e.g., C/C++) based conservative approach. Given that JNI interactions often require C/C++ code to invoke Java methods and access fields while relying on Java class or object information, conducting an independent analysis of C/C++ to over-approximate all possible input to native code can lead to the generation of redundant and imprecise specifications (as detailed in Section 4.2.0.1). Note that specifications may need to be updated if the available source code including caller information is changed.

4.1.3 Our Solution

We introduce *Caller-Sensitive Specification* (CSS), a new approach to generate precise and concise specifications for native calls within JNI environments. Given its automated specification extraction approach, CSS aims to improve efficiency and reduce the size of specifications to track value-flows across JNI programs. Unlike the conventional approach of analyzing isolated C/C++ functions, our caller-sensitivity pertains to contextual information (including aliases, method signatures, and tainted flows) across language boundaries, that is when a native callee is called by Java caller methods. The soundness of CSS is based on the assumption that all source code is available. This strategy enables us to identify the data attributes existing at the boundary between Java and C/C++ interactions – for instance, direct and indirect value-flows via parameter passings and returns. Subsequently, utilizing insights provided by the Java analyzer, the C/C++ static analyzer contributes

to the extraction of value-flows of the corresponding native C/C++ functions. The outcomes of the C/C++ function's value-flows are conveyed back to the Java analyzer in the form of a specification, namely CSS, facilitating the iterative value-flow analysis.

Incorporating the cross-language context of called native functions into summarization can reduce redundancy with improved analysis efficiency and reduced storage space. Furthermore, our automated CSS approach can be used as a compact program representation for existing single-language static analyzers to aid in cross-language modeling. This compatibility empowers static analyzers to seamlessly execute effective cross-language static analysis with slight code adjustments. Though more precise given caller-sensitive information, the soundness of CSS is based on the availability of both Java and native source code. The specifications interface, capable of accommodating methods from diverse languages, demonstrates its versatility by supporting new languages without necessitating extensive modifications.

In summary, the following are our contributions:

- We introduce CSS, a new caller-sensitive specification approach designed to enhance the static value-flow analysis of JNI programs. This approach, which takes into account the calling context across languages, yields precise and compact specifications for efficient cross-language analysis.

- CSS integrates smoothly with Java static analysis tools specifically for JNI contexts. The incorporation of our proposed caller-sensitive specifications enables these tools to handle Java/Native interactions more efficiently within their existing frameworks.

- We have conducted comprehensive experiments aimed at comparing the value-flow analysis using CSS in JNI programs. Through the application of CSS, existing Java static analyzers demonstrate enhanced performance in tracing value-flow within JNI programs, outperforming current cross-language approaches.

4.1.4 Framework Overview

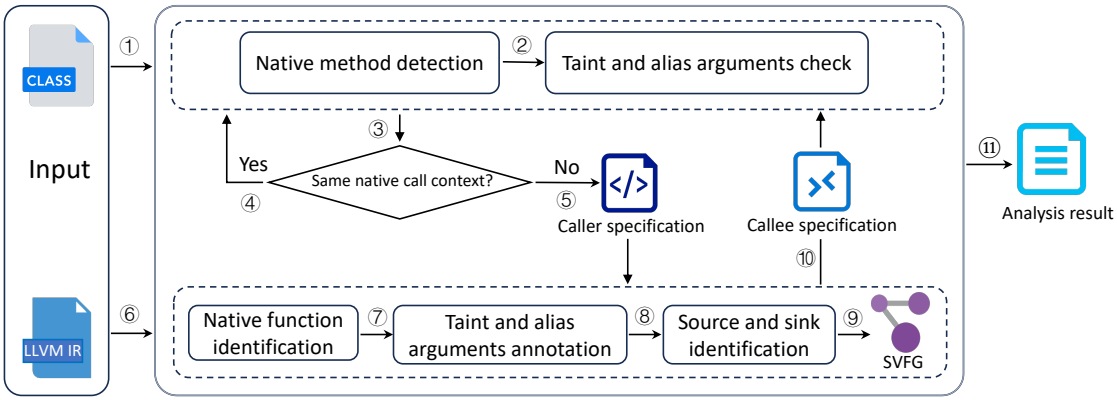


Figure 4.2: The workflow of our cross-language taint analysis with caller-sensitive specifications

Figure 4.2 illustrates our taint analysis workflow with CSS for JNI programs. It starts by analyzing the bytecode to identify tainted data sources. To demonstrate the generalization of our CSS summarization, we use recent popular taint analyzers Flowdroid [9], Tai-e [128] and, WALA [44] for Android and Java. Analyzers are employed to track the flow of tainted data and gather contextual information for specifying native method callers. Upon finding a called native method (Step ①) with tainted arguments (Step ②), it checks whether the same calling context has been previously analyzed (Step ③). If the same calling context analysis has been performed before, the Java static analyzer will reuse the pre-generated na-

tive call specification to reduce redundant analysis (Step ④). Otherwise, a caller specification (Step ⑤) is generated to record the current native calling context.

The caller specification contains method signature, and information about tainted and aliased arguments. For callee specification, we use SVF, a C/C++ static analyzer [120] to combine this information with LLVM IR to identify native functions (Step ⑥) and annotate tainted and aliased arguments (Step ⑦). Sources and sinks in native code are identified based on caller specifications and user configurations (Step ⑧).

The Sparse Value-Flow Graph (SVFG) [122] (Step ⑨) represents interprocedural value-flows across C/C++ functions, helping identify paths from native sources to native sinks. A caller-sensitive specification for each callee is generated using SVFG (Step ⑩), including native method signatures, callbacks, native sources of tainted data and sinks. This specification is then sent back to the Java static analyzer to continue the analysis. As illustrated in Figure 5.1, the process of generating caller and callee specifications is conducted in an iterative manner until a fixed point is reached (Step ⑪).

4.2 A Motivating Example

Figure 4.3 illustrates a JNI example demonstrating how sensitive data propagates between Java (Figure 4.3(a)) and C++ (Figure 4.3(b)). The interface *void native_c_callee(User, User)*, declared with the "*native*" keyword, serves as the entry point for the Java code to execute the corresponding C++ function. The function *void Java_JavaCaller_native_c_callee (JNIEnv*, jobject, jobject, jobject)* (Lines C2 and C3) represents the C/C++ function implementation of this native method, which will handle the data that passed from the Java side.

The method *caller()* (Line J3) in the Java side loads the sensitive user data (Line J5) from the request and stores it in an object of *User*. In addition, another object *loginUser* will load and store the object *user* (Line J7). Because the *loginUser* and *user* refer to the same memory, *loginUser.username* and *user.username* are aliased. Finally, the C++ function will be invoked at Line J8 by calling the native method *native_c_callee()* with two aliased arguments *user* and *loginUser*.

In C++ function *void Java_JavaCaller_native_c_callee*(), there exists a potential data leakage of sensitive source data. The *username* of the second parameter, *b*, is obtained through the three JNI functions at Lines C7-9 and then stored into the *userInfo* of object *model* (Line C10), ultimately being leaked to a log file by calling *Log.print()* at Line C11. Since *b.username* and *a.username* are aliased, *a.username* is also leaked when calling *Log.print()*. Next, we will briefly compare CSS and other approaches when building native summarizations to trace sensitive data and detect information leakages across languages.

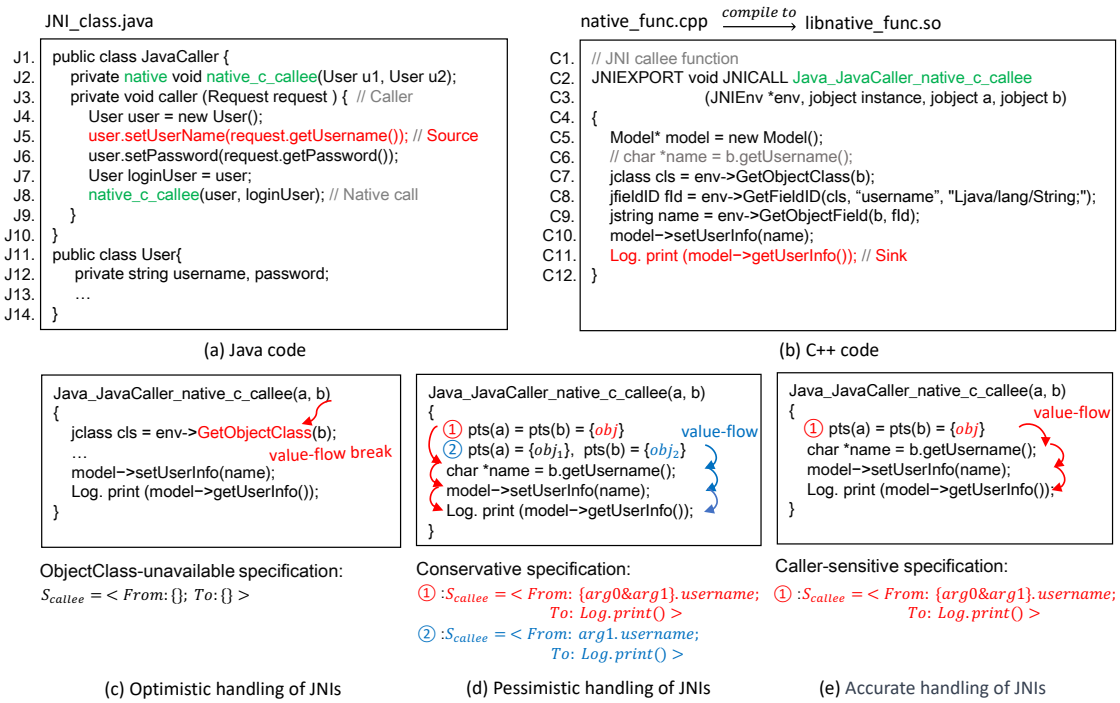


Figure 4.3: An illustrating JNI example with Java and native code

4.2.0.1 Taint analysis with caller-insensitive specification (CIS)

Conventional Java static analyzers typically conduct a manually defined solution for native library summarizations. These approaches require developers or users to provide value-flow specifications that identify source-to-sink paths of the native functions for the analyzers [52, 128]. However, ensuring the accuracy of manually defined specifications can be challenging because they are often isolated from the cross-language calling context, which is caller-insensitive (Caller-insensitive Specification, CIS), and mainly focuses on analyzing the behaviour of native functions. As a result, the accuracy of such specifications is hard to guarantee. Furthermore, these manually defined specifications either selectively consider certain cases (such as excluding alias situations) or include all possible taint and alias situations.

Optimistic handling of JNIs. If the analysis is conducted exclusively on *Java_JavaCaller_native_c_callee*(), the first challenge is JNI functions. As the C/C++ analyzer encounters *getObjectClass(b)* at Line C7, it faces difficulty in obtaining information about the object *b*, which results in a broken value-flow of the taint analysis at this line. This hinders the creation of a valid source-sink specification, as depicted in the ObjectClass-unavailable specification in Figure 4.3(c). This optimistic approach assumes no side effects on unresolved JNI functions, simplifying analysis but risking unsound results.

Pessimistic handling of JNIs. Even if a C/C++ analyzer has access to information about object *b*, in order to maintain soundness, the pessimistic or conservative approach assumes that C/C++ code can execute any action. A conservative specification would include the following two alias cases for the function *Java_JavaCaller_native_c_callee()*:

Case 1: alias(arg0, arg1) == true (red value-flow ①)

$S_{callee} = \langle From : \{arg0\&arg1\}.username, To : Log.print() \rangle$

Case 2: alias(arg0, arg1) == false (blue value-flow ②)

$S_{callee} = \langle From : arg1.username, To : Log.print() \rangle$.

For the conservative specification in Figure 4.3(d), *pts(v)* represents the points-to set of variable v, Lines C7-9 are simplified to *char *name = b.getUsername();* for easy understanding. However, these redundancy cases in CIS can impact the analysis efficiency if the conditions are complicated in large-size summarizations, and it could also lead to over-tainting and impact the performance of the analysis.

4.2.0.2 Taint analysis with Caller-sensitive Specification (CSS)

A taint analyzer with CSS typically comprises a Java analyzer and a C/C++ analyzer, each responsible for analyzing the specific programming language. The Java analyzer first performs taint analysis on the Java side and traces the propagation of sensitive data. In this example, the source data of the *username* loaded from the *request* is marked and traced by the Java taint analyzer. Due to *loginUser* and *user* are made to be aliases at Line J7, and thus *loginUser.username* is also tainted. When calling native method *native_c_callee*(), as the Java analyzer lacks traceability to C/C++ code, the taint analysis will be handed over to the C/C++ analyzer. Prior to this, the Java analyzer will generate a caller specification $S_{caller} = \langle\ taint : \{arg0\&arg1\}.username;\ alias : \{arg0, arg1\};\ argTypes : \{User, User\} \rangle$ that includes cross-language context information about this native call.

The C/C++ analyzer then takes over the analyzing task using the caller specification S_{caller}. Because S_{caller} contains the object information of b (i.e., *User* in the caller), the functionality of three JNI functions can be successfully parsed, leading to the retrieval of b's username. *b.username* is ultimately leaked to the *Log.print()* function. Since S_{caller} indicates that *b.username* and

a.username are aliased, *a.username* also leaks to the *Log.print()* function. When the analysis on the C/C++ side is completed, a specification representing the taint analysis result in the C/C++ side of the callee is generated in the form of $S_{callee} = \langle From : \{arg0\&arg1\}.username; To : Log.print()\rangle$ (Caller-sensitive specification in Figure 4.3(e)). This generated specification is then sent back to the Java analyzer for further taint tracing. The above value-flow analysis process reveals that accurate value-flow for C/C++ functions is achieved given the knowledge of cross-language contexts.

4.2.0.3 Taint analysis with JN-SAF and JuCify

JN-SAF [138] is an Android static analysis framework that handles native method calls. It constructs the call graph from *request.getUsername()* to *Log.print()* and performs taint analysis. While JN-SAF can identify the source-to-sink path in this small example, its scalability is constrained for larger programs due to path explosion problems with Angr's [116] symbolic execution. Moreover, it may yield inconsistent results when dealing with binary code compiled at different optimization levels [6]. Another Android static analysis framework, JuCify [108], aims to construct a unified model for Android code. However, Jucify is unable to reconstruct the behavior of pure native functions. Consequently, its tainted flow is broken at *setUserInfo()* and *print()*. These limitations affect the accuracy and scalability of cross-language static analysis in these tools.

4.3 Approach

In this section, we provide details for how to extract caller-sensitive specifications to assist in JNI program static analysis.

$\mathbb{P}$ $(parameter) = \{p_i \mid p_i \in \mathbb{T}, 0 \leq i < n\}$
$\mathbb{S}$ $(signature) = \{(m, r, P) \mid m \in \mathbb{M}, r \in \mathbb{T}, P \in \mathbb{P}\}$
$\mathbb{D}$ $(taintArgs) = \{a_i \mid taint(a_i) = true, 0 \leq i < n\}$
$\mathbb{A}$ $(aliasArgs) = \{(a_j, a_k) \mid alias(a_j, a_k) = true, 0 \leq j \neq k < n\}$
$S_{caller} = \{(S_e, D, A) \mid S_e \in \mathbb{S}, D \in \mathbb{D}, A \in \mathbb{A}\}$

Figure 4.4: Three-tuple caller specification

4.3.1 Caller Specification Extraction on Java Side

We employ alias analysis and taint analysis in a Java taint analyzer (e.g., FlowDroid, Tai-e and, WALA) to construct caller specifications. In these tools, a forward taint analysis is integrated with an on-demand alias analysis. A native callee function can be invoked by multiple callers from Java code with each call involving different arguments being passed to the callee's parameters. Please note that our caller-sensitive approach, unlike the traditional context-sensitive analysis, focuses on language boundaries, where the caller-sensitive contextual information transferred to native code can be computed through any (context-insensitive or context-sensitive) Java alias and taint analysis. We consider the argument information passed from the Java side caller before extracting value-flow specifications for C/C++ side functions. By following this approach, we can generate specific specifications that precisely reflect the program's behavior, as opposed to relying on a single, all-encompassing conservative specification that might fail to accurately capture the caller-sensitive behaviors.

The analysis begins by identifying sources and sinks, after which data from the input sources is labeled starting from the entry points. Throughout this process, each callsite is examined to determine if the method being called is a native method (indicated by the "*native*" keyword in its declaration). If the method is native and its arguments contain source data when being invoked, a three-tuple specification $S_{caller} = (S_e, D, A)$ (Figure 4.4) is generated to capture the calling context of this native method.

Specifically, S_e contains the signature of the called native method, including fully qualified method m (package name, class name, and method name), the return type r and the list of n parameter types P. In order to identify tainted arguments and aliases, D and A are introduced. D represents a list of arguments indicating which argument a_i is tainted or not ($taint(a_i) = true$ or $false$). A represents a list of pairs of arguments, where each pair (a_j, a_k) represents an alias relationship between argument a_j and argument a_k ($alias(a_j, a_k) = true$ or $false$). If tainted data and aliases occur at a deeper level within the arguments, such as $a_j.f$ and $a_k.g$, we mark the specific level of the taint and alias relationships in fields D and A.

When marking tainted parameters, a recursive access path may arise. For example, when a parameter's field points back to the parameter itself ($a.f = a$), the length of the access path can potentially become infinite ($a.f$, $a.f.f$, $a.f.f...f$). We use k-limiting [57] to address this issue. We set a fixed value k, signifying the maximum length for all access paths within our analysis. In cases where the analysis produces an access path that surpasses this predetermined value k, it is truncated and annotated with an asterisk. For instance, when $k = 2$, "$o.f.f.f$" is represented as "$o.f.f.*$" and all objects pointed to by fields after "$o.f.f$" are considered tainted. Moreover, using k-limiting to truncate the access path is sound because $o.f.f.f$ is a subset of $o.f.f$, and using $o.f.f$ to represent $o.f.f.f$ will cause the result of static analysis to be conservative.

But before generating a new S_{caller}, the Java analyzer will check whether the same native call context has been encountered previously. This context involves the call to the same native method and the same aliases and tainted flows. If such a context has occurred before, it reuses the existing native call specification (S_{callee})

JNI_class.java

```
J1.  public class JavaCaller {
J2.      private native void native_c_callee (User u1, User u2);
J3.      private void caller(Request request ){
J4.          User user = new User();
J5.          user.setUserName(request.getUsername());
J6.          user.setPassword(request.getPassword()) ;
J7.          User loginUser = user;
J8.          native_c_callee(user, loginUser);
J9.      }
J10. }
```

Caller's Specification

$$
\begin{aligned}
&\boldsymbol{S_{caller}} = \{\\
&\quad \boldsymbol{calleeMethod}: "JavaCaller:\ void\ native_c_callee(User, User)",\\
&\quad \boldsymbol{taintedArgs}: [arg0.username, arg1.username],\\
&\quad \boldsymbol{aliasArgs}: [arg0, arg1],\\
&\}
\end{aligned}
$$

Figure 4.5: Caller's specification extraction

and continues the analysis of the Java code. Otherwise, a new S_{caller} is generated for the subsequent C/C++ code analysis.

For example, the caller specification S_{caller} of Java method *caller()* is like the format of Figure 4.5. *calleeMethod* (Field S_e) records the signature of the invoked native method *native_c_callee()*. *taintedArgs* (Field D) and *aliasArgs* (Field A) indicate that the *username* of first and second arguments of this native call are both tainted and aliased.

4.3.2 Callee's Value-flow Specification Extraction on C/C++ Side

Once the caller specification S_{caller} is generated, it will be used by C/C++ analyzers to produce the callee specification with source-sink information for native functions. The caller specifications are combined with the LLVM IR compiled by C/C++ code to produce a CSS containing the value-flows of tainted arguments.

Figure 5.1 illustrates the five steps involved in generating a caller-sensitive (callee) specification:

- **Native function identification**: Accurate mapping of a native method in Java to its corresponding function in C/C++ is necessary to locate the implementation code of the method, as naming conventions for native methods differ between the two languages.
- **Taint and alias arguments annotation**: This step can assist in more precisely tracking tainted arguments in determined native function and identifying alias relationships among them.
- **Sources and sinks determination in native code**: Native sources and sinks need to be specified to generate interprocedural value-flows in native code.
- **Sparse value-flow graph (SVFG) construction**: Sparse value-flow graph is then constructed to capture the def-use chains of variables for sources and sinks.
- **Caller-sensitive specification generation**: This is accomplished by solving graph reachability on SVFG, which entails traversing from sources to sinks in native code. According to whether the sources can reach the sinks, the corresponding specification is generated.

Before the five steps mentioned earlier, C/C++ code is converted to LLVM IR using *clang* or other tools. In LLVM IR, each instruction is in Static Single Assignment (SSA) form, ensuring that each variable is assigned only once. LLVM IR comprises two types of variables: top-level variables and address-taken variables. Top-level variables, which consist of stack virtual registers (prefixed with "%") and

global variables (prefixed with "@"), are explicit and maintain the SSA form. On the other hand, address-taken variables are accessed indirectly through "*Load*" or "*Store*" instructions and encompass stack objects, heap objects, and global objects.

4.3.2.1 Native function identification

As described in Section 2.2, the naming conventions for native methods differ between Java and C/C++. In Java, native methods are typically declared using the "*native*" keyword, followed by the method signature. To ensure accurate mapping between Java and C/C++, functions that adhere to the JNI naming rules and those registered dynamically through *(env)→RegisterNatives()* within the *JNI_OnLoad()* function in C/C++ code will be searched to find the counterpart of the Java native method (green curved arrow in Figure 4.6).

4.3.2.2 Taint and alias arguments annotation

Fields D (taintArgs) and A (aliasArgs) in S_{caller} are used after the C/C++ function is determined. D is used to mark which arguments are "tainted", which can help eliminate the influence of other "untainted" arguments and focus only on the tainted ones. A is used when allocating abstract objects for the C/C++ function arguments. If two arguments are aliased, only one object is allocated for both of them. Conversely, if they are not aliased, then two different objects are allocated for each of the arguments, respectively. For example, in Figure 4.6, since *taintedArgs* (Field D) in S_{caller} indicates that the field *username* of first and second arguments are tainted, then the objects pointed by *a.username* and *b.username* are marked as sources and their value-flows will be tracked. In addition, *aliasArgs* (Field A) indicates that parameters a and b are aliased, so only one object obj_1 is assigned to *a.username* and *b.username* (blue curved arrow).

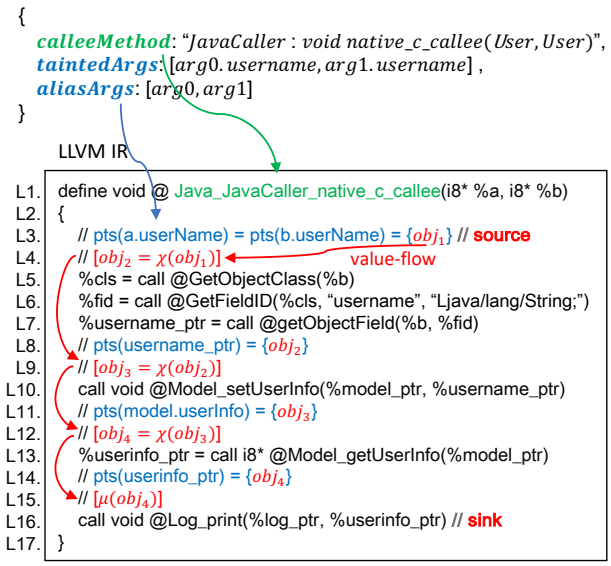


Figure 4.6: A fragment of LLVM IR and its memory SSA form of the motivating example in Figure 4.3

4.3.2.3 Sources and sinks determination in native code

The parameters indicated in the field D of S_{caller} are marked as sources (or native sources), such as the field *username* of the first and second parameters in Figure 4.6. Sinks in native code (or native sinks) need to determine whether the tainted data flows to return value or is leaked to the outside world (data confidentiality is violated). Some sink functions can be configured before the analysis, such as *fprintf()* or *__android_log_print()*, to check whether native sources are used as arguments of these native sink functions. In the motivating example, we are interested in whether the user's *name* or *password* information is written to the logs, so *Log.print()* is marked as a native sink.

4.3.2.4 Sparse value-flow graph (SVFG) construction

Sparse pointer analysis propagates pointer information from variable definitions to their uses through pre-computed def-use chains. Sparse value-flow graph (SVFG) [121, 122] captures def-use chains and value-flow via assignments for all

memory locations represented by both top-level and address-taken pointers. After getting the S_{caller} and LLVM IR, SVFG is constructed to capture the value-flow paths within and across procedural boundaries. During the construction of SVFG, the side-effects of instructions such as "*Load*", "*Copy*", "*Store*", "*Phi*", and "*Call*" are annotated using χ and μ functions. χ and μ functions were used to represent potential defs and uses of a memory object o at stores or loads, where $\mu(o)$ indicates the use of variable o, and $o = \chi(o)$ represents the def and use of o [32].

However, JNI C/C++ functions differ from regular C/C++ functions and require addressing the following three issues:

Constructing points-to sets of arguments Before constructing the SVFG of a determined C/C++ function, one challenge is that the caller on the C/C++ side is absent, as the caller resides on the Java side. Consequently, the points-to set of the arguments is initially empty. To resolve this, it becomes necessary to construct the points-to sets of the arguments based on the fields D and A in the caller specification S_{caller}. When multiple arguments have alias relationships, only one object is created (e.g., obj_1 in Figure 4.6). Conversely, for arguments without alias relationships, separate objects are created.

JNI type conversion functions modeling Another difficulty in building the SVFG arises from JNI-provided functions. JNI functions are a set of native programming interfaces provided by the JDK, enabling developers to interact with Java objects, fields, and methods in native code. Type conversion functions are a subset of JNI functions specifically used to convert data between Java and C/C++ data types. Given the distinct data type systems between Java and C/C++, JNI functions play a crucial role in Java and C/C++ communication.

CSS aims to automatically summarize the native application code (C/C++ code) by modelling the behaviour of JNI APIs. These APIs when used in C/C++ have pre-defined semantics. CSS supports modeling of JNI interactions between Java and C in a caller-sensitive manner for precise summarization of native code. When compiling JNI functions into LLVM IR, their source code is often not included. We have abstracted their side-effects to address this challenge in value-flow analysis. This strategy improves value-flow analysis by focusing on user-defined logic and efficiently handling the complexities of JNI functions. For example, we replaced the body of *const char* addr = env→ GetStringUTFChars(env, addr_, NULL)*, which retrieves a pointer to a UTF-8 encoded string representing a Java String object, with a store statement **addr=addr_*, allowing us to handle JNI functions without access to their source code.

JNI callbacks to Java Methods and Fields Handling In native code, when callbacks to Java fields and methods are needed, a series of JNI functions are used. These functions contain the complete signatures of the Java fields and methods being called back, such as class names, method names, and, types. If C/C++ analysis can correctly resolve the callbacks based on the signatures of the Java fields and methods, we will parse these callbacks. For example, Lines L5-7 are considered an operation to retrieve the "*username*" field of object *b*. For callbacks with tainted values that the C/C++ analysis cannot resolve, we record the signatures of callback Java fields or methods and mark which field or argument is tainted. This information is then passed to the Java analyzer after native code analysis.

4.3.2.5 Caller-sensitive specification generation

Once the SVFG is built, the value-flow paths of top-level and address-taken variables can be obtained by solving a graph reachability problem on it. For example,

$$\mathbb{P}\ (parameter) = \{p_i \mid p_i \in \mathbb{T}, 0 \leq i < n\}$$
$$\mathbb{S}\ (signature) = \{(m, r, P) \mid m \in \mathbb{M}, r \in \mathbb{T}, P \in \mathbb{P}\}$$
$$\mathbb{F}\ (field) = \{(f, p) \mid f \in \mathbb{M}, p \in \mathbb{T}\}$$
$$\mathbb{Q}\ (source) = \{a_i \mid taint(a_i) = true, 0 \leq i < n\}$$
$$\mathbb{B}\ (sink) = \{(ret \mid S_k) \mid S_k \in \mathbb{S}\}$$
$$\mathbb{J}\ (callback) = \{(F_j \mid S_j) \mid F_j \in \mathbb{F}, S_j \in \mathbb{S}\}$$
$$S_{callee} = \{(S_e, Q, B, J) \mid S_e \in \mathbb{S}, Q \in \mathbb{Q}, B \in \mathbb{B}, J \in \mathbb{J}\}$$

Figure 4.7: Four-tuples callee specification

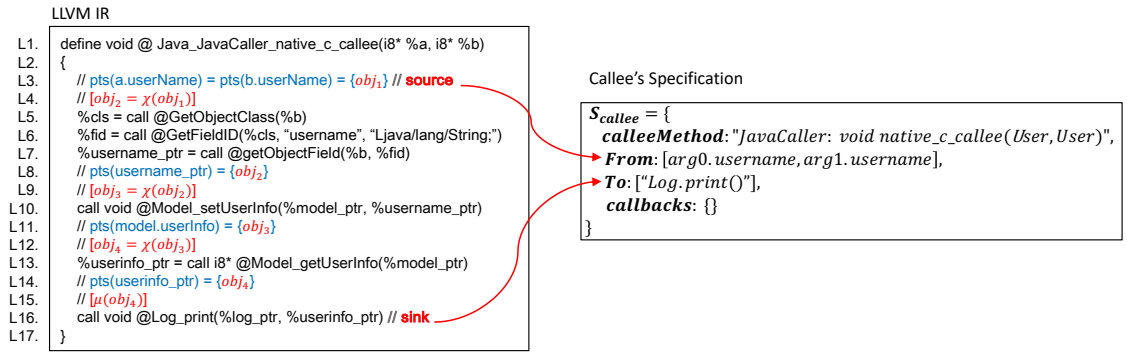


Figure 4.8: Callee's specification extraction

the value-flow of address-taken variables obj_1 in Figure 4.6 is $L3 \rightarrow L4 \rightarrow L9 \rightarrow L12 \rightarrow L15$ (red curved arrow). Finally, the value-flow passed in the function *Log_print()* through an argument. Then a four-tuple caller-sensitive specification $S_{callee} = (S_e, Q, B, J)$, as shown in Figure 4.7, is created to record the tainted value-flow.

S_{callee} is similar to S_{caller}, but fields D and A in S_{caller} are replaced by Q and B in S_{callee}. Additionally, we introduce J to record call-back Java methods S or Java fields F in native code, where F contains field names and types. When S_{callee} is returned to the Java static analyzer for use, S_e is used to identify and locate which native method is called. Q contains the taint arguments flowing to a sink in the native function. B indicates the destination of the tainted arguments in Q. B has two destinations: *ret* means that it flows back to the Java side through the return value, S_k is the signature of the sink function, indicating that arguments in

Q finally flows to the sink function. J contains Java methods and fields that are accessed sequentially in the order they are invoked from the native code callback.

For instance, Figure 4.8 shows the CSS of the function *Java_JavaCaller_native_c_callee*(), which captures the tainted value-flow originating from the field *username* of first and second arguments (*From: [arg0.username, arg1.username]*) and directs it towards the native function *Log.print()* (*To: [Log.print()]*). Because we can deduce from S_{caller}'s parameter information that the role of Lines L5-7 in Figure 4.6 is to retrieve the *username* from *b*, so *callbacks* (Field J) in Figure 4.8 is empty. Otherwise, we need to record callback information for Java fields.

4.4 Evaluation

We present the evaluation of our CSS, aiming to assess its ability to precisely summarize the value-flows of the native code to be used by existing Java analyzers. We conducted three experiments to validate our approach.

Experiment 1: Android benchmark *NativeFlowBench* For this experiment, we utilize the benchmark *NativeFlowBench* as introduced in a previous study [138]. This benchmark contains 23 hand-crafted Android JNI applications, offering a wide spectrum of interoperability features. Our analysis employs Flowdroid [9] to conduct value-flow analysis on the Java code and SVF [120] for C/C++ code. This combination of analysis tools allows us to evaluate the effectiveness of CSS in tracking value-flows across both Java and C/C++ components of these Android applications.

Experiment 2: Java benchmark *JavaNativeBench*. To further evaluate the robustness of CSS, we create a set of custom-designed applications *Ja-*

vaNativeBench that simulate real-world JNI interactions. These applications are crafted to cover scenarios not addressed in *Experiment 1*. We leverage Tai-e [128], WALA [44], Flowdroid, and, JN-SAF to perform value-flow analysis. In addition, we conduct a comparative experiment between CSS and another summary-based approach to evaluate the quality of CSS.

Experiment 3: Real-world JNI projects. To assess the practical applicability and correctness of CSS within real-world contexts, we examine ten real-world JNI applications sourced from Github. These applications represent diverse Android projects that extensively utilize JNI, providing a comprehensive and robust evaluation platform for our CSS approach. We use FlowDroid for value-flow analysis on the Java components and SVF for the C/C++ components.

Spanning these three experiments, our objective is to demonstrate the effectiveness of the proposed CSS in precisely summarizing value-flows within the native code and proficiently tracing value-flows across JNI programs. Utilizing FlowDroid, Tai-e and, WALA for Android and Java applications separately demonstrates how CSS integrates with various Java static analyzers, demonstrating its ability to improve single-language analyzers for cross-language analysis.

Environment setup. All of our evaluations were performed on 12th Gen Intel(R) Core(TM) i7-12700 with 16GB of RAM. The OS. is Ubuntu 22.04.3 LTS with Linux 6.2.0-33-generic 64-bit. Before experimenting with each project, sources and sinks will be predetermined, and a manual check for ground truths will be conducted to confirm the existence and number of paths from sources to sinks. This manual analysis is performed by systematically inspecting program code to enumerate all feasible source-to-sink paths, independent of tool outputs, ensuring both precision (no false positives) and completeness (all feasible paths). As sources and sinks are fixed and benchmarks are available, the procedure is reproducible. Addi-

tionally, projects with excessively long running times due to bugs in the baseline tools will be manually terminated to ensure a fair comparison.

The experimental results are publicly available [1].

4.4.1 Results on *NativeFlowBench* benchmark

In *NativeFlowBench*, all the projects are Android-based. Therefore, we chose a popular Android static analysis tool, Flowdroid [9], as a baseline in this experiment. JN-SAF [138] and JuCify [108] are two tools directly related to cross-language taint analysis. Both can perform source-sink data flow analysis for Android apps and only require specifying sources and sinks for the experimental projects. Additionally, we integrated CSS into Flowdroid to assess whether our proposed approach can enhance the capabilities of a single-language static analyzer Flowdroid for cross-language data-flow analysis.

Table 4.1 presents the results of four different tools on *NativeFlowBench*, namely FlowDroid (abbreviated as FD), JN-SAF (JS), Jucify (JC), and FlowDroid-CSS (FD-CSS).

FlowDroid, primarily focused on analyzing Java and Android applications, encountered challenges when handling native calls. It did not process these native calls if no manual configuration was provided for their source-sink paths, often leading to unsound modeling, as demonstrated by its result (8.7% precision) in Table 4.1.

FlowDroid-CSS, a hybrid approach that uses FlowDroid with CSS summarization of C/C++ code, showed significant improvement over FlowDroid. By leveraging CSS to provide C/C++ value-flow information, FlowDroid-CSS significantly

[1] https://zenodo.org/records/17151880

Table 4.1: Results of *NativeFlowBench* benchmarks. FD, JS, JC, FD-CSS denotes the result of FlowDroid, JN-SAF, Jucify and FlowDroid-CSS.

NativeFlowBench	FD	JS	JC	FD-CSS
icc_javatonative	×	✓	×	×
icc_nativetojava	×	✓	×	✓
native_complexdata	×	✓	×	✓
native_complexdata_stringop	×	∗	×	×
native_dynamic_register_multiple	×	✓	×	✓
native_heap_modify	×	✓	×	×
native_leak	×	✓	×	✓
native_leak_array	×	✓	×	✓
native_leak_dynamic_register	×	✓	×	✓
native_method_overloading	×	×	×	✓
native_multiple_interactions	×	✓	×	✓
native_multiple_libraries	×	✓	×	✓
native_noleak	✓	×	✓	✓
native_noleak_array	✓	∗	✓	✓
native_nosource	×	×	×	×
native_pure	×	✓	×	×
native_pure_direct	×	✓	×	×
native_pure_direct_customized	×	✓	×	×
native_set_field_from_arg	×	✓	✓	✓
native_set_field_from_arg_field	×	✓	×	✓
native_set_field_from_native	×	✓	×	×
native_source	×	✓	✓	×
native_source_clean	∗	×	×	✓
Precsion	8.7%	73.9%	17.4%	60.9%

✓: True Positive; ∗ : False Positive; ×: False Negative;

enhances FlowDroid's capability to handle JNI programs, achieving 60.9% precision.

JN-SAF performs relatively well (73.9% precision) compared to some other tools analyzed given that the benchmark was created by the authors of JN-SAF themselves. For cases where entry points not located in native code, the differing performances of FlowDroid-CSS and JN-SAF on certain benchmarks, including *native_method_overloading*, *native_noleak*, *native_noleak_array*, *native_source_clean*, and *native_complexdata_stringop*, primarily depend on the differences in the data flow analysis algorithms of the two tools. For instance, JN-SAF had a false positive on *native_noleak_array* because it cannot distinguish different indexes of a Java array. Another false positive occurred for *native_complexdata_stringop* due to its imprecise string analysis and overapproximated the contents of the string. In contrast, FlowDroid-CSS did not analyze the content of string variables, resulting in false negatives.

The biggest challenge that FlowDroid-CSS faced is scenarios where the entry points and sources are located in native code. This limitation arises from the requirement of caller information provided by the Java side for further value-flow analysis. Without calling context information, subsequent native code value-flow analysis cannot be completed. As a result, FlowDroid-CSS encountered difficulties in handling certain cases, including *icc_javatonative*, *native_heap_modify*, *native_pure*, *native_pure_direct*, *native_pure_direct_customized*, *native_set_field_from_native*, and *native_source*. These scenarios involve native calls initiated by the C/C++ side, making the caller information unavailable for the Java-based analysis, leading to failure in generating correct specifications.

Table 4.2: Results (R) and Time(s) (T) of *JavaNativeBench* benchmarks

JavaNativeBench	Tai-e				WALA				Flowdroid				JN-SAF	
Name	CIS		CSS		CIS		CSS		CIS		CSS			
	Result	Time	Result	Time	Result	Time	Result	Time	Result	Time	Result	Time	Result	Time
Java2Java	✓	4.09	✓	4.01	✓	2.98	✓	3.04	✓	6.94	✓	6.12	×	1.12
Java2Native	✓	3.98	✓	4.04	✓	3.03	✓	3.12	✓	7.1	✓	6.14	×	1.09
Java2JavaAlias	✓	4.07	✓	3.88	✓	3.04	✓	3.28	✓	6.09	✓	6.28	×	1.04
Java2JavaNoalias	*	4.02	✓	5.37	*	3.11	✓	4.23	*	6.18	✓	8.24	×	1.12
Java2NativeAlias	✓	3.97	✓	4.15	✓	3.02	✓	3.24	✓	6.06	✓	6.31	×	0.99
Java2NativeNoAlias	*	4.15	✓	5.23	*	3.39	✓	4.21	*	6.37	✓	7.87	×	1.21
Java2JavaFieldAlias	×	4.23	✓	4.08	×	3.22	✓	3.31	×	6.16	✓	6.28	×	1.15
Java2JavaFieldNoAlias	×	4.16	✓	5.12	×	3.27	✓	4.52	×	6.28	✓	8.56	×	1.09
Java2NativeFieldAlias	×	4.08	✓	3.98	×	3.23	✓	3.47	×	6.29	✓	6.51	×	1.11
Java2NativeFieldNoAlias	×	4.21	✓	5.23	×	3.43	✓	4.34	×	6.5	✓	8.29	×	1.12

✓: True Positive; ∗ : False Positive; ×: False Negative;

Jucify, another tool capable of handling Android cross-language value-flow analysis, exhibited limitations (17.4% precision), particularly in scenarios involving purely native functions or native leaks. This constraint is due to Jucify's original design, which did not explicitly cater to such scenarios, where native functions are prevalent in many JNI programs.

Note that the current version of JN-SAF is outdated, as it can only analyze NDKs compiled in specific versions. However, the authors of JN-SAF did not specify which NDK versions can be analyzed; the problem was also mentioned by the author as per the GitHub issue [6]. Therefore, the Android APKs used in *NativeFlowBench* are provided by JN-SAF.

In summary, the CSS significantly improves FlowDroid's taint analysis, particularly for native calls in Java code. While limitations exist in handling specific JNI scenarios, CSS shows promise in enhancing the effectiveness of value-flow analysis in the presence of native interactions.

4.4.2 Results on *JavaNativeBench* benchmark

Apart from *NativeFlowBench*, we designed *JavaNativeBench*, specifically targeting scenarios where Java callers invoke C/C++ functions with aliased and tainted

```
     // Java
J1.  public static void main(String[] args){
J2.      Data d1, d2;
J3.      d1.data = source();
J4.      d2.data = "…";
J5.      String s = nativeTransfer(d1, d2);
J6.      sink(s);
J7.  }

     // C
C1.  char* findSubstring(const char* string) {
C2.      const char* subString = "…";
C3.      return strstr(string, subString);
C4.  }
C5.  jstring Java_Main_nativeTransfer(jobject obj1, jobject obj2) {
C6.      jclass cls= GetObjectClass(obj1);
C7.      jfieldID fd = GetFieldID(cls, "data", "Ljava/lang/String;");
C8.      jstring dataString = GetObjectField(obj1, fd);
C9.      char* originalString = GetStringUTFChars(dataString);
C10.     char* modifiedString = findSubstring(originalString);
C11.     jstring resultString = NewStringUTF(modifiedString);
C12.     jclass cls2 = GetObjectClass(obj2);
C13.     … // Operations on the jobject obj2
C14.     return resultString;
C15. }
```

Figure 4.9: Code in *Java2JavaFieldNoAlias*

arguments. We employ two additional Java static analysis tools, Tai-e [128] and WALA [44], alongside Flowdroid and JN-SAF, to analyze the value-flow. Similar to FlowDroid, Tai-e and WALA focus solely on the analysis of Java code in the absence of native method models. Additionally, we also have a experiment between CSS and another summary-based specification SemanticSpec [67].

4.4.2.1 CIS and CSS

To validate the effectiveness of CSS, we also conducted a comparative experiment using caller-insensitive specifications (CIS). Unlike CSS, CIS does not consider Java caller information and directly analyzes the C/C++ code to extract relevant value-flows. For this experiment, we use a conservative version of CIS that considers all alias relationships among arguments in native functions. The focus was on determining the existence of paths from tainted arguments to sinks in the value-flows.

Table 4.2 presents the results and times for the three tools: Tai-e, WALA, and, Flowdroid, respectively, using CIS, CSS, and JN-SAF applied to the *JavaNativeBench* evaluation. Tai-e-CSS, WALA-CSS and, Flowdroid-CSS demonstrated promising results, successfully analyzing all test cases in the *JavaNativeBench*. Additionally, Tai-e-CIS, WALA-CIS and, Flowdroid-CIS achieved success in four cases, while in the remaining six test cases, there were four false negatives and two false positives. Tai-e-CSS/WALA-CSS/Flowdroid-CSS and Tai-e-CIS/WALA-CIS/Flowdroid-CIS both indicate that providing value-flow information about native calls can be beneficial for the value-flow analysis of JNI programs.

The failures observed in CIS (Tai-e-CIS, WALA-CIS, and, Flowdroid-CIS) are primarily attributed to two main reasons. First, without access to calling context information, CIS lacks the necessary context to determine whether certain arguments have alias relationships. To ensure the conservativeness of the analysis results, it must consider all alias scenarios, such as in Figure 4.9 where *obj1* and *obj2* have the same type *jobject* (Line C5), which may lead to false positives (*Java2JavaNoAlias* and *Java2NativeNoAlias*). At the same time, this over-approximation can obscure real aliasing behaviors, resulting in false negatives in some cases. However, before conducting taint analysis on C/C++ code, CSS (Tai-e-CSS, WALA-CSS, and, Flowdroid-CSS) already knew the alias and taint relationships between native call parameters. Therefore, it excludes irrelevant scenarios during the taint analysis, making the generated specifications precise and concise, which reduces the incidence of false positives.

The second reason for failures is when dealing with JNI-provided functions, such as *GetObjectClass(obj1)* in Figure 4.9 (Line C6), where CIS lacks caller information to determine the specific class information corresponding to the object *obj* in Java. This lack of class or object information hinders the generation of effective

specifications for these scenarios. The failures observed in *Java2JavaFieldAlias*, *Java2JavaFieldNoAlias*, *Java2NativeFieldAlias*, and *Java2NativeFieldNoAlias* can also be attributed to these reasons. However, CSS, informed by the caller specification, get the specific java object information represented by *jobject*, allowing the taint analysis to continue.

The reason JN-SAF failed on all 10 test cases is that it cannot analyze the native code compiled with the newer NDK, and JN-SAF did not specify which version of the NDK used to compile their APKs [6]. In Experiment 1, we used the pre-compiled APKs provided by the authors of JN-SAF, not the ones compiled by us because JN-SAF was unable to properly parse the native code we compiled ourselves.

However, a limitation of CSS is that if the same native method is called in different contexts from Java callers (i.e., under different taint and alias arguments), the same native method needs to be reanalyzed and resummarized to address the different contexts. In contrast, for the same context, CSS can reuse pre-generated callee specifications. This situation is evident from the runtime of *Java2JavaNoalias*, *Java2NativeNoAlias*, *Java2JavaFieldNoalias* and *Java2NativeFieldNoalias*, where, in these four cases, there are multiple calls to the same native method under different contexts.

4.4.2.2 Comparison of Different Specifications

Lee et al. [67] also used a summary-based method for JNI analysis aimed at finding interaction bugs. For clarity, we refer to their summary as SemanticSpec. SemanticSpec first extracts semantic summaries from C code, translates them into Java, and integrates them for whole-program analysis.

Table 4.3: Comparison of quality, taint analysis results and runtime between SemanticSpec and CSS

JavaNativeBench	SemanticSpec			CSS		
Name	Quality	Result	Time	Quality	Result	Time
Java2Java	O	✓	2.23	O	✓	2.20
Java2Native	O	✓	2.21	O	✓	2.19
Java2JavaAlias	O	✓	2.19	O	✓	2.20
Java2JavaNoalias	O	✓	2.20	O	✓	2.22
Java2NativeAlias	O	✓	2.22	O	✓	2.22
Java2NativeNoAlias	O	✓	2.23	O	✓	2.21
Java2JavaFieldAlias	R,F	×	2.39	O	✓	2.21
Java2JavaFieldNoAlias	R,F	×	2.44	O	✓	2.20
Java2NativeFieldAlias	O	✓	2.23	O	✓	2.22
Java2NativeFieldNoAlias	O	✓	2.21	O	✓	2.21

O: Correct specification; R: Redundant specification;
F: Incorrect specification;
✓: True Positive; ×: False Negative;

Table 4.3 compares the performance of the static analyzer Tai-e using SemanticSpec and CSS on *JavaNativeBench*. The comparison includes taint analysis results, runtime (excluding the time to generate SemanticSpec and CSS), and specification quality. Specification quality is evaluated based on the following criteria: (1) a *correct* specification contains the intended value-flows without redundancies; (2) a *redundant* specification includes the intended value-flows but also unnecessary cases; (3) an *incorrect* specification lacks the intended value-flows. We categorized them based on their effects on the accuracy of the taint analysis results and the analysis time. Correct specifications produced accurate results with minimal analysis time by capturing intended value-flows without redundancies. Redundant specifications yielded correct results but had longer analysis times due to including unnecessary cases. Incorrect specifications led to erroneous taint analysis results by missing intended value-flows.

Like CIS, SemanticSpec extracts C code semantics without caller context and translates it into Java for analysis, potentially retaining redundant information. For

example, in *Java2JavaFieldNoAlias* of Figure 4.9, SemanticSpec is conservative and hence does not distinguish between tainted *obj1* and untainted *obj2*, leading to specifications that include unnecessary semantics about *obj2*. This lack of caller context can result in redundant specifications and slightly longer analysis times compared to CSS, as seen in *Java2JavaFieldAlias* and *Java2JavaFieldNoAlias* of Table 4.3. Additionally, SemanticSpec generates summaries for the target guest language and struggles with some complex C features and functions. For instance, in the function *findSubstring()* of Figure 4.9, it imprecisely models the return value as a new, unrelated string, leading to undetected taint flows in cases like *Java2JavaFieldAlias* and *Java2JavaFieldNoAlias*. CSS generates concise caller-sensitive specifications without unnecessary cases by leveraging the taint and alias information of parameters provided by the caller. The experimental results demonstrate that CSS detects all tainted value flows in our benchmarks.

The results of the above experiments highlight the significance of incorporating CSS in JNI analysis. By leveraging caller specification information, CSS can effectively track alias relationships and handle JNI functions, thereby improving the precision and accuracy of value-flow analysis for JNI programs.

By incorporating CSS into existing Java static analyzers (e.g., FlowDroid, Taie and, WALA), the existing established static tools and their algorithms can be leveraged for cross-language analysis. This integration approach enables the static analyzers to take advantage of their existing value-flow analysis techniques in the Java domain and supplement them with the value-flow information from the native (C/C++) world, effectively bridging the gap between the two languages.

4.4.3 Results on real-world JNI programs

In addition to conducting experiments with *NativeFlowBench* and *JavaNativeBench*, we further tested real JNI programs to validate the accuracy of CSS in extracting value-flow information from native code. For this evaluation, we selected ten JNI-related Android projects from GitHub for our analysis using three tools: JN-SAF, JuCify and, FlowDroid-CSS.

4.4.3.1 Implementation

We selected 10 repositories from around 50 GitHub candidates identified with "JNI" and "NDK" keywords, focusing on three criteria: scale, diversity, and path length. Repositories were classified by size as small (1-15 KLoC), medium (15-100 KLoC), or large (over 100 KLoC), with our choices including 4 small, 4 medium, and 2 large. For diversity, we assigned one repository each to categories like "Security and Encryption", "Logging", and "Email" and selected 3 for "File/Data Management" and 2 each for "Audio/Image Processing" and "Privacy/Control". Path lengths were categorized from short (1-3 hops), medium(4-8 hops) to long (9+ hops), with selections of 4 short, 4 medium, and 2 long paths.

For the selection of sources, we primarily chose functions that involve sensitive data, such as *getDeviceId()*. As for sinks, we selected functions that could potentially leak sensitive data, such as *__android_log_print()*, which writes data to files. We limited the sources and sinks to two scenarios: (1) Sources in Java code, and sinks in native code; (2) Sources in Java code, and sinks in Java code. In the second scenario, the data generated by the source method in Java code flows into the native code and then returns to the Java world. The reason we did not consider the other two scenarios with the sources in native code, namely (1) Sources in native code, the sinks in Java code; (2) Sources in native code, the sinks in native

code, is because before processing the value-flow in native code, the Java side needs to provide the caller specifications. In other words, Java is the host language, and native code is the guest language, so the Java side needs to serve as the starting point for source generation in value-flow analysis.

Before the experiments, we manually inspected the code of these Android projects to determine value-flow paths between sources and sinks. With these value-flow paths as ground truth, we can more clearly compare the performance of the JN-SAF, JuCify and FlowDroid-CSS. Table 4.4 summarizes the value-flow analysis results of ten projects. The project characteristics include the name of the application (APP), the total number of lines of code in thousands (Loc), the percentage of code written in Java and C/C++ languages, and the total number of value-flow paths in each application (Total). The table also provides information on the performance of JN-SAF, JuCify and FlowDroid-CSS. For each tool, metrics such as "Recall", Recall-to-Total ("R/T") ratio, and execution "Time" (in seconds) are presented. "Recall" measures the effectiveness of the tool in identifying the correct value-flow paths, while The "R/T" ratio indicates the precision of the tool, and the "Time" shows the speed of the tool in processing Java and C/C++ code separately in the application.

4.4.3.2 Performance of JN-SAF, JuCify and FlowDroid-CSS

From the experimental results, it can be seen that value-flow analysis with CSS can correctly detect the source-sink paths in nine out of the ten projects, while JuCify succeeded in only two out of the ten projects. But JN-SAF failed for all projects. The failure of JN-SAF is attributed to its latest version being outdated in terms of analyzing native code. This prevents it from correctly locating or analyzing the .so files and conducting the analysis (The analysis time for C/C++ of JN-SAF

Table 4.4: Value-flow analysis results of three static analyzers JN-SAF, JuCify and, FlowDroid-CSS on ten Android JNI applications. The metrics include the total number of lines of code in thousands (Loc), the percentage of Java code (Java), the percentage of C/C++ code (C/C++), the number of true positive results (Recall) out of the total true positive results (Total) in each application, and the precision of the tools (R/T, Recall-to-Total). The table also reports the time taken in Java and C/C++ separately in seconds (Time) for each tool.

APP	Loc(K)	Java(%)	C/C++(%)	Total	JN-SAF				JuCify				FlowDroid-CSS			
					Recall	R/T(%)	Time(s)		Recall	R/T(%)	Time(s)		Recall	R/T(%)	Times	
							Java	C/C++			java	C/C++			Java	C/C++
android-aes-jni	1.3	12.6	86.8	1	0	0	5.7	-	0	0	1.0	194.4	1	100	3.7	0.2
LoggingApp	1.4	36.2	37.3	4	0	0	4.4	-	0	0	4.0	372.21	4	100	3.1	0.4
samba-documents-provider	11	81.2	16.0	1	0	0	14.3	-	1	100	21.0	37.0	1	100	10.9	2.3
base64encode	12	27.1	31.6	2	0	0	14.5	1.3	0	0	2.0	304.4	0	0	8	1.2
AndroidUn7zip	16	4.9	94.8	2	0	0	20.7	-	0	0	8.0	367.9	2	100	20.7	5.6
VoiceChange	26	60.5	23.5	6	0	0	23.8	-	0	0	25.0	444.8	6	100	23.9	6.4
FairEmail	63	55.4	38.5	1	0	0	69.2	-	1	100	22.0	492.7	1	100	28.9	10.2
Log4a	85	38.5	32.3	1	0	0	52.8	-	0	0	32.0	143.3	1	100	41.4	15.4
tracker-control-android	349	23.7	23.7	1	0	0	203.5	-	0	0	21.0	300.4	1	100	40.6	16.7
oboe	160	35.1	61.1	1	0	0	32	-	0	0	12.0	992.5	1	100	55	20.3
Avg	**72.5**	**37.5**	**44.6**	**2**	**0**	**0**	53.9	0.13	**0.2**	**20**	17.9	364.9	**1.8**	**90**	**23.6**	**7.9**

in Table 4.4 is "-"). Different NDK versions may employ various optimizations and compilation techniques, which can significantly impact the final results. The applications used in Experiment 3 were compiled by ourselves, and the projects used in Experiment 1 were compiled by the authors of JN-SAF. Moreover, JN-SAF did not specify the versions of NDK that can be analyzed [6].

The main reason for Jucify's failure is that the value-flow analysis in C/C++ code involves pure native functions. The current JuCify can only reconstruct native calls about Java-C interoperation parts but does not consider pure native functions, meaning it is unable to handle native functions and native leaks. Hence, the scope of JuCify's value-flow analysis is limited, as it cannot identify value-flow paths where sensitive information is leaked through pure C/C++ functions, which is a common scenario in real-world data breaches. Additionally, JuCify uses Angr for symbolic execution to analyze C/C++ code, but symbolic execution faces the issue of path explosion. The results from Table 4.4 indicate that the time JuCify takes to analyze native code using Angr is at nearly 46 times that of FlowDroid-CSS. This is even when JuCify only focuses on JNI-provided functions and but not the

```
C1: JNIEXPORT jstring JNICALL Java_base64encode (JNIEnv* env,
C2:                                              jobject obj, jstring str)
C3: {
C4:     const char* strs;                                   value-flow
C5:     strs = (env)->GetStringUTFChars(str,NULL);
C6:     char* str2=base64encode(strs);   // mathematical operations
C7:     env->ReleaseStringUTFChars(str,strs);
C8:     jstring result = (env)->NewStringUTF(str2);
C9:     free(str2);
C10:    return result;
C11: }
```

Figure 4.10: *base64encode* example

pure C/C++ code. If all logic within the native code were considered, the analysis time for the native code might continue to increase. On the other hand, JuCify cannot accurately reconstruct native code behavior because it uses heuristics to handle native code behavior, rather than tracking value-flow based on the native source code, causing a loss of precision.

As for FlowDroid-CSS, since we use native source code to conduct value-flow analysis, it does not miss the value-flow information of native code like JuCify does. By employing SVFG for sparse value-flow analysis and constructing the def-use chain of variables, it also avoids the path explosion problem encountered when using symbolic execution to analyze native code in Jucify. However, CSS still failed to detect one value-flow path in the project *base64encode*.

In project *base64encode*, a java-to-java value-flow path needs to be detected. The source data *str*, originating from the Java side, passes through the C function *Java_base64encode()* (Figure 4.10). Once encoded, *str* returns to the Java side as a return value. However, the *base64encode()* function located at Line C6 performs mathematical and encoding operations on the *str* variable. These operations represent an implicit value-flow originating from the argument and reaching its return value within *base64encode()*, while SVFG's capabilities are limited to

constructing explicit value-flows for the string variables (e.g., *str*). Therefore, the value-flow path is disconnected at the *base64encode()* function (red curved arrow), making it impossible to generate a complete java-to-java value-flow path. Tracking implicit flows is a semantic problem [61] and the common approach is to manually write a specification for implicit flows to define the value-flows. For example, we can configure the side-effect of *base64encode()* as a store operation of **str2 = strs*. Then CSS can accurately identify the value-flow path in this project. However, in the experiment, we did not summarize the side-effects of the functions involved between native sources and sinks for the sake of fairness in comparison.

In summary, value-flow analysis with CSS outperforms JuCify in terms of successfully detecting paths with a higher success rate and a lower average analysis time.

4.5 Related Work

Being a widely adopted programming language, Java has attracted considerable scholarly interest due to its distinctive features, including its capability for cross-language interaction. Tan et al. [126] carried out an empirical security study on JNI bugs in JDK's native code, providing remedies and insights into these bugs. Li and Tan [72,107] developed approaches to identify bugs in situations within JNI programs. Kondoh and Onodera [63] concentrated on four types of JNI-specific errors that native compilers do not capture. Shi et al. [112] utilized specifications to summarize information flow in Android native libraries to support taint analysis in Android apps. However, these studies primarily focus on native code and do not consider the interaction at the boundary between Java and native code.

Tan et al. [127] extend Java Virtual Machine Language (JVML) language for C code to assist static analysis for JNI programs. JN-SAF [138] proposed by Wei et al. first constructs a precise topologically sorted call graph for JNI programs. This framework alternates between Java and C/C++ static analyzers, ensuring accurate value-flow tracking throughout the application. JN-SAF's scalability can be constrained by path explosion problems arising from Angr's symbolic execution. Furthermore, it might yield inconsistent outcomes when handling binary codes with different optimization levels [6]. Samhi et al. [66] aimed at unifying bytecode and native code to facilitate comprehensive static analysis of Android apps. They introduced JuCify as a significant step towards this goal, which generates a native call graph that is merged with the bytecode call graph using links obtained through symbolic execution. However, a key limitation of JuCify it that it is hard to reconstruct native function behavior with high precision, impacting the effectiveness of the analysis in many cross-language context scenarios. Lee et al. [67] also used a summary-based method for JNI analysis, where the approach first extracts semantic summaries from C code and then integrates them with Java code for a whole-program analysis. Our work differs from theirs in that CSS is caller-sensitive, acquiring information from the Java callers to yield a compact and precise summarization. Hence our approach can be seen as a precision-driven enhancement on top of their methods if all source code is available.

Aside from cross-language static analysis of JNI programs, there is research focusing on static analysis involving other languages. Lee et al. [66] introduced HybriDroid for executing static analysis between Java and JavaScript. Monat et al. [95] and Li et al. [73] developed static analyzers for identifying runtime errors and reference count adjustments in Python programs using C extensions. Furr et al. [46,47,88] performed type safety checking targets OCaml's FFI to C and JNI. Li

et al. [77] compiled Rust and C into LLVM IR and employed abstract interpretation to detect cross-language memory management issues in Rust.

4.6 Chapter Summary

This chapter introduces CSS, a new approach to specification extraction known as caller-sensitive specification. Compared to using conservative specifications, this approach takes the cross-language calling context of native C/C++ functions into account to generate more accurate and compact specifications (side-effects) of the native functions being called from Java. By focusing on the side-effects that are sensitive to the cross-language calling context, the CSS approach can reduce the number of unnecessary specifications. In addition, the CSS can be easily integrated with Java single-language static analysis tools, enabling cross-language analysis with no or light code modifications. The results of the experiment demonstrate that CSS is capable of precisely tracking the value-flows in JNI programs over several recent tools. Considering the challenges in developing an effective static analyzer for JNI programs, the CSS approach presented in this paper offers valuable insights for enhancing static analysis tools in JNI contexts.

Chapter 5

LLM-Driven Hierarchical API Specification Inference

5.1 Overview

This chapter introduces SPECGURU, a framework that leverages Large Language Models (LLMs) to automatically generate points-to API specifications. SPECGURU employs a novel bottom-up approach that begins with leaf functions (those without dependencies) and progressively builds specifications through rigorous validation. These validated specifications subsequently serve as abstractions at call sites of higher-level functions along the call hierarchy, effectively eliminating the need to process complex source code directly with LLMs. Our approach incorporates a robust self-validation mechanism using automatically synthesized test cases for differential testing throughout the specification inference process, which reduces error propagation and improves the incremental correctness of the generated specifications.

Our experimental evaluation on 15 third-party C libraries shows that SPECGURU significantly outperforms existing specification inference tools, generating 21% more specifications than Spectre and 46% more than c-summary. Moreover, our bottom-up abstraction technique combined with automated testing methodology enables the discovery of a more comprehensive set of specifications while effectively preventing error propagation, achieving analysis results comparable to those obtained using complete API source code.

5.1.1 Motivation

Despite the importance of API specifications for static analysis, existing approaches to inferring points-to specifications face significant limitations when addressing the complexities of modern software systems. Current techniques primarily rely on manual annotation, static analysis, or dynamic execution [16–18, 33, 59, 67]. Manual approaches, while potentially precise, demand substantial domain expertise and impose prohibitive engineering costs at scale. Static analysis methods typically require intricate rule definitions to handle language-specific features and often falter when confronted with incomplete information, particularly at API boundaries where standard library functions like memcpy or strcpy may lack accessible implementations. Dynamic analysis approaches necessitate extensive instrumentation infrastructure, depend heavily on the quality of execution oracles, and fundamentally suffer from coverage limitations that may omit critical pointer behaviors in rarely-executed code paths. Given these limitations, there remains a clear need for a more effective approach capable of generating accurate points-to specifications while gracefully handling complex language constructs (e.g., nested structures, multi-level function calls, and sophisticated control flows) and incomplete information scenarios. Such an approach would bridge a critical gap in the static

analysis ecosystem, enabling reliable and efficient analysis of client code that interacts with third-party APIs without requiring standard whole-program iterative analysis, which repeats when the client code's program state changes (e.g., due to code modifications) or when different client code calls the same APIs, thus avoiding redundant re-analysis of the APIs' source code.

Recent advancements in Large Language Models (LLMs) have revealed significant capabilities across diverse software engineering tasks, including code generation, translation, and synthesis [25–27, 39, 75, 106]. The sophisticated ability of LLMs to analyze and comprehend source code presents a promising approach for generating API specifications through direct examination of API implementations. By systematically processing API source code, LLMs can infer complex pointer behaviors and data flows by leveraging their pre-trained knowledge of code semantics and common programming patterns. LLMs offer distinct advantages over traditional specification inference approaches. Unlike conventional static analysis techniques that require explicit rule formulation for handling language-specific features, LLMs inherently recognize various programming idioms and structural patterns through their extensive training on code repositories. Furthermore, in contrast to dynamic analysis methods that necessitate comprehensive instrumentation to capture execution traces, LLMs can reason about potential pointer behaviors across different control flow paths without requiring concrete execution scenarios. These capabilities directly address the fundamental limitations of existing approaches: they circumvent the manual rule engineering demanded by static analysis while overcoming the coverage constraints and runtime overhead intrinsic to dynamic analysis.

5.1.2 Challenges

While the application of LLMs to API specification generation presents a promising direction, significant challenges emerge in this domain. The effectiveness of LLM-generated specifications is constrained primarily by two fundamental challenges inherent to applying these models in the context of program analysis:

Challenge#1: Reliable specification generation. LLMs exhibit inherent limitations in reasoning accuracy, often producing erroneous outputs or hallucinations [10, 56, 110]. These problems become particularly acute when analyzing complex code structures with intricate call hierarchies, sophisticated language features, and domain-specific programming idioms—precisely the context where API specification generation operates. This challenge intensifies when reasoning about pointer relationships and memory behavior in languages like C, where subtle implementation details significantly influence the correctness of derived specifications.

Challenge#2: Specification validation. Ensuring the correctness of generated specifications is crucial for static analysis applications. While test-driven validation is an effective approach [25–27, 39, 75, 106], it traditionally demands comprehensive, high-quality test suites that impose significant manual engineering overhead, and even carefully designed test suites may miss edge cases or subtle pointer behaviors [90]. This validation gap demands automated, reliable techniques for generating comprehensive test cases that can validate LLM-generated specifications effectively.

5.1.3 Our Solution

To address these challenges, we present SPECGURU, a novel framework that systematically leverages LLMs to generate precise API points-to specifications. For **Challenge#1**, we implement a hierarchical **divide-and-conquer** approach that

resolves the fundamental tension between providing comprehensive API context and accommodating LLMs' context limitations. Beginning with leaf functions (those without API dependencies), we generate and validate specifications that subsequently serve as concise abstractions when analyzing higher-level functions in a **bottom-up** manner guided by the call graph structure. This progressive abstraction technique preserves essential semantic information while maintaining manageable input sizes, enabling effective specification inference across complex call hierarchies without exceeding the model's reasoning capacity. For **Challenge#2**, we introduce a novel **self-validation** methodology that builds upon recent findings demonstrating LLMs' proficiency in generating high-quality test cases [40, 74, 80, 135]. We employ **differential testing** with LLM-synthesized test cases to systematically compare execution behaviors between original implementations and generated specifications. This self-validation mechanism operates at each level of the call hierarchy, ensuring specification correctness throughout the entire bottom-up abstraction chain and preventing error propagation that would otherwise compound across successive layers of abstraction.

The bottom-up points-to specification inference combined with self-validation mechanism decomposes the complex task of library API specification inference into manageable components. This approach enables SpecGuru to efficiently generate and validate concise, precise specifications for third-party libraries incrementally, circumventing the limitations of attempting to process entire library implementations at once. By systematically building specifications from leaf functions upward and validating each component through differential testing, SpecGuru achieves both analytical precision and scalability while maintaining strong correctness guarantees throughout the specification hierarchy. We conducted experiments on 15 third-party C libraries to validate the performance of SpecGuru. SpecGuru out-

performs traditional static/dynamic analysis-based specification generation methods by inferring more specifications. Compared to using isolated or the complete source code of APIs under analysis, our bottom-up approach demonstrated better performance, and the automated testing strategy effectively prevented the spread of incorrect specification information. Additionally, the generated API specifications can achieve static analysis results comparable to those obtained using their source code.

In summary, our major contributions are as follows:

- A novel bottom-up approach for API specification generation using LLMs that first processes leaf functions, validates their specifications, then progressively builds higher-level specifications using validated lower-level abstractions.
- A robust self-validation mechanism that employs differential testing with LLM-synthesized test cases at each abstraction level, ensuring specification correctness and preventing error propagation throughout the specification inference hierarchy.
- An implementation and an evaluation to demonstrate SPECGURU's effectiveness of generating points-to specifications.

5.1.4 Framework Overview

Figure 5.1 illustrates the framework overview of SPECGURU: (a) Preprocessing, (b) Bottom-Up Specification Inference, (c) Specification Self-Validation, and (d) Client Applications (e.g., pointer analysis, taint analysis). Stages (b) and (c) are interleaved during execution to ensure specification quality. Initially, SPECGURU preprocesses the source code to partition functions based on their call dependency

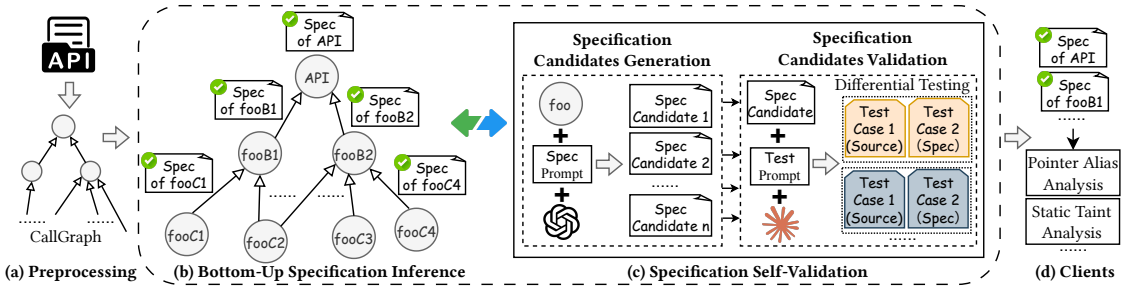


Figure 5.1: Specguru framework overview.

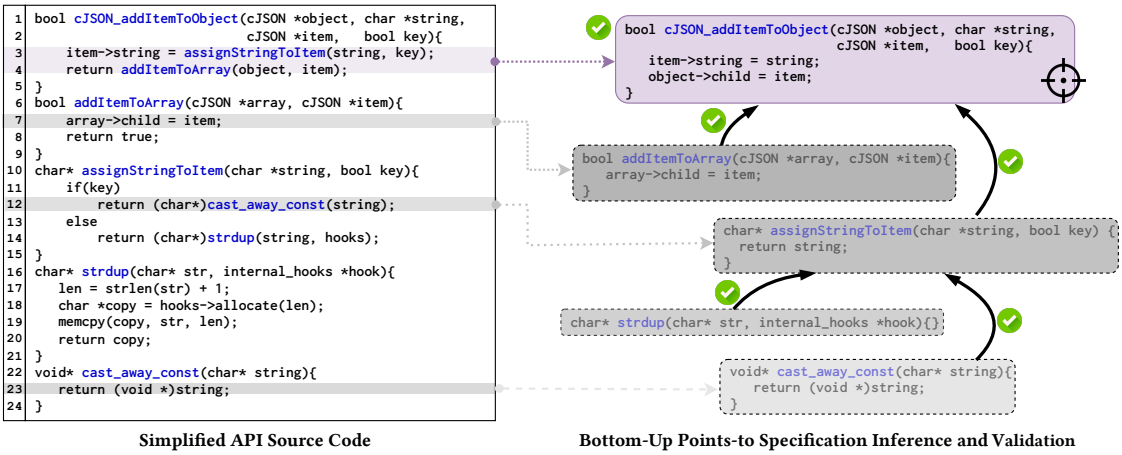


Figure 5.2: A motivating example illustrating how Specguru performs bottom-up points-to specification inference and validation for the library API `cJSON_addItemToObject` extracted from the `cJSON`.

relationships. The system then iteratively processes functions in bottom-up order: for each function, a specification is generated and immediately validated through differential testing before proceeding to functions that depend on it. This integrated approach of generation and validation ensures that only validated specifications propagate to the analysis of dependent functions. The resulting validated specifications are then provided to client applications for subsequent client applications, such as pointer alias analysis [35] and taint analysis [82].

5.2 A Motivating Example

Figure 5.2 illustrates the key idea of SPECGURU's API points-to specification generation through an API function, cJSON_addItemToObject (ℓ_1-ℓ_5), extracted from the third-party C library `cJSON` [49]. The third parameter item of cJSON_addItemToObject is ultimately assigned to the child field of the first parameter object via ℓ_7 of addItemToArray when invoked at ℓ_4. Furthermore, the second parameter string propagates through a sequence of function calls—first to assignStringToItem at ℓ_3, then to castAwayConst at ℓ_{12}—before being assigned to the string field of the third parameter item at ℓ_{23}.

When analyzing client code that uses cJSON_addItemToObject, traditional static analyzers typically perform a whole-program iterative analysis. This process involves repeatedly analyzing the complete implementation of the cJSON_addItemToObject API function and all its callees whenever the program state changes, to ensure analytical precision. However, this repeated analysis can be avoided by employing a concise specification (depicted in the purple rounded rectangle in Figure 5.2). This specification succinctly captures the essential pointer operations among parameters, representing the core behavior where a cJSON object (item) is added to another cJSON object (object) and assigned a name (string). By substituting this specification for the full implementation, identical pointer analysis results can be achieved while significantly reducing analytical complexity, condensing 24 lines of intricate code into just 4 lines of focused implementation.

Bottom-Up Specification Inference. SPECGURU implements a bottom-up approach integrated with divide-and-conquer principles for LLMs to perform points-to specification inference. This methodology leverages a key insight: points-to information naturally propagates from callees to callers within function hierarchies. Rather than attempting to analyze the entire implementation at once

or inline all callees into the root function, SPECGURU systematically decomposes the problem according to the call graph. Specifically, SPECGURU initiates analysis with leaf functions (those making no calls to other functions within the same library) before ascending the call hierarchy. This strategic decomposition transforms the complex task of analyzing entire libraries into modular, function-level analyses that are significantly more manageable. As illustrated in Figure 5.2, SPECGURU first synthesizes specifications for the leaf functions castAwayConst, strdup, and addItemToArray. These specifications are subsequently incorporated into the analysis of their respective callers—assignStringToItem and ultimately cJSON_addItemToObject—thereby facilitating efficient and comprehensive specification inference throughout the entire library API.

Specification Self-Validation. Another challenge is the validation of inferred specifications: third-party libraries serve diverse purposes across numerous domains, making it impractical to rely on pre-existing test cases for each library. Manual inspection or custom test case development would be prohibitively resource-intensive and undermine the automation benefits of our approach. To address this validation challenge, SPECGURU implements a self-validation strategy that leverages recent research demonstrating LLMs' effectiveness in generating high-quality test cases [40, 74, 80, 135]. This approach enables systematic validation of inferred points-to specifications without requiring domain-specific knowledge or pre-existing test suites.

For specification validation, SPECGURU employs a systematic differential testing approach. First, the test case generation LLM synthesizes test_case_1 based on the function signature and the identified points-to relationships between parameters and return values. Subsequently, SPECGURU derives test_case_2 by replicating test_case_1 but substituting the function's implementation with its inferred spec-

ification. Through comparative execution of these paired test cases, SPECGURU evaluates whether the specification accurately captures the original implementation's pointer behavior.

```
*** test_case_1 ***:
int main(){
 ...
 # use source code
 addItemToArray(arr,it);
 if(arr == it)
 {...}
 ...
}
```

```
*** test_case_2 ***:
int main(){
 ...
 # use specification
 addItemToArray(arr,it);
 if(arr == it)
 {...}
 ...
}
```

As demonstrated in the listing above, SPECGURU generates paired test cases to verify each inferred points-to relationship. For instance, when validating the alias relationship between parameters arr and it in addItemToArray from Figure 5.2, the test cases specifically examine whether this pointer relationship is preserved. If both if statements in the paired test cases pass, the specification is considered correct, as it accurately captures the pointer behavioral semantics of the original implementation. This systematic validation process extends throughout the entire call hierarchy, applying the same rigorous validation to every function from leaf nodes to their callers. Such comprehensive validation ensures the integrity of all specifications generated during SPECGURU's bottom-up analysis, effectively mitigating the risk of error propagation across the specification chain.

5.3 Approach

5.3.1 Preprocessing

Given a set of functions $\mathbb{F}$ in a targeted library, since SPECGURU focuses on generating API specifications related to pointer analysis for functions in $\mathbb{F}$, it is required that the function's parameters or return value are of pointer types. This requirement is because point-to information naturally propagates from callee to caller via pointer-type parameters and return values, which are the channels through which aliasing information flows. SPECGURU excludes functions where either the return type and all parameter types are non-pointers, or only one of them is a pointer type, sets their specification function bodies to empty to indicate the absence of pointer relationships, and adds them to the summarized specification set $\mathbb{S}$. This helps avoid unnecessary LLM calls. The targeted functions are defined as:

$$\mathbb{F}' = \{f \in F \mid |\{t_i \mid \rho(t_i), i \in \{r\} \cup \{0, 1, ..., n\}\}| \geq 2\}$$

where t_r represents the return value type, $t_0, ..., t_n$ represent parameter types, and $\rho(t)$ indicates whether type t is a pointer type. Furthermore, we categorize functions in $\mathbb{F}'$ into leaf function set $\mathbb{L}$ and non-leaf function set $\mathbb{B}$ based on their callees for generating specifications in a bottom-up manner.

Definition 5.1 (Leaf Function) *A function ℓ in $\mathbb{L}$ is a leaf function if and only if the set of its callees $Callee(\ell)$ satisfies: $Callee(\ell) = \emptyset$ or, $Callee(\ell) = \{c \mid c \in Callee(\ell), c \notin \mathbb{F}$ or $c = \ell\}$.*

Example 5.1 *Leaf functions can only call functions defined outside the library (external APIs). Functions like* strdup, castAwayConst *and* addItemToArray *in Fig-*

ure 5.2 are leaf functions, as strdup*'s call to* memcpy *from the C standard library does not affect its leaf status since* memcpy *is an external API.*

Definition 5.2 (Non-Leaf Function) *A function δ in $\mathbb{B}$ is a non-leaf function if and only if its callee set $Callee(\delta)$ satisfies: $Callee(\delta) \cap \{m \mid m \in \mathbb{F}, m \neq \delta\} \neq \emptyset$.*

Example 5.2 cJSON_addItemToObject *and* assignStringToItem *in Figure 5.2 are non-leaf functions since they call other functions within the same library.*

Functions in $\mathbb{F}'$ often call third-party APIs without available source code. For such cases, we do not download the source code of these third-party APIs. Instead, we leverage LLMs that use their prior code training to infer external API functionalities and points-to behaviors from API signatures and calling context. For example, C standard library functions typically exist in binary form. Given char* result = strstr(str, substr) called in a API under analysis, LLMs can infer that this is a string search function returning the first occurrence of the substring, without requiring access to source code or documentation. Even if LLMs make incorrect inferences, all generated specifications are validated through differential testing (See Section 5.3.3 for details), ensuring that incorrect specifications are filtered out and only correct and executable specifications are retained.

5.3.2 Bottom-Up Specification Inference

Based on the categorized sets of leaf functions $\mathbb{L}$ and non-leaf functions $\mathbb{B}$, SPECGURU employs a bottom-up analysis approach, starting with the leaf functions. As shown in Figure 5.3, to get the candidate specification S'_ℓ for each leaf function $\ell \in \mathbb{L}$:

$$S'_\ell \leftarrow \mathcal{M}(D_{spec}, E, C_\ell)$$

$$S'_\ell \leftarrow \mathcal{M}(D_{spec}, E, C_\ell) \quad (1) \qquad S'_\delta \leftarrow \mathcal{M}(D_{spec}, E, C_\delta, S_{cs}) \quad (4)$$
$$S_\ell \leftarrow \mathcal{D}\big(T^1(S'_\ell), T^2(C_\ell)\big) \quad (2) \qquad S_\delta \leftarrow \mathcal{D}\big(T^1(S'_\delta), T^2(C_\delta)\big) \quad (5)$$
$$\mathbb{S} \leftarrow \mathbb{S} \cup \{S_\ell\} \quad (3) \qquad \mathbb{S} \leftarrow \mathbb{S} \cup \{S_\delta\} \quad (6)$$

Figure 5.3: Leaf and Non-leaf function specification inference

the prompts fed into the language model $\mathcal{M}$ include specification description D_{spec}, examples of the specifications E, and the source code of the leaf function C_ℓ. Once the candidate specification S'_ℓ is generated, a differential test $\mathcal{D}\big(T^1(S'_\ell), T^2(C_\ell)\big)$ is conducted. If the test passes, the specification S_ℓ (which is the same as S'_ℓ but has successfully passed the differential test and been selected from the specification candidate set; see details in Section 5.3.3) is considered correct and added to the specification set $\mathbb{S}$. This process continues until all leaf function specifications have been added to $\mathbb{S}$.

Then SPECGURU proceed to handle each non-leaf function $\delta \in B$ recursively. Similar to obtaining specifications for leaf functions, to get candidate specification S'_δ for a non-leaf function δ, the prompt to the language model $\mathcal{M}$ not only includes the desired specification description D_{spec}, examples of the specifications E, the source code of the non-leaf function C_δ, but also include information about δ's callees S_{cs}:

$$S'_\delta \leftarrow \mathcal{M}(D_{spec}, E, C_\delta, S_{cs})$$

The callee information S_{cs} varies based on two types of non-leaf functions:

- *Non-leaf functions without mutual recursion:* For functions that do not involve mutual recursion, S_{cs} contains only the specifications of its callees that have already been summarized in $\mathbb{S}$:

$$S_{cs} \leftarrow Callee_F(\delta) \cap \mathbb{S}$$

where $Callee_F(\delta)$ represents the set of callees in F, excluding external API calls.

- *Non-leaf functions with mutual recursion:* For functions involving mutual recursion, $Callee_F(\delta)$ contains at least one unsummarized callee. Therefore, S_{cs} includes both the specifications of summarized callees and the source code of unsummarized callees:

$$S_{cs} \leftarrow \{Callee_F(\delta) \cap \mathbb{S}\} \cup \{Callee_F(\delta) \cap \{F - \mathbb{S}\}\}$$

For mutually recursive functions, the combined source code may exceed the LLM's token limit. To address this, we first calculate the total tokens after including all callees' specifications or source code. If this exceeds the maximum limit, we iteratively remove the callee with the highest token count from the mutually recursive function set until the total falls within the token limit.

Once S'_δ is obtained, it undergoes differential testing to validate its correctness. If the test passes, S_δ (same as S'_δ but has passed the differential test) is added to $\mathcal{S}$. This recursive process ensures that specifications are generated in a bottom-up manner, with each non-leaf function building upon the validated specifications of its dependencies.

Example 5.3 *For 5 functions in Figure 5.2,* SPECGURU *firstly generates specifications for leaf functions* castAwayConst*,* strdup*, and* addItemToArray*. For non-leaf function* assignStringToItem*, since it calls* castAwayConst *and* strdup*, their specifications are combined with* assignStringToItem*'s source code as input to the LLM to generate its specification. Similarly, for another non-leaf function* cJSON_addItemToObject*, the specifications of* addItemToArray *and* assignStringToItem *are combined with its source code as input to generate its specification.*

5.3.3 Specification Self-Validation

Since LLMs can make mistakes, we cannot rely on a single output S'_f from LLMs for identifying the ground truth $\hat{S}_f$ (the ideal specification that correctly captures all pointer relationships Φ for function f). Instead, SPECGURU generates multiple specification candidates for function f and compiles each generated specification S'_{f_j}. We drop all specifications that fail to compile or execute properly. Only specifications that successfully pass both compilation and execution tests are then included in the candidate solution set $\mathcal{S}'_f = \{S'_{f_1}, S'_{f_2}, ..., S'_{f_n}\}$. This validation step ensures that all candidates in $\mathcal{S}'_f$ are syntactically correct and executable, eliminating potential issues with malformed or non-functional specifications. SPECGURU then selects from $\mathcal{S}'_f$ the candidate solution S'_{f_i} that most closely matches the ground truth $\hat{S}_f$. Nonetheless, this approach presents two key challenges:
(1) How to validate that each candidate specification S'_{f_i} correctly preserves the pointer relationships Φ?
(2) Since we have no access to the ground truth $\hat{S}_f$ as a reference, how can SPECGURU select the validated candidate specification S'_{f_i} from $\mathcal{S}'_f$ that most closely matches $\hat{S}_f$?

5.3.3.1 Validation

In the validation stage, we continue to employ LLMs to generate test cases T_i for validating each specification candidate S'_{f_i}. The feasibility of using LLM-generated test cases to validate LLM-identified specifications is based on the following assumptions [25]:

- The generation of specification candidate S'_{f_i} and test case T_i are independent processes with distinct context prompts;

- The types of errors are diverse, making the probability that a faulty test case T_i correctly validates an incorrect specification candidate S'_{f_i} very low.

To further reduce false positives caused by LLM hallucinations and to validate whether S'_{f_i} preserves the same pointer properties Φ as the API source code C_f, SPECGURU generates two test cases for each S'_{f_i}: $T_i^1(S'_{f_i})$ and $T_i^2(C_f)$. The only difference between $T_i^1(S'_{f_i})$ and $T_i^2(C_f)$ is that the $T_i^1(S'_{f_i})$ calls the specification candidate S'_{f_i} while $T_i^2(C_f)$ calls the source code C_f. For simplicity, we use T_i^1 and T_i^2 to represent $T_i^1(S'_\ell)$ and $T_i^2(C_f)$ when it does not cause confusion.

Therefore, for candidate set $\mathcal{S}'_f$, SPECGURU generate a set of test case pairs:

$$T = \{\{S'_{f_1}, (T_1^1, T_1^2)\}, \{S'_{f_2}, (T_2^1, T_2^2)\}, ..., \{S'_{f_n}, (T_n^1, T_n^2)\}\}$$

to filter out specification candidates that fail differential testing. Specifically, for the generation of T_i^1, the language model $\mathcal{M}'$ is provided with a prompt that includes: (1) a test case description D_{test}, which outlines the requirements for creating test cases to validate the function's behavior; (2) the identified property instances description P for the specification candidate S'_{f_i} (e.g., ϕ_{copy} property instance of (array->child, item) of addItemToArray function); and (3) the source code C_f of the function f. These components are combined to instruct $\mathcal{M}'$ to generate a test case T_i^1 that exercises the function and validates the specified properties:

$$T_i^1 \leftarrow \mathcal{M}'(D_{test}, P, C_f)$$

and T_i^2 is derived by **replacing** the function implementation C_f in T_i^1 with specification candidate S'_{f_i}:

$$T_i^2 \leftarrow \mathcal{R}(T_i^1, S'_{f_i})$$

The only difference between T_i^1 and T_i^2 is the function body of f, which allows us to perform differential testing between them:

$$\mathcal{D}(T_i^1, T_i^2)$$

By comparing the outputs between T_i^1 and T_i^2, SPECGURU can identify any discrepancies or inconsistencies, which may indicate potential inaccuracies in S'_{f_i} or flaws in the generated test cases T_i^1 and T_i^2. If the outputs of T_i^1 and T_i^2 are consistent, SPECGURU consider S'_{f_i} to be a correct candidate specification. This differential testing process helps to increase our confidence in the correctness of the instance S'_{f_i} and the effectiveness of the generated test cases.

Example 5.4 *Consider the* cJSON_addItemToArray *function from Section 5.2. SPECGURU first employs an LLM to detect the ϕ_{copy} property between the first parameter (*array*) and second parameter (*item*), generating a specification candidate S'_{f_1} containing corresponding ϕ_{copy} property instance. Next, SPECGURU makes an independent LLM call with a different prompt context to create* `Test_Case_1` *with two key components: (1) Invocations of* cJSON_addItemToArray *using the original implementation, and (2) Validation checks for pointer aliasing between* array *and* item*.*

SPECGURU then derives `Test_Case_2` *by duplicating* `Test_Case_1` *and replacing* cJSON_addItemToArray*'s implementation with specification candidate S'_{f_1}. After separately compiling and executing both test cases, SPECGURU compares their outputs. Consistent results between the test cases confirm that S'_{f_1} accurately captures the pointer property demonstrated in the original implementation.*

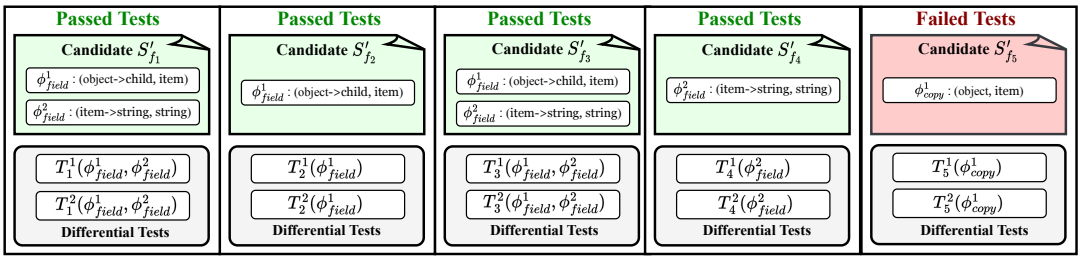


Figure 5.4: Specification candidates S'_{f_1}, S'_{f_2}, S'_{f_3}, and S'_{f_4} pass the differential test (inliers), candidate S'_{f_5} fails (outlier).

5.3.3.2 Selection

To identify the candidate S'_{f_i} that most closely aligns with the target solution $\hat{S}_f$, SPECGURU employ a strategy inspired by the classic RANSAC algorithm from machine learning [25, 43]: A candidate S'_{f_i} is considered to best align with $\hat{S}_f$ if it satisfies two criteria: (1) it passes the differential test, confirming its behavioral consistency with the original function implementation, and (2) it contains the maximum number of pointer analysis property instances, indicating it captures the most comprehensive set of properties.

The inliers are then grouped based on identical sets of property ϕ instances they contain. The objective is to identify the group(s) whose shared property instance set has the maximum cardinality. If a single group possesses the largest set of property instances, a representative S'_{f_i} is selected from it. When multiple groups have property instance sets of equal size, a representative is randomly chosen from each maximal group. These representatives are merged by taking the union of their property instances to form the final candidate solutions. While selecting groups with the maximum number of property instances may not guarantee a complete set of properties, this union-based merging strategy helps mitigate this limitation by combining property instances from multiple maximal groups, thereby increasing the likelihood of capturing a more comprehensive specification. This approach ensures

we prioritize specifications that not only have the most property instances, but also exhibit consistency in their property instance composition.

Example 5.5 *As shown in Fig. 5.4, candidate specifications for the function* cJSON_addItemToObject *are evaluated:* S'_{f_1}, S'_{f_2}, S'_{f_3}, S'_{f_4}, *and* S'_{f_5}. *These candidates contain the following points-to property instances:* S'_{f_1} *and* S'_{f_3} *include* $\{\phi^1_{field}, \phi^2_{field}\}$; S'_{f_2} *includes* $\{\phi^1_{field}\}$; S'_{f_4} *includes* $\{\phi^2_{field}\}$; *and* S'_{f_5} *includes* $\{\phi^1_{copy}\}$. *After differential testing,* S'_{f_1}, S'_{f_2}, S'_{f_3}, *and* S'_{f_4} *pass and are classified as inliers, while* S'_{f_5} *fails due to its incorrect* ϕ^1_{copy} *property and is labeled as an outlier. The inliers are grouped based on identical sets of property instances:* $\{S'_{f_1}, S'_{f_3}\}$ *share* $\{\phi^1_{field}, \phi^2_{field}\}$, $\{S'_{f_2}\}$ *contains* $\{\phi^1_{field}\}$, *and* $\{S'_{f_4}\}$ *contains* $\{\phi^2_{field}\}$. *Since the group* $\{S'_{f_1}, S'_{f_3}\}$ *has the largest set of property instances (two instances:* ϕ^1_{field}, ϕ^2_{field}*),* SPECGURU *selects either* S'_{f_1} *or* S'_{f_3} *as the representative for the desired specification* $\hat{S}_f$, *which best captures the function's behavior.*

5.4 Evaluation

5.4.1 Experiment Setup

5.4.1.1 Research Questions

Our evaluation aims to answer the following research questions:

- **RQ1. (Comparison with SOTA)** Can SPECGURU outperform the state-of-the-art specification generation methods?
- **RQ2. (Ablation: Bottom-Up Analysis)** How effective is the bottom-up API specification summarization?

Table 5.1: Statistics of 15 C third-party libraries. #APIs is the number of function definitions in the library, #LoC is the lines of code, #TSpec is the number of verified specifications, #Stars is the github/gitlab stars.

ID	Library	#Version	#APIs	#LoC	#TSpecs	#Stars
1	`libvpx`	1.15.2	31	362K	10	934
2	`libtiff`	4.7.0	172	108K	15	90
3	`liblouis`	3.34.0	34	37K	4	298
4	`cJSON`	1.7.15	112	10K	24	10.1K
5	`zlib`	1.3.1	84	32K	6	6.2K
6	`libmagic`	5.45	33	2.4K	10	1.2K
7	`lcms`	2.17	286	45K	19	634
8	`cstr`	1.0.0	83	18K	10	1.6K
9	`parson`	1.5.3	144	3.3K	30	1.3K
10	`libpcap`	1.10.5	84	69K	7	2.9K
11	`utf8`	1.0.0	38	4K	14	1.6K
12	`miniz`	3.0.2	175	26K	9	2.1K
13	`bstring`	1.0.0	130	9K	4	134
14	`Jansson`	2.14.1	95	13K	22	3.2K
15	`lodepng`	1.0.0	74	12K	9	2.2K

- **RQ3. (Ablation: Specification Validation)** How does specification validation affect the performance of SPECGURU?

- **RQ4. (Practical Usefulness)** How do the SPECGURU-generated specifications enhance static analysis in practical applications?

5.4.1.2 Benchmarks

For our evaluation, we selected 15 widely used third-party C libraries as benchmarks, with their statistics presented in Table 5.1. These libraries provide specialized functionality beyond the C standard library. We include all 8 libraries from the dataset used by Spectre [60], along with 7 additional libraries we collected to expand our benchmark. To ensure representativeness and practical relevance, we only included libraries that either participate in Google's OSS-Fuzz [111] project or have garnered more than 1,000 stars on GitHub/Gitlab, indicating significant community adoption and real-world usage. Furthermore, these libraries have been

Table 5.2: The number of specifications generated by Spectre, c-summary, and Specguru across libraries. Numbers in parentheses indicate unique specifications not found by the other two methods.

Method	libvpx	libtiff	liblouis	cJSON	zlib	libmagic	lcms	cstr	parson	libpcap	utf8	miniz	bstring	Jansson	lodepng	Total
Spectre	4/(0)	8/(2)	1/(0)	23/(0)	5/(0)	8/(1)	16/(0)	4/(0)	18/(2)	2/(0)	14/(0)	8/(7)	2/(0)	22/(1)	6/(0)	141/(13)
c-summary	5/(2)	6/(0)	1/(0)	14/(0)	6/(0)	7/(0)	14/(1)	5/(1)	17/(2)	4/(1)	11/(0)	2/(0)	2/(1)	17/(0)	6/(0)	117/(7)
Specguru	8/(3)	13/(5)	4/(3)	23/(0)	6/(0)	8/(1)	18/(0)	8/(1)	28/(13)	6/(2)	14/(0)	3/(1)	2/(1)	21/(0)	9/(3)	171/(33)

extensively used as benchmarks in prior research [87, 144] on C library analysis and testing.

5.4.1.3 Baselines

We compared Specguru with several baselines to assess its performance.

- **Spectre** presented in Chapter 3 [60] is a state-of-the-art fuzzing-based tool designed to automatically generate aliasing specifications for C library APIs, particularly when source code is unavailable. It leverages manually crafted API driver programs to capture aliasing relationships between parameters and return values during fuzzing, producing precise aliasing specifications to enhance static program analysis.

- **c-summary** [67]: *c-summary* is a state-of-the-art static analysis-based tool that automatically generates function summaries by analyzing C function source code and converting its input/output states into semantic summaries. Although other recent automated specification generation techniques exist [5, 45, 86, 132], they are designed specifically for Java and rely on Java language features (RQ1). Consequently, these techniques are not applicable to our experimental context.

- **Isolated analysis (Specguru-IA)**: Specguru-IA serves as a baseline technique within the Specguru framework. It analyzes only the source code of the target function itself, without including any callee functions' source code, to generate specifications. This approach does not incorporate self-validation. (RQ2)

- **Complete code analysis (Specguru-CCA)**: SPECGURU-CCA functions as another baseline technique within the SPECGURU framework. It utilizes both the target function's source code and the source code of all its callees to generate specifications. Like SPECGURU-IA, it does not employ self-validation. (RQ2)

- **Bottom-up analysis without validation (Specguru-BUA)**: SPECGURU-BUA operates as a baseline within the SPECGURU framework. It employs the same bottom-up analysis strategy as the full SPECGURU implementation but lacks the self-validation component. (RQ2, RQ3)

5.4.1.4 Ground Truth.

To the best of our knowledge, there are no existing API specifications for the 15 benchmark C libraries. Therefore, to establish ground truth specifications, we collected all API specifications generated by all methods used in our evaluation and manually verified each specification against the actual source code implementation. We additionally used the static analysis tool SVF [120] to assist in verifying pointer relationships, providing an automated cross-check to complement the manual inspection. We followed Definition 2.6 to construct ground truth based on the fundamental pointer relationships. Through careful source code analysis and verification, we identified all correct specifications as the ground truth for each library. To ensure accuracy, the ground truth was cross-checked by a PhD candidate and a postdoctoral researcher. The final ground truth is shown in Table 5.1 under the column #TSpec. The number of specifications in #TSpec varies across libraries, mainly because we focus on points-to specifications generation between parameters and return values. Some libraries are centered on numerical computation or internal logic and involve few pointer interactions between arguments and return values, leading to fewer relevant specifications.

5.4.1.5 Evaluation Metrics

We use standard metrics from information retrieval [102] to evaluate the performance of specification generation techniques. If a generated specification matches the ground truth, it is considered a true positive (TP); otherwise, it is a false positive (FP). Specifications in the ground truth that are not generated are counted as false negatives (FN).

5.4.1.6 Implementation

We implemented SPECGURU using GPT-4-1106-preview [97] for specification generation and Claude-3.5-sonnet-20240620 [4] for test case generation and validation, both with temperature=1.0. The complete pipeline costed approximately $360 in API usage ($130 for specification generation, $230 for validation) and required 10 hours of computational time. Our approach is not tied to specific LLMs; any models with capabilities similar to GPT-4 should work effectively. For evaluating API points-to specifications, we employed a widely-used C/C++ static analysis framework SVF [120] with Andersen's pointer analysis [3], an inclusion-based points-to analysis through constraint resolution. In the specification validation, we compare points-to relations derived from original API source code against those obtained using our specifications through dynamic differential testing. Experiments ran on a 64-bit Linux machine (Ubuntu-22.04) with a 1.8GHz 8-Core AMD Ryzen 7 5700U CPU and 16GB RAM, using OpenAI and Anthropic Claude APIs (500 RPM limit each). We repeated each experiment five times to mitigate randomness.

5.4.2 RQ1: Comparison with SOTA

Table 5.2 shows that Spectre infers 141 specifications across 15 libraries, while SPECGURU generates 171 specifications, representing an increase of about 21%

over Spectre. Similarly, c-summary infers 117 specifications, which is about 46% fewer than SPECGURU. Additionally, SPECGURU identifies 33 unique specifications not found by the other two methods, compared to Spectre's 13 unique specifications and c-summary's 7 unique specifications.

The reason that c-summary fails to generate many specifications is due to its inability to handle complex language features. Specifically, c-summary is based on static code analysis but lacks fine-grained modeling of complex language features. c-summary can only process a subset of C language syntax, struggling with complex features such as arrays, arithmetic operations, and calls to external APIs without source code (Section 5.2 in [67]). For example, consider the function utf8codepoint that `c-summary` fails to generate the specification:

```
utf8_int8_t* utf8codepoint(const utf8_int8_t* str,
              utf8_int32_t* out_codepoint)
{
  if (0xf0==(0xf8&str[0])) {
    *out_codepoint=((0x07&str[0])<<18)|((0x3f&str[1])<<12)|((0x3f&str[2])
   <<6)|(0x3f&str[3]);
    str+=4;
  ...
  } else {
    *out_codepoint=str[0];
    str+=1;
  }
  return (utf8_int8_t *)str;
}
```

This function heavily relies on pointer arithmetic (str += N) and bitwise manipulations to decode a UTF-8 codepoint from the input. However, c-summary cannot

precisely model such pointer updates and their relation to the return value. As a result, it fails to infer the specification that the return value points into the same string as the input parameter str, albeit at a different offset.

In contrast, Spectre is based on fuzzing and determines whether alias relationships exist between parameters and return values by observing API execution results. Spectre's dynamic nature allows it to avoid the need for detailed analysis rules like c-summary, instead relying solely on execution results for judgment, which leads to better specification generation results, discovering 14 more specifications than c-summary. However, the limitation of Spectre's dynamic approach is that it can only observe results from executed paths, and cannot analyze corner cases that are not executed to generate specifications, resulting in fewer specifications generated compared to SPECGURU.

However, we also observe that Spectre identifies 13 specifications and c-summary identifies 7 specifications that SPECGURU fails to generate. Among these 20 specifications, 17 were not produced by SPECGURU because the self-validation mechanism conservatively filtered out some candidates when the LLM could not generate executable test cases or when the tests failed to achieve adequate branch coverage of the API. While this self-validation feature introduces some false negatives, it ultimately improves the overall generation accuracy and can sometimes yield more true positives by preventing error propagation during the bottom-up generation process. A detailed analysis of the self-validation feature's performance is presented in Section 5.4.4. The remaining 3 specifications involved cases of mutual recursion, where two functions call each other cyclically (i.e., function A calls function B, which in turn calls function A). In such cases, SPECGURU typically analyzes the source code of both A and B simultaneously. However, for these 3 specifications, the combined code exceeded the LLM's context window and com-

Table 5.3: Comparison of four strategies for specification inference: Specguru-IA/CCA/BUA, and Specguru. #Diff is the average number of test cases for each validated specification. #Tokens is the average number of tokens in the prompt for generating specifications for each API function.

Strategies	Metrics	Libraries															Average	Total (Precision)
		libvpx	libtiff	liblouis	cJSON	zlib	libmagic	lcms	cstr	parson	libpcap	utf8	miniz	bstring	Jansson	lodepng		
	#TSpecs	10	15	4	24	6	10	19	10	30	7	14	9	4	22	9	12.87	193
Specguru -IA	#TP	8	11	4	13	5	9	13	6	19	4	13	5	2	12	5	8.60	129 (73.7%)
	#FP	2	2	0	4	2	0	2	7	3	0	0	9	11	2	2	3.07	46
	#Tokens	723	619	534	187	416	287	528	98	188	573	267	577	289	198	423	393.80	5907
Specguru -CCA	#TP	10	12	3	16	4	6	15	6	22	4	13	2	1	14	7	9.00	135 (67.2%)
	#FP	3	5	2	4	1	0	3	7	9	3	0	11	11	4	3	4.40	66
	#Tokens	4132	4986	4218	1197	3432	1166	2783	945	1614	3212	682	1767	1410	465	2987	2333.07	34996
Specguru -BUA	#TP	7	12	4	22	7	7	17	7	20	6	14	5	4	14	7	10.20	153 (78.1%)
	#FP	0	2	0	7	1	0	3	5	5	1	1	7	9	0	2	2.87	43
	#Tokens	867	872	643	502	874	626	578	370	504	765	375	775	610	254	507	608.13	9122
BUAV	#TP	8	13	4	23	6	8	18	8	28	6	14	3	2	21	9	11.40	171 (91.4%)
	#FP	2	1	0	2	1	0	1	2	1	1	0	0	3	2	0	1.07	16
	#Diff	3.70	2.50	3.65	2.06	4.65	1.80	2.80	2.80	1.90	3.10	1.67	4.53	3.40	3.50	4.20	3.04	42.56
	#Tokens	834	683	543	425	853	646	564	319	662	712	366	790	648	234	536	587.67	8815

prehension capacity, causing certain elements to be overlooked and resulting in specification generation failure.

Answer to RQ1: Specguru significantly outperforms the state-of-the-art API specification inference tools, generating 21% more specifications than Spectre and 46% more specifications than c-summary across the 15 benchmark libraries. Additionally, Specguru identifies 33 unique specifications not found by the other two methods.

5.4.3 RQ2: Bottom-Up Analysis

Table 5.3 presents the results of three different strategies—Specguru-IA, Specguru-CCA, and Specguru-BUA—for generating specifications from 15 third-party libraries. The table shows the number of ground-truth specifications (#TSpecs), true positives (#TP), false positives (#FP), and tokens processed under each strategy.

Specguru-IA analyzes only the target function's source code without including callees. It identifies 175 (#TP + #FP = 129 + 46) specification candidates, of which 129 are true positives. Its false negatives of 64 out of 193 exceed those of

SPECGURU-CCA (58 out of 193) and SPECGURU-BUA (40 out of 193). The false positives of 46 out of 175 for SPECGURU-IA are lower than SPECGURU-CCA (66 out of 201) but higher than SPECGURU-BUA (43 out of 196). Libraries such as `cJSON` and `cstr` frequently employ wrapper functions. When analyzing APIs like cJSON_GetObjectItem without access to its callee get_object_item, LLMs struggle to identify pointer properties Φ (Definition 2.6). Despite using fewer tokens (average 393.80), this limited context significantly reduces SPECGURU-IA's effectiveness.

SPECGURU-CCA includes both the target function and all its callees' source code. It generates 201 specification candidates (135 + 66), with 135 correct. While its false negatives decrease to 58, its false positives increase to 66, likely due to LLM hallucination when processing large code volumes. SPECGURU-CCA processes the most tokens among all strategies, averaging 2333.07 tokens per function.

SPECGURU-BUA employs a progressive approach, first summarizing specifications for leaf functions and then analyzing non-leaf functions using these summaries. It identifies 196 specification candidates (153 + 43) with 153 correct specifications, achieving the lowest false negatives (40) while maintaining moderate false positives (43). Processing an average of 608.13 tokens, SPECGURU-BUA requires more computation than SPECGURU-IA but substantially less than SPECGURU-CCA, offering an effective balance between accuracy and efficiency.

Figure 5.5 also shows that SPECGURU-BUA consistently outperforms the other methods across all evaluation metrics. It achieves the highest precision (0.781 vs. 0.737 for SPECGURU-IA and 0.672 for SPECGURU-CCA), recall (0.793 vs. 0.668 for SPECGURU-IA and 0.699 for SPECGURU-CCA), and F1 score (0.787 vs. 0.701 for SPECGURU-IA and 0.685 for SPECGURU-CCA). While these results establish SPECGURU-BUA as the most effective strategy, its lack of validation mechanisms remains problematic. Of its 196 generated specifications, 43 were false positives,

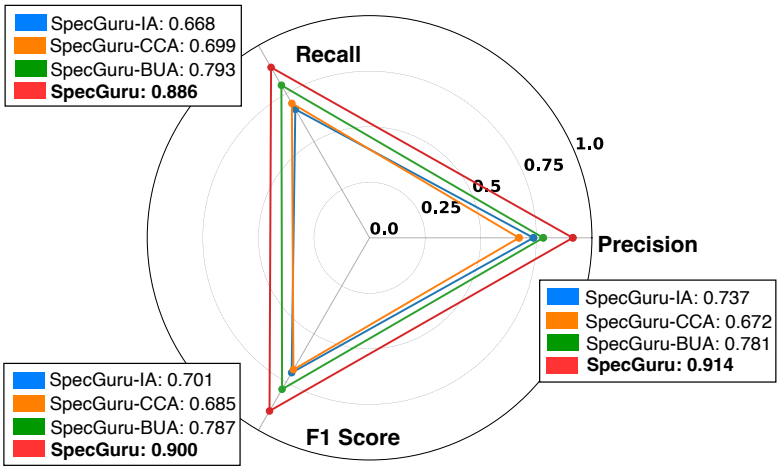


Figure 5.5: Comparison of four strategies in terms of precision, recall, and F1 Score.

underscoring the necessity of validation to ensure accuracy and prevent error propagation.

Answer to RQ2: The bottom-up analysis strategy is better than both isolated analysis and complete code analysis in terms of both quantity and quality of correct specification inference.

5.4.4 RQ3: Specification Validation

The self-validation mechanism significantly improved precision from 78.1% of SpecGuru-BUA to 91.4% of SpecGuru, as shown in Table 5.3. This demonstrates that self-validation effectively prevents the propagation and accumulation of erroneous specifications. SpecGuru maintains the highest precision (0.914), recall (0.886), and F1 score (0.900) compared to all other strategies, as illustrated in Figure 5.5.

For certain libraries (`zlib`, `miniz`, `bstring`), SpecGuru-BUA identifies more true positives than SpecGuru, indicating that the self-validation process occasionally fails to generate valid test cases and thus conservatively excludes some specification candidates. However, self-validation significantly reduces false positives

while paradoxically enabling the discovery of more correct specifications in other libraries. This occurs because early error elimination prevents error propagation during the bottom-up summarization process.

Among the 16 false positives from SPECGURU that passed differential testing, the primary issue stems from limitations in test-based validation, which occasionally fails to generate sufficiently diverse test cases that would expose incorrect specifications. Despite these false positives, the 171 correct specifications generated by our tool represent 88.6% of all validated specifications in our benchmark. The validation process required an average of only 3.04 test case generation attempts per specification, demonstrating the efficiency of LLM-based test case generation for specification validation. For libraries with intricate data structures and complex call relationships, even LLMs with strong code comprehension capabilities encounter difficulties. Similar to human test engineers who require domain expertise, LLMs struggle with certain complex codebases. For instance, the `miniz` library—which implements sophisticated compression algorithms with complex data structures—presented significant challenges: although 9 APIs were identified as having property Φ, only 3 passed the validation process due to these inherent complexity barriers.

Answer to RQ3: The specification validation via LLM-generated test cases and differential testing effectively minimized the accumulation and propagation of incorrect specifications during bottom-up specification inference.

5.4.5 RQ4: Practical Usefulness

To evaluate the practical utility of our generated specifications, we conducted experiments with two widely-used static analysis applications: pointer alias analysis [35] and taint analysis [82]. For each application, we compared the results obtained

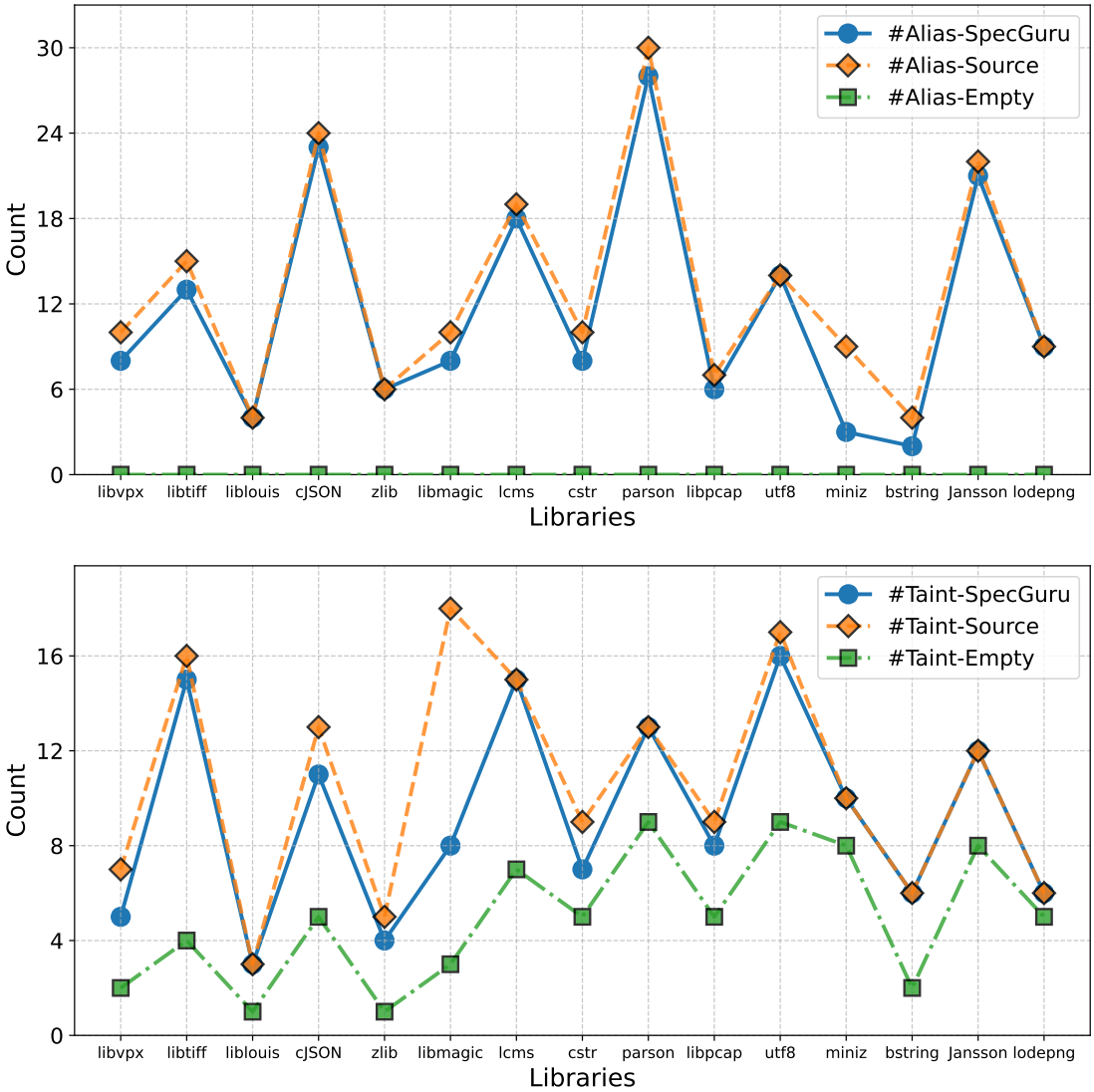


Figure 5.6: Static pointer alias analysis (upper) and taint analysis (lower).

using: (1) SPECGURU-generated API specifications, (2) original API source code, and (3) no specifications. These experiments evaluate the effectiveness of the inferred points-to specifications in two downstream tasks: taint flow detection and pointer alias analysis. The specifications aim to improve bug detection by identifying additional tainted flows and to enhance the precision of alias information, which is critical for accurate reasoning about program behavior.

For alias analysis, we compared three configurations: Alias-SpecGuru (using generated specifications), Alias-Source (using original API source code), and Alias-Empty (no points-to information). We inserted alias checks at 193 API call points where ground truth specifications existed. As shown in Figure 5.6 (upper), Alias-Empty yielded limited alias results due to the absence of API points-to information. Alias-SpecGuru correctly identified aliases for 171 aliases. While Alias-Source covered all 193 alias checkpoints due to having access to the complete API source code, Alias-SpecGuru achieved 88% coverage (171/193) without requiring access to API implementation details.

For taint analysis, we inserted source and sink points in client code and compared three configurations: Taint-SpecGuru (using generated specifications), Taint-Source (using original API source code), and Taint-Empty (no taint propagation information). Figure 5.6 (lower) shows that Taint-Empty detected only 74 taint flows, which were flows that did not pass through APIs requiring pointer information. Taint-SpecGuru identified 139 flows, while Taint-Source found 159 flows due to having complete API implementation information. The improvement of Taint-SpecGuru over Taint-Empty demonstrates that SpecGuru-generated specifications significantly enhance the ability of static analysis tools to detect potential security vulnerabilities, even without access to API source code.

Answer to RQ4: The specifications generated by SpecGuru can replace the source code in static pointer analysis. Additionally, taint analysis demonstrates the practical benefits of SpecGuru, as it can enhance the detection of potential security vulnerabilities by providing points-to information.

5.5 Related Work

Specification Inference. Existing methods for specification inferenc can be broadly categorized as static or dynamic. Static methods [7, 16, 67, 83, 105, 115] infer specifications by analyzing the source code's structure, variable types, and flow patterns. Dynamic methods [17, 18, 33, 60, 92], on the other hand, infer specifications by observing program behavior during execution using techniques like runtime monitoring and profiling. Some methods also leverage machine learning to infer specifications, such as unsupervised API aliasing inference [38] and semi-supervised taint specification inference [31]. The types of specifications inferred can range from taint analysis [33, 118], which monitors sensitive data flow, to points-to analysis [17, 18], which identifies potential object references for pointer variables. Most of these works focus on generating Java API specifications. The efforts most relevant to our work, which target C API specification generation, are the static analysis-based c-summary [67] and the dynamic analysis-based (Fuzzing) Spectre [60]. Evaluation results indicate that c-summary struggles with complex language features, while Spectre fails to handle corner cases, limiting their ability to generate comprehensive C library specifications.

Automatic Test Case Generation. Automated test case generation can significantly cut down the time and resources spent on manual test case creation. Prior research in this area, mainly employing search-based heuristics [5, 45, 86, 100] or pre-trained language models like BART [69, 104, 130], has its drawbacks. While search-based heuristic methods often struggle with the diversity and volume of the test cases they generate, pre-trained language models usually necessitate extensive training and labeled data, which might not be easily accessible for different properties or new datasets.

LLM for Static Program Analysis. Recent research has leveraged LLMs for various static program analysis tasks, including verification strategy selection [119], software specification inference [65,89], and loop invariant generation [81,143]. Approaches like [133, 134] integrate LLMs with external tools to enhance dataflow analysis and bug detection. These orthogonal research directions and their innovations combining formal analysis with LLMs' code comprehension capabilities inspired our SPECGURU design.

5.6 Threats to validity

Datasets. A limitation of our study is that the evaluated open-source libraries may have been included in the training data of the employed LLMs, potentially introducing bias. Ideally, experiments would use novel, unseen libraries. However, specifications for rarely used libraries would provide limited practical value, somewhat mitigating this concern.

Correctness. While LLMs effectively generate specifications for complex libraries, they cannot guarantee soundness or completeness due to issues like hallucinations and limited test coverage. This is especially challenging for APIs with complex object parameters, where generating comprehensive test cases is difficult. Despite these limitations, our method produces more specifications than existing tools with high precision and a low false positive rate. In the future, we will focus on improving test coverage and handling complex parameters.

Model Selection. In our evaluation, we used GPT-4 and Claude 3.5, and empirically found that LLMs with capabilities similar to GPT-4 (such as Gemini 1.5 Pro) perform comparably in specification inference and validation tasks. While more advanced models might yield better results, our focus is on the method for

automatically generating and validating specifications, rather than the impact of different models.

5.7 Chapter Summary

This chapter introduces SPECGURU, a novel framework that harnesses Large Language Models (LLMs) to generate precise points-to API specifications for third-party C libraries through a hierarchical, bottom-up approach. By starting with leaf functions and progressively building validated specifications for higher-level functions, SPECGURU effectively addresses the challenges of complex semantics and incomplete API information. The integration of a robust self-validation mechanism, utilizing LLM-generated test cases and differential testing, ensures the correctness of specifications and prevents error propagation across the call hierarchy. Our experimental evaluation on 15 widely-used C libraries demonstrates that SPECGURU significantly outperforms existing specification inference tools, generating 21% more specifications than Spectre and 46% more than c-summary. These specifications significantly enhance static analysis applications, such as pointer alias and taint analysis, by providing accurate abstractions without requiring implementation details. Despite limitations in handling mutual recursion and ensuring full test coverage, SPECGURU represents a significant advancement in automated specification inference, offering a scalable and practical solution for improving the reliability and efficiency of static analysis in software systems interfacing with external APIs. Future work will focus on enhancing test case diversity and addressing complex library structures to further improve specification accuracy and coverage.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

In this thesis, we explored three critical challenges in static program analysis for software that relies on external APIs, particularly when source code is unavailable, cross-language boundaries create semantic discontinuities, or API semantics are complex. We propose automated API specification generation techniques to enhance the precision, automation, and scalability of static analysis, focusing on C/C++ library APIs and cross-language Java Native Interface (JNI) programs. By leveraging static analysis, fuzzing, and large language models (LLMs), we develop methods to generate accurate aliasing, taint, and points-to specifications, enabling effective analysis of API-dependent software without manual effort.

Three new static approaches are developed: (1) Spectre, a fuzzing-based framework that automatically infers aliasing specifications for C library APIs, even without complete source code, achieving high precision in pointer analysis through lightweight alias-check observers and grey-box fuzzing techniques. (2) CSS, a caller-sensitive specification framework that enhances cross-language taint anal-

ysis for JNI programs by integrating Java calling contexts with C/C++ behavior modeling, significantly improving precision over context-insensitive methods. (3) SPECGURU, a hierarchical LLM-driven approach that generates and validates points-to specifications for C libraries using bottom-up inference and differential testing, achieving high accuracy and scalability for complex APIs. These contributions collectively advance static analysis by providing automated, precise, and scalable solutions for real-world software systems, with evaluations demonstrating their effectiveness across benchmarks and applications.

6.2 Future Work

Refining Analysis with Program Slicing. To advance the precision of API specification generation, we plan to focus on enhancing the analysis of individual functions by leveraging program slicing techniques in the future. Overly large API code contexts often overwhelm LLMs, leading to reduced accuracy in specification generation. By employing program slicing, we can isolate code segments to critical operations between parameters and return values, significantly reducing the code scale processed by LLMs. This targeted approach enables LLMs to focus on essential behaviors, improving the accuracy of the generated specifications.

Enhancing API Call Relationship Modeling. Another direction for future work is to deepen the understanding of API call relationships to better capture their composite behaviors. The thesis notes that SPECGURU 's inference for wrapper functions, is limited by incomplete callee context. By systematically analyzing inter-API dependencies, including data-flow and control-flow relationships, we can more effectively model their interactions. Integrating static analysis of call dependencies with SPECTRE 's dynamic fuzzing techniques and using LLMs to interpret

semantic connections across API invocation paths would enable comprehensive behavior exploration.

Developing Robust Automated Specification Validation. Improving the validation of generated API specifications is critical to ensure their reliability and reduce reliance on manual intervention. The thesis points out that SPECGURU 's differential testing struggles with test case diversity, leading to false positives in complex libraries. Future work could develop a more robust, automated validation framework by combining symbolic execution, formal verification, and advanced differential testing. Additionally, equipping LLMs with contextual information, such as function semantics or interface contracts, could guide the generation of diverse, adversarial test cases, enhancing specification evaluation, minimizing false positives and negatives, and ensuring high-fidelity specifications for large-scale, rapidly evolving API ecosystems.

Generating Cross-Language Specifications from Binaries. To extend the capabilities of the CSS framework, which currently relies on C/C++ source code for cross-language taint analysis in JNI programs, future work could focus on generating specifications when only binary or executable files are available. This scenario is common in real-world applications where native library source code is proprietary or unavailable. Building on SPECTRE 's fuzzing-based approach, we plan to develop a new approach that targets native code in cross-language contexts, addressing challenges such as reconciling Java's managed memory model with C/C++'s unmanaged memory at JNI boundaries. This involves designing fuzz drivers that simulate Java-to-Native calls, capturing data flows and pointer interactions across language boundaries. LLMs can enhance this process by analyzing JNI boundary contexts, such as Java method signatures and parameter types, to guide fuzzing input generation toward relevant API behaviors, improving coverage of taint and aliasing

properties. Techniques like binary instrumentation and dynamic taint tracking can further refine specification inference by observing runtime behaviors without source code. This approach would enable precise cross-language specification generation for JNI programs, even in black-box scenarios, broadening the applicability of static analysis to diverse, real-world software systems.

Bibliography

[1] A. Albarghouthi, I. Dillig, and A. Gurfinkel. Maximal specification synthesis. *ACM SIGPLAN Notices*, 51(1):789–801, 2016.

[2] G. Ammons, R. Bodík, and J. R. Larus. Mining specifications. *ACM Sigplan Notices*, 37(1):4–16, 2002.

[3] L. O. Andersen. *Program analysis and specialization for the C programming language*. PhD thesis, University of Cophenhagen, 1994.

[4] Anthropic. Claude ai. https://claude.ai.

[5] A. Arcuri. Many independent objective (mio) algorithm for test suite generation. In *Search Based Software Engineering: 9th International Symposium, SSBSE 2017, Paderborn, Germany, September 9-11, 2017, Proceedings 9*, pages 3–17. Springer, 2017.

[6] arguslab. Nativeflowbench github issue. https://github.com/arguslab/NativeFlowBench/issues/1, 2019.

[7] S. Arzt and E. Bodden. Stubdroid: automatic inference of precise data-flow summaries for the android framework. In *Proceedings of the 38th International Conference on Software Engineering*, pages 725–735, 2016.

[8] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. *ACM sigplan notices*, 49(6):259–269, 2014.

[9] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. L. Traon, D. Octeau, and P. D. McDaniel. Flowdroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. In M. F. P. O'Boyle and K. Pingali, editors, *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14, Edinburgh, United Kingdom - June 09 - 11, 2014*, pages 259–269. ACM, 2014.

[10] O. Asare, M. Nagappan, and N. Asokan. Is github's copilot as bad as humans at introducing vulnerabilities in code? *Empirical Software Engineering*, 28(6):129, 2023.

[11] P. Avgustinov, O. de Moor, M. P. Jones, and M. Schäfer. QL: Object-oriented Queries on Relational Data. In S. Krishnamurthi and B. S. Lerner, editors, *30th European Conference on Object-Oriented Programming (ECOOP 2016)*, volume 56 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 2:1–2:25, Dagstuhl, Germany, 2016. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[12] M. Barbar and Y. Sui. Compacting points-to sets through object clustering. *Proceedings of the ACM on Programming Languages*, 5(OOPSLA):1–27, 2021.

[13] M. Barbar, Y. Sui, H. Zhang, S. Chen, and J. Xue. Live path cfi against control flow hijacking attacks. In *Information Security and Privacy: 23rd Australasian Conference, ACISP 2018, Wollongong, NSW, Australia, July 11-13, 2018, Proceedings 23*, pages 768–779. Springer, 2018.

[14] M. Barbar, Y. Sui, H. Zhang, S. Chen, and J. Xue. Live path control flow integrity. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*, pages 195–196, 2018.

[15] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The oracle problem in software testing: A survey. *IEEE transactions on software engineering*, 41(5):507–525, 2014.

[16] O. Bastani, S. Anand, and A. Aiken. Specification inference using context-free language reachability. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 553–566, 2015.

[17] O. Bastani, R. Sharma, A. Aiken, and P. Liang. Active learning of points-to specifications. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 678–692, 2018.

[18] O. Bastani, R. Sharma, L. Clapp, S. Anand, and A. Aiken. Eventually sound points-to analysis with specifications. In *33rd European Conference on Object-Oriented Programming (ECOOP 2019)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2019.

[19] E. Bendersky. pycparser. https://github.com/eliben/pycparser, 2023.

[20] R. Bodík and S. Anik. Path-sensitive value-flow analysis. In *Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '98, page 237–251, New York, NY, USA, 1998. Association for Computing Machinery.

[21] M. Böhme, V.-T. Pham, and A. Roychoudhury. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 1032–1043, 2016.

[22] S. Buro, R. L. Crole, and I. Mastroeni. On multi-language abstraction - towards a static analysis of multi-language programs. In D. Pichardie and M. Sighireanu, editors, *Static Analysis - 27th International Symposium, SAS 2020, Virtual Event, November 18-20, 2020, Proceedings*, volume 12389 of *Lecture Notes in Computer Science*, pages 310–332. Springer, 2020.

[23] C. Cadar, D. Dunbar, D. R. Engler, et al. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, volume 8, pages 209–224, 2008.

[24] L. Chaoran. *Vulnerability Detection in Android*. PhD thesis, Swinburne University of Technology, 2023.

[25] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022.

[26] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[27] X. Chen, C. Liu, and D. Song. Execution-guided neural program synthesis. In *International Conference on Learning Representations*, 2018.

[28] X. Cheng, X. Nie, N. Li, H. Wang, Z. Zheng, and Y. Sui. How about bug-triggering paths? - understanding and characterizing learning-based vulner-

ability detectors. *IEEE Transactions on Dependable and Secure Computing*, 2024.

[29] X. Cheng, G. Zhang, H. Wang, and Y. Sui. Path-sensitive code embedding via contrastive learning for software vulnerability detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA '22. ACM, 2022.

[30] S. Cherem, L. Princehouse, and R. Rugina. Practical memory leak detection using guarded value-flow analysis. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '07. Association for Computing Machinery, 2007.

[31] V. Chibotaru, B. Bichsel, V. Raychev, and M. Vechev. Scalable taint specification inference with big code. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 760–774, 2019.

[32] F. Chow, S. Chan, S.-M. Liu, R. Lo, and M. Streich. Effective representation of aliases and indirect memory operations in ssa form. In *In International Conference on Compiler Construction*, pages 253–267, Berlin, Heidelberg, 1996. Springer.

[33] L. Clapp, S. Anand, and A. Aiken. Modelgen: mining explicit information flow specifications from concrete executions. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, pages 129–140, 2015.

[34] N. Coppik, O. Schwahn, and N. Suri. Memfuzz: Using memory accesses to guide fuzzing. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, pages 48–58. IEEE, 2019.

[35] A. Diwan, K. S. McKinley, and J. E. B. Moss. Type-based alias analysis. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation*, PLDI '98, page 106–117, New York, NY, USA, 1998. Association for Computing Machinery.

[36] M. Dowd, J. McDonald, and J. Schuh. *The art of software security assessment: Identifying and preventing software vulnerabilities*. Pearson Education, 2006.

[37] Z. Du and Y. Li. Hastefuzz: Full-speed fuzzing. In *2023 IEEE/ACM International Workshop on Search-Based and Fuzz Testing (SBFT)*, pages 73–75. IEEE, 2023.

[38] J. Eberhardt, S. Steffen, V. Raychev, and M. Vechev. Unsupervised learning of api aliasing specifications. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 745–759, 2019.

[39] K. Ellis, M. Nye, Y. Pu, F. Sosa, J. Tenenbaum, and A. Solar-Lezama. Write, execute, assess: Program synthesis with a repl. *Advances in Neural Information Processing Systems*, 32, 2019.

[40] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software*

Engineering: Future of Software Engineering (ICSE-FoSE), pages 31–53. IEEE, 2023.

[41] J. Ferrante, K. J. Ottenstein, and J. D. Warren. The program dependence graph and its use in optimization. *ACM Trans. Program. Lang. Syst.*, 9(3):319–349, jul 1987.

[42] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse. AFL++: Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, Aug. 2020.

[43] M. A. Fischler and R. C. Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.

[44] W. L. for Analysis. Wala, 2018.

[45] G. Fraser and A. Arcuri. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, pages 416–419, 2011.

[46] M. Furr and J. S. Foster. Checking type safety of foreign function calls. In V. Sarkar and M. W. Hall, editors, *Proceedings of the ACM SIGPLAN 2005 Conference on Programming Language Design and Implementation, Chicago, IL, USA, June 12-15, 2005*, pages 62–72. ACM, 2005.

[47] M. Furr and J. S. Foster. Checking type safety of foreign function calls. *ACM Trans. Program. Lang. Syst.*, 30(4):18:1–18:63, 2008.

[48] K. B. Gallagher and J. R. Lyle. Using program slicing in software maintenance. *IEEE TSE*, 17(8):751–761, 1991.

[49] D. Gamble. cJSON: Ultralightweight json parser in ansi c. https://github.com/DaveGamble/cJSON.

[50] P. Godefroid, M. Y. Levin, and D. Molnar. Sage: whitebox fuzzing for security testing. *Communications of the ACM*, 55(3):40–44, 2012.

[51] P. Godefroid, M. Y. Levin, D. A. Molnar, et al. Automated whitebox fuzz testing. In *NDSS*, volume 8, pages 151–166, 2008.

[52] M. I. Gordon, D. Kim, J. H. Perkins, L. Gilham, N. Nguyen, and M. C. Rinard. Information flow analysis of android applications in droidsafe. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015, San Diego, California, USA, February 8-11, 2015*, Reston, VA, USA, 2015. The Internet Society.

[53] B. Hardekopf and C. Lin. Flow-sensitive pointer analysis for millions of lines of code. In *International Symposium on Code Generation and Optimization*, CGO '11, pages 289–298. IEEE, 2011.

[54] Y. He, Y. Gu, P. Su, K. Sun, Y. Zhou, Z. Wang, and Q. Li. A systematic study of android non-sdk (hidden) service api security. *IEEE Transactions on Dependable and Secure Computing*, 20(2):1609–1623, 2022.

[55] Infer. Facebook Infer: a tool to detect bugs in Java and C/C++/Objective-C code, 2021. https://fbinfer.com/.

[56] K. Jesse, T. Ahmed, P. T. Devanbu, and E. Morgan. Large language models and simple, stupid bugs. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 563–575. IEEE, 2023.

[57] N. D. Jones and S. S. Muchnick. Flow analysis and optimization of lisp-like structures. In *Proceedings of the 6th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 244–256, 1979.

[58] Julia Community. Calling c and fortran code, 2019. Accessed: 2025-06-17.

[59] S. Kan, Y. Gao, Z. Zhong, and Y. Sui. Cross-language taint analysis: Generating caller-sensitive native code specification for java. *IEEE Transactions on Software Engineering*, 2024.

[60] S. Kan, Y. Li, W. He, Z. Xing, L. Zhu, and Y. Sui. Spectre: Automated aliasing specifications generation for library apis with fuzzing. *ACM Transactions on Software Engineering and Methodology*, 2025.

[61] D. King, B. Hicks, M. Hicks, and T. Jaeger. Implicit flows: Can't live with 'em, can't live without 'em. In R. Sekar and A. K. Pujari, editors, *Information Systems Security, 4th International Conference, ICISS 2008, Hyderabad, India, December 16-20, 2008. Proceedings*, volume 5352 of *Lecture Notes in Computer Science*, pages 56–70. Springer, 2008.

[62] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks. Evaluating fuzz testing. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, pages 2123–2138, 2018.

[63] G. Kondoh and T. Onodera. Finding bugs in java native interface programs. In B. G. Ryder and A. Zeller, editors, *Proceedings of the ACM/SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2008, Seattle, WA, USA, July 20-24, 2008*, pages 109–118. ACM, 2008.

[64] S. Kumar and S. K. Shukla. *The State of Android Security*, pages 17–22. Springer Singapore, Singapore, 2020.

[65] T. Le-Cong, B. Le, and T. Murray. Can llms reason about program semantics? a comprehensive evaluation of llms on formal specification inference. *arXiv preprint arXiv:2503.04779*, 2025.

[66] S. Lee, J. Dolby, and S. Ryu. Hybridroid: static analysis framework for android hybrid applications. In D. Lo, S. Apel, and S. Khurshid, editors, *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, ASE 2016, Singapore, September 3-7, 2016*, pages 250–261. ACM, 2016.

[67] S. Lee, H. Lee, and S. Ryu. Broadening horizons of multilingual static analysis: Semantic summary extraction from C code for JNI program analysis. In *35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21-25, 2020*, pages 127–137. IEEE, 2020.

[68] Y. Lei and Y. Sui. Fast and precise handling of positive weight cycles for field-sensitive pointer analysis. In *Static Analysis: 26th International Symposium, SAS 2019, Porto, Portugal, October 8–11, 2019, Proceedings 26*, pages 27–47. Springer, 2019.

[69] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*, 2019.

[70] C. Li, X. Chen, R. Sun, M. Xue, S. Wen, M. E. Ahmed, S. Camtepe, and Y. Xiang. Cross-language android permission specification. In *FSE 2022*, pages 772–783.

[71] J. Li, B. Zhao, and C. Zhang. Fuzzing: a survey. *Cybersecurity*, 1(1):1–13, 2018.

[72] S. Li and G. Tan. Exception analysis in the java native interface. *Sci. Comput. Program.*, 89(PART C):273–297, Sept. 2014.

[73] S. Li and G. Tan. Finding reference-counting errors in python/c programs with affine analysis. In R. E. Jones, editor, *ECOOP 2014 - Object-Oriented Programming - 28th European Conference, Uppsala, Sweden, July 28 - August 1, 2014. Proceedings*, volume 8586 of *Lecture Notes in Computer Science*, pages 80–104, Berlin, Heidelberg, 2014. Springer.

[74] T. O. Li, W. yi Zong, Y. Wang, H. Tian, Y. Wang, and S. C. Cheung. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 14–26, 2023.

[75] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

[76] Y. Li, Y. Xue, H. Chen, X. Wu, C. Zhang, X. Xie, H. Wang, and Y. Liu. Cerebro: Context-aware adaptive fuzzing for effective vulnerability detection. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, FSE '19, page 533–544, New York, NY, USA, 2019. ACM.

[77] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. Detecting cross-language memory management issues in rust. In V. Atluri, R. D. Pietro, C. D. Jensen,

and W. Meng, editors, *Computer Security - ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part III*, volume 13556 of *Lecture Notes in Computer Science*, pages 680–700, Cham, 2022. Springer.

[78] H. Liang, X. Pei, X. Jia, W. Shen, and J. Zhang. Fuzzing: State of the art. *IEEE Transactions on Reliability*, 67(3):1199–1218, 2018.

[79] S. Liang. *The Java native interface: programmer's guide and specification*. Addison-Wesley Professional, 1999.

[80] K. Liu, Y. Liu, Z. Chen, J. M. Zhang, Y. Han, Y. Ma, G. Li, and G. Huang. Llm-powered test case generation for detecting tricky bugs. *arXiv preprint arXiv:2404.10304*, 2024.

[81] R. Liu, G. Li, M. Chen, L.-I. Wu, and J. Ke. Enhancing automated loop invariant generation for complex programs with large language models. *arXiv preprint arXiv:2412.10483*, 2024.

[82] B. Livshits and M. S. Lam. Finding security vulnerabilities in java applications with static analysis. In *Proceedings of the 14th USENIX Security Symposium*, 2005.

[83] B. Livshits, A. V. Nori, S. K. Rajamani, and A. Banerjee. Merlin: Specification inference for explicit information flow problems. *ACM Sigplan Notices*, 44(6):75–86, 2009.

[84] B. Livshits, M. Sridharan, Y. Smaragdakis, O. Lhoták, J. N. Amaral, B.-Y. E. Chang, S. Z. Guyer, U. P. Khedker, A. Møller, and D. Vardoulakis. In defense of soundiness: A manifesto. *Commun. ACM*, 58(2):44–46, 2015.

[85] D. Lo and S.-C. Khoo. Smartic: Towards building an accurate, robust and scalable specification miner. In *Proceedings of the 14th ACM SIGSOFT international symposium on Foundations of software engineering*, pages 265–275, 2006.

[86] S. Lukasczyk and G. Fraser. Pynguin: Automated unit test generation for python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pages 168–172, 2022.

[87] Y. Lyu, Y. Xie, P. Chen, and H. Chen. Prompt fuzzing for fuzz driver generation. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3793–3807, 2024.

[88] M. Furr and J. S. Foster. Polymorphic type inference for the JNI. In P. Sestoft, editor, *Programming Languages and Systems, 15th European Symposium on Programming, ESOP 2006, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2006, Vienna, Austria, March 27-28, 2006, Proceedings*, volume 3924 of *Lecture Notes in Computer Science*, pages 309–324. Springer, 2006.

[89] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu. Specgen: Automated generation of formal program specifications via large language models. *arXiv preprint arXiv:2401.08807*, 2024.

[90] W. M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[91] S. Mergendahl, N. Burow, and H. Okhravi. Cross-language attacks. In *Proceedings 2022 Network and Distributed System Security Symposium. NDSS*, volume 22, pages 1–17.

[92] F. Molina, M. d'Amorim, and N. Aguirre. Fuzzing class specifications. In *Proceedings of the 44th International Conference on Software Engineering*, pages 1008–1020, 2022.

[93] A. Møller and M. I. Schwartzbach. Static program analysis, October 2018. Department of Computer Science, Aarhus University, `http://cs.au.dk/~amoeller/spa/`.

[94] A. Møller and M. Schwarz. Automated detection of client-state manipulation vulnerabilities. *Transactions on Software Engineering and Methodology*, 23(4), August 2014. Earlier version in Proc. 34th International Conference on Software Engineering (ICSE) 2012.

[95] R. Monat, A. Ouadjaout, and A. Miné. A multilanguage static analysis of python programs with native C extensions. In C. Dragoi, S. Mukherjee, and K. S. Namjoshi, editors, *Static Analysis - 28th International Symposium, SAS 2021, Chicago, IL, USA, October 17-19, 2021, Proceedings*, volume 12913 of *Lecture Notes in Computer Science*, pages 323–345. Springer, 2021.

[96] musl libc team. musl c library. `https://musl.libc.org/`, 2023.

[97] OpenAI. Chatgpt. `https://chat.openai.com`.

[98] Oracle. Java native interface specification, 2019. Accessed: 2025-06-17.

[99] A. Ouadjaout and A. Miné. A library modeling language for the static analysis of c programs. In *International Static Analysis Symposium*, pages 223–247. Springer, 2020.

[100] A. Panichella, F. M. Kifetew, and P. Tonella. Reformulating branch coverage as a many-objective optimization problem. In *2015 IEEE 8th international*

conference on software testing, verification and validation (ICST), pages 1–10. IEEE, 2015.

[101] polyglot compiler. Jlang, 2020. Accessed: Oct. 23, 2023.

[102] D. M. W. Powers. Evaluation: From precision, recall and f-measure to roc, informedness, markedness & correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011.

[103] Python Software Foundation. Extending and embedding the python interpreter, 2019. Accessed: 2025-06-17.

[104] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

[105] M. K. Ramanathan, A. Grama, and S. Jagannathan. Static specification inference using predicate mining. *ACM SIGPLAN Notices*, 42(6):123–134, 2007.

[106] B. Roziere, J. M. Zhang, F. Charton, M. Harman, G. Synnaeve, and G. Lample. Leveraging automated unit tests for unsupervised code translation. *arXiv preprint arXiv:2110.06773*, 2021.

[107] S. Li and G. Tan. JET: exception checking in the java native interface. In C. V. Lopes and K. Fisher, editors, *Proceedings of the 26th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2011, part of SPLASH 2011, Portland, OR, USA, October 22 - 27, 2011*, pages 345–358. ACM, 2011.

[108] J. Samhi, J. Gao, N. Daoudi, P. Graux, H. Hoyez, X. Sun, K. Allix, T. F. Bissyandé, and J. Klein. Jucify: A step towards android code unification for enhanced static analysis. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*, pages 1232–1244. ACM, 2022.

[109] A.-D. Schmidt, H.-G. Schmidt, L. Batyuk, J. H. Clausen, S. A. Camtepe, S. Albayrak, and C. Yildizli. Smartphone malware evolution revisited: Android next target? In *2009 4th International conference on malicious and unwanted software (MALWARE)*, pages 1–7. IEEE, 2009.

[110] R. Schuster, C. Song, E. Tromer, and V. Shmatikov. You autocomplete me: Poisoning vulnerabilities in neural code completion. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1559–1575, 2021.

[111] K. Serebryany. {OSS-Fuzz}-google's continuous fuzzing service for open source software. 2017.

[112] C. Shi, C. C.-C. Cheng, and Y. Guan. Libdroid: Summarizing information flow of android native libraries via static analysis. *Forensic Science International: Digital Investigation*, 42:301405, 2022.

[113] Q. Shi, X. Xiao, R. Wu, J. Zhou, G. Fan, and C. Zhang. Pinpoint: Fast and precise sparse value flow analysis for million lines of code. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '18. ACM, 2018.

[114] Q. Shi, P. Yao, R. Wu, and C. Zhang. Path-sensitive sparse analysis without path conditions. In *Proceedings of the 42nd ACM SIGPLAN International*

Conference on Programming Language Design and Implementation, PLDI '21. ACM, 2021.

[115] S. Shoham, E. Yahav, S. Fink, and M. Pistoia. Static specification mining using automata-based abstractions. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis*, pages 174–184, 2007.

[116] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna. Sok: (state of) the art of war: Offensive techniques in binary analysis. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 138–157, 2016.

[117] M. Sridharan, S. J. Fink, and R. Bodik. Thin slicing. In *Proceedings of the 28th ACM SIGPLAN conference on programming language design and implementation*, pages 112–122, 2007.

[118] C.-A. Staicu, M. T. Torp, M. Schäfer, A. Møller, and M. Pradel. Extracting taint specifications for javascript libraries. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pages 198–209, 2020.

[119] J. Su, L. Deng, C. Wen, S. Qin, and C. Tian. Cfstra: Enhancing configurable program analysis through llm-driven strategy selection based on code features. In *International Symposium on Theoretical Aspects of Software Engineering*, pages 374–391. Springer, 2024.

[120] Y. Sui and J. Xue. Svf: Interprocedural static value-flow analysis in llvm. In *Proceedings of the 25th International Conference on Compiler Construction*, CC. ACM, 2016.

[121] Y. Sui, D. Ye, and J. Xue. Static memory leak detection using full-sparse value-flow analysis. In *Proceedings of the 2012 International Symposium on Software Testing and Analysis*, pages 254–264, 2012.

[122] Y. Sui, D. Ye, and J. Xue. Detecting memory leaks statically with full-sparse value-flow analysis. *IEEE Transactions on Software Engineering*, pages 107–122, 2014.

[123] M. Sutton, A. Greene, and P. Amini. *Fuzzing: brute force vulnerability discovery.* Pearson Education, 2007.

[124] A. Takanen, J. D. Demott, C. Miller, and A. Kettunen. *Fuzzing for software security testing and quality assurance.* Artech House, 2018.

[125] K. Tam, S. J. Khan, A. Fattori, and L. Cavallaro. Copperdroid: Automatic reconstruction of android malware behaviors. In *Ndss*, pages 1–15, 2015.

[126] G. Tan and J. Croft. An empirical security study of the native code in the JDK. In P. C. van Oorschot, editor, *Proceedings of the 17th USENIX Security Symposium, July 28-August 1, 2008, San Jose, CA, USA*, pages 365–378, Berkeley, CA, USA, 2008. USENIX Association.

[127] G. Tan and G. Morrisett. Ilea: inter-language analysis across java and c. In R. P. Gabriel, D. F. Bacon, C. V. Lopes, and G. L. S. Jr., editors, *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2007, October 21-25, 2007, Montreal, Quebec, Canada*, pages 39–56. ACM, 2007.

[128] T. Tan and Y. Li. Tai-e: A static analysis framework for java by harnessing the best designs of classics. *CoRR*, abs/2208.00337, 2022.

[129] The Rust Project Developers. Foreign function interface, 2019. Accessed: 2025-06-17.

[130] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617*, 2020.

[131] O. H. Veileborg and A. Møller. Eliminating abstraction overhead of Java stream pipelines using ahead-of-time program optimization. *Proceedings of the ACM on Programming Languages (PACMPL)*, 4(OOPSLA):168:1–168:29, 2020.

[132] C. Wang, J. Zhang, R. Wu, and C. Zhang. Dainfer: Inferring api aliasing specifications from library documentation via neurosymbolic optimization. *Proceedings of the ACM on Software Engineering*, 1(FSE):2469–2492, 2024.

[133] C. Wang, W. Zhang, Z. Su, X. Xu, X. Xie, and X. Zhang. Llmdfa: Analyzing dataflow in code with large language models. *Advances in Neural Information Processing Systems*, 37:131545–131574, 2024.

[134] C. Wang, W. Zhang, Z. Su, X. Xu, and X. Zhang. Sanitizing large language models in bug detection with data-flow. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3790–3805, 2024.

[135] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 2024.

[136] Z. Wang, L. Zhang, C. Cao, N. Luo, X. Luo, and P. Liu. How does naming affect llms on code analysis tasks? *arXiv preprint arXiv:2307.12488*, 2023.

[137] F. Wei, Y. Li, S. Roy, X. Ou, and W. Zhou. Deep ground truth analysis of current android malware. In M. Polychronakis and M. Meier, editors, *Detection of Intrusions and Malware, and Vulnerability Assessment - 14th International Conference, DIMVA 2017, Bonn, Germany, July 6-7, 2017, Proceedings*, volume 10327 of *Lecture Notes in Computer Science*, pages 252–276. Springer, 2017.

[138] F. Wei, X. Lin, X. Ou, T. Chen, and X. Zhang. JN-SAF: precise and efficient ndk/jni-aware inter-language static analysis framework for security vetting of android applications with native code. In D. Lie, M. Mannan, M. Backes, and X. Wang, editors, *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, pages 1137–1150. ACM, 2018.

[139] W. Weimer and G. C. Necula. Mining temporal specifications for error detection. In *Tools and Algorithms for the Construction and Analysis of Systems: 11th International Conference, TACAS 2005, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2005, Edinburgh, UK, April 4-8, 2005. Proceedings 11*, pages 461–476. Springer, 2005.

[140] M. Weiser. Program slicing. In *ICSE '81*, pages 439–449. IEEE Press, 1981.

[141] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, H. Li, S.-C. Cheung, and C. Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. *arXiv preprint arXiv:2404.00762*, 2024.

[142] C. Wen, H. Wang, Y. Li, S. Qin, Y. Liu, Z. Xu, H. Chen, X. Xie, G. Pu, and T. Liu. Memlock: Memory usage guided fuzzing. In *Proceedings of the*

ACM/IEEE 42nd International Conference on Software Engineering, pages 765–777, 2020.

[143] G. Wu, W. Cao, Y. Yao, H. Wei, T. Chen, and X. Ma. Llm meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 406–417, 2024.

[144] H. Xu, W. Ma, T. Zhou, Y. Zhao, K. Chen, Q. Hu, Y. Liu, and H. Wang. Ckgfuzzer: Llm-based fuzz driver generation enhanced by code knowledge graph. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 243–254. IEEE, 2025.

[145] Y. Xu, S. Huang, M. Geng, Y. Wan, X. Shi, and D. Chen. code_transformed: The influence of large language models on code. *arXiv preprint arXiv:2506.12014*, 2025.

[146] M. Zhang, Y. Tian, Z. Xu, Y. Dong, S. H. Tan, and C. Sun. Lpr: Large language models-aided program reduction.

[147] J. Zhao, M. G. Burke, and V. Sarkar. Parallel sparse flow-sensitive points-to analysis. In *Proceedings of the 27th International Conference on Compiler Construction*, pages 59–70, 2018.

[148] H. Zhu, T. Dillig, and I. Dillig. Automated inference of library specifications for source-sink property verification. In C. Shan, editor, *Programming Languages and Systems - 11th Asian Symposium, APLAS 2013, Melbourne, VIC, Australia, December 9-11, 2013. Proceedings*, volume 8301 of *Lecture Notes in Computer Science*, pages 290–306, Cham, 2013. Springer.